

UNIVERSITÉ HASSAN II DE CASABLANCA

FACULTÉ DES SCIENCES BEN M'SIK

MASTER EN DATA SCIENCE ET BIG DATA

L'Intelligence Artificielle au Service de l'Automatisation :

CONCEPTION ET MISE EN OEUVRE D'AGENTS IA POUR L'AMÉLIORATION DE L'EFFICACITÉ

Pour l'obtention du diplôme de Master en Data Science et Big Data

Présenté par

Ouirra SALAH EDDINE

Devant le Jury

Président :	Pr. Abdelaziz MARZAK
Encadrante :	Pr. Rachida AIT ABDELOUAHID
Examinatrice :	Pr. Soukaina MERZOUK

Encadrants techniques

Président :	M. Safouan CHALOUAH
Encadrant :	M. Youssef DIOURI

20 Septembre 2025

Dédicace

Je dédie ce travail à...

À mes parents pour leur soutien inconditionnel et leur amour.

À mes collègues et amis pour leur aide et leur support précieux tout au long de ce projet.

À l'équipe **OFFZONE** pour leur collaboration et leur confiance en moi.

À toutes les personnes qui ont cru en moi et m'ont encouragé à donner le meilleur de moi-même.

À tous ceux qui ont contribué de près ou de loin à la réussite de ce projet.

Je leur exprime ma gratitude et ma reconnaissance pour leur soutien et leur confiance. Ce travail est le fruit de notre engagement et de notre détermination.

Je suis fier de ce que nous avons accompli et reconnaissant pour l'opportunité de travailler sur ce projet. Puisse ce travail inspirer et motiver les futures générations d'ingénieurs et de chercheurs.

Je remercie également l'entreprise pour la confiance qu'elle m'a accordée et pour les ressources mises à ma disposition.

À tous, je vous remercie de votre soutien et de votre présence à mes côtés. Je suis honoré de partager ce travail avec vous.

Remerciements

Au terme de ce travail, je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, à sa réussite.

Je souhaite remercier tout particulièrement mon encadrante Madame la Professeure **Rachida AIT ABDELOUAHID** pour son encadrement précieux, sa patience, sa disponibilité et ses conseils avisés qui ont guidé ma réflexion tout au long de ce projet.

Mes remerciements s'adressent également aux membres du jury, Monsieur le Professeur **Abdelaziz MARZAK** et Madame la Professeure **Soukaina MERZOUK**, pour l'honneur qu'ils me font en acceptant d'évaluer ce travail et pour le temps qu'ils y consacrent.

Je remercie mes collègues pour leur collaboration et leurs conseils précieux.

Je suis reconnaissant envers mon entreprise, OFFZONE, pour les ressources et les opportunités qu'elle m'a offertes.

Je remercie mes amis et ma famille pour leur soutien et leur encouragement constants.

Je suis reconnaissant envers tous ceux qui ont lu et commenté mon travail, contribuant ainsi à son amélioration.

Je remercie les développeurs et les chercheurs qui ont créé les outils et les technologies que j'ai utilisés.

Je remercie mon institution pour les ressources et les installations qu'elle m'a offertes.

Enfin, je remercie tous ceux qui liront ce rapport et contribueront à l'avancement de ce domaine. Je suis honoré d'avoir pu travailler sur ce projet et je suis reconnaissant pour l'expérience acquise.

Résumé

L'objectif principal de ce projet était de concevoir et de développer des agents intelligents capables d'automatiser des tâches répétitives et d'améliorer la scalabilité ainsi que les opérations de l'entreprise en utilisant des modèles LLM locaux.

Pour atteindre cet objectif, j'ai développé plusieurs agents spécialisés, en commençant par un agent **Slack** qui suit la présence des collaborateurs et valide leurs messages quotidiens à l'aide d'un modèle de langage large (LLM) pour déterminer leur validité, ainsi qu'un agent **Zendesk** qui gère les tickets et suggère des réponses en fonction d'une base de connaissances (RAG).

Un agent **commercial** a également été conçu pour trouver des clients potentiels et les qualifier pour une connexion automatique, ainsi que des agents capables d'envoyer des messages de confirmation de commande et de suivi d'expédition via **WhatsApp** et par appel téléphonique, en s'intégrant aux API de diverses plateformes de commerce électronique telles que **Shopify**, **WooCommerce** et **Jumia**.

Pour centraliser la gestion de ces agents, j'ai mis en place un système basé sur **Redis**, **MongoDB**, **FastAPI** et **Next.js**, permettant aux utilisateurs non techniques de visualiser les exécutions et les informations des agents.

Ce travail vise à optimiser les processus internes, à réduire les tâches manuelles et à améliorer les opérations globales de l'entreprise.

Abstract

This project presents the design and development of intelligent agents for automating repetitive tasks in a Software as a Service (SaaS) company, leveraging artificial intelligence and machine learning technologies.

We created specialized agents to track collaborator presence, manage tickets, find potential leads, connect user information, and send order notifications. These agents are integrated using technologies such as **n8n**, **Slack API**, **Zendesk API**, **FastAPI**, **Next.js**, **Redis**, and **MongoDB**.

Our approach utilizes **Large Language Models (LLMs)** for message validation and **Retrieval-Augmented Generation (RAG)** for ticket management. The system is designed to be user-friendly for non-technical users, enabling them to view agent executions and information through a centralized interface.

The aim of this project is to improve company efficiency, reduce manual labor, and optimize internal processes. It demonstrates the potential of intelligent agents in enhancing SaaS company operations, with a focus on scalability and reliability. Our solution is designed to adapt to evolving business needs : by harnessing the power of AI, workflows can be streamlined and business growth accelerated.

The agents are designed to work seamlessly together, providing a comprehensive automation solution and demonstrating the effectiveness of intelligent agents in automating tasks and improving business outcomes.

Table des matières

LISTE DES ABRÉVIATIONS	15
Introduction Générale	16
1 CONTEXTE GÉNÉRAL ET POSITIONNEMENT DU PROJET	18
Contexte Général et Positionnement du Projet	18
1.1 Contexte Stratégique et Justification Scientifique	18
1.1.1 Le Virage Paradigmatique vers l’Automatisation Intelligente	18
1.1.2 Cadre Technologique et Méthodologique	19
1.2 Problématique Scientifique et Enjeux Opérationnels	20
1.2.1 Identification des Points de Friction Opérationnels	20
1.2.2 Les Défis Techniques et Scientifiques	21
1.3 Objectifs et Portée du Projet	21
1.3.1 Finalité du Projet	21
1.3.2 Objectifs Spécifiques et Mesurables	21
1.3.3 Résultats Attendus et Indicateurs de Succès	22
2 Cadre Opérationnel et Stratégique : L’Entreprise d’Accueil OFFZONE	24
Cadre Opérationnel et Stratégique : L’Entreprise d’Accueil OFFZONE	24
2.1 Présentation d’OFFZONE	24
2.1.1 Mission et Positionnement Stratégique	25
2.1.2 Un Acteur Engagé dans l’Écosystème Numérique	25
2.2 Pôles d’Expertise et Environnement Technologique	25

2.2.1	Pôles d'Expertise	26
2.2.2	Environnement Technologique	26
2.3	Méthodologie Agile et Cadre de Travail Scrum	27
2.3.1	La Méthodologie Agile et le Cadre Scrum	27
2.3.2	Les réunions Hebdomadaires de Synchronisation et de Planification . .	27
2.3.3	le "Compte Rendu" Quotidien	28
2.4	Présence Géographique	29
3	Fondements Théoriques et Concepts Clés	31
	Fondements Théoriques et Concepts Clés	31
3.1	Le Paradigme des Systèmes Multi-Agents (SMA)	31
3.1.1	Définition et Principes Fondamentaux	31
3.1.2	Propriétés et Avantages Architecturaux	32
3.1.3	Topologies d'Interaction : Les Architectures de SMA	33
3.1.4	Le Cycle Opérationnel d'un Agent : Le Paradigme Perception-Délibération- Action	34
3.2	RAG : La Synergie entre Récupération d'Information et Génération	35
3.2.1	Définition et Principe Fondamental	35
3.2.2	Fonctionnement Détaillé	35
3.2.3	Avantages et Rôle Stratégique dans les SMA	36
4	État de l'Art des Technologies Fondamentales	38
	État de l'Art des Technologies Fondamentales	38
4.1	Les Grands Modèles de Langage (LLM)	38
4.1.1	Définition et Rupture Architecturale du Transformer	38
4.1.2	Les Modèles Open Source : Le Compromis Performance-Coût	39
4.1.3	Formalisation des Critères de Sélection et Modèle Retenu	40
4.2	L'Ingénierie de Prompts (Prompt Engineering)	41
4.2.1	Définition et Rôle Stratégique	41

4.2.2	Types Fondamentales	42
4.3	Les Plongements Sémantiques (Embeddings)	43
4.3.1	Définition : De la Donnée au Vecteur	43
4.3.2	Applications et Avantages dans le Projet	43
5	Analyse et Conception du Système	45
	Analyse et Conception du Système	45
5.1	Spécifications et Exigences du Système	45
5.1.1	Exigences Fonctionnelles	46
5.1.2	Exigences Non-Fonctionnelles	46
5.2	Architecture Générale du Système	47
5.3	Les serveurs Model Context Protocol (MCP)	48
5.3.1	Principe et Rôle du MCP	49
5.3.2	Communication Inter-Agents au sein de n8n	49
5.3.3	Interaction avec les Serveurs Publics	49
5.4	Analyse des Risques et Défis Techniques	50
5.5	Positionnement par Rapport aux Approches Existantes	50
6	Architecture Technologique et Stack d'Implémentation	52
	Architecture Technologique et Stack d'Implémentation	52
6.1	Fondations de l'Intelligence Artificielle et Orchestration	52
6.1.1	n8n [10]	53
6.1.2	CrewAI [11]	54
6.2	Couche de Service et Exposition (Back-End)	54
6.2.1	FastAPI [12]	54
6.3	Interface Utilisateur (Front-End)	55
6.3.1	Next.js [13]	55
6.4	Gestion des Données et Persistance	56
6.4.1	MongoDB	56

6.4.2	Qdrant [14]	56
6.4.3	Redis	57
6.5	Intégration de Services Tiers et Canaux de Communication	59
6.5.1	WhatsApp Business Cloud API	59
6.5.2	Retell AI	59
6.6	Industrialisation et Déploiement	60
6.6.1	Docker	60
7	Implémentation et Cas d'Usage	62
	Implémentation et Cas d'Usage	62
7.1	Conteneurisation avec Docker	62
7.1.1	n8n AI Kit Auto-Hébergé	63
7.1.2	Personnalisation de l'Image n8n pour les Besoins Spécifiques	64
7.1.3	Gestion Centralisée des Secrets et Configurations	65
7.1.4	Exposition Sécurisée pour le Développement avec ngrok	67
7.2	Agent de Supervision	68
7.2.1	Fonctionnalités Principales	70
7.2.2	Avantages de l'Agent de Supervision	70
7.2.3	Exemples de Notifications Automatisées	71
7.2.4	Orchestration du Flux de Travail avec n8n	72
7.2.5	Cœur du Traitement : Le Nœud de l'Agent IA	75
7.3	Agent d'Assistance à la Gestion des Tickets de Support	78
7.3.1	Architecture et Principe de Fonctionnement	78
7.3.2	Orchestration n8n	80
7.3.3	Constitution de la Base de Connaissances par Apprentissage Rétrospectif	82
7.3.4	Résultats en Pratique	84
7.4	Agent de Prospection Commerciale	86
7.4.1	Architecture Globale du Processus de Prospection	86
7.4.2	Phase 1 : Identification des Cibles via l'API Google Search	87

7.4.3	Phase 2 : Enrichissement et Qualification des Données	89
7.4.4	Phase 3 : Séquence d'Engagement Automatisé via l'Automatisation du Navigateur	90
7.4.5	Orchestration du Workflow avec n8n	92
7.5	Agents Spécialisés pour le E-Commerce	94
7.5.1	Agent de Confirmation Proactive des Commandes	94
7.5.2	Agent de Suivi Actif des Livraisons	98
7.6	Plateforme de Gestion Centralisée	101
7.6.1	Architecture Fonctionnelle et Cas d'Utilisation	102
7.6.2	Gestion des Utilisateurs et des Permissions (Accès Admin)	102
7.6.3	Tableau de Bord : Suivi des Performances et des Erreurs	104
7.6.4	Supervision des Agents en Temps Réel et Historique	106
7.6.5	Interaction Conversationnelle avec les Agents	108
7.7	Défis Rencontrés et Solutions Apportées	109
7.7.1	Tests et Validation	109
7.7.2	Optimisation des Performances	110
Conclusion Générale et Perspectives		111
7.8	Bilan du Projet et Synthèse des Apports	111
7.9	Perspectives d'Évolution et Travaux Futurs	111
7.9.1	Court Terme : Consolidation et Améliorations Incrémentales	111
7.9.2	Moyen Terme : Extensions Stratégiques	112
7.9.3	Long Terme : Visions d'Avenir et Potentiel de Rupture	112

Table des figures

2.1	Logo d'OFFZONE	24
2.2	Logo de la plateforme de visioconférence Google Meet.	28
2.3	Logo de la plateforme de messagerie collaborative Slack.	28
2.4	Les pays où OFFZONE est présent	30
3.1	Schéma conceptuel d'un Système Multi-Agents, illustrant les interactions entre agents autonomes et leur environnement commun.	32
3.2	Workflow du modèle RAG, de la requête initiale à la réponse finale augmentée.	36
4.1	Schéma simplifié de l'architecture Transformer, mettant en évidence les mécanismes d'attention et les couches feed-forward.	39
4.2	types des prompts pour un LLM	42
4.3	Fonctionnement d'Embedding	43
5.1	Diagramme de l'architecture en quatre couches du système multi-agents.	47
5.2	diagramme de fonctionnement des serveurs MCP	48
6.1	Logo de n8n	53
6.2	Logo de crewai	54
6.3	Logo de FastApi	54
6.4	Logo de NextJs	55
6.5	Logo de MongoDB.	56
6.6	Logo de Qdrant	56
6.7	Logo de Redis.	57
6.8	Modèle redis Pub/Sub	58
6.9	Schéma API Whatsapp	59

6.10	Logo de retell Ai	59
6.11	Logo de Docker	60
7.1	Conteneur Docker avec tout l'environnement n8n	63
7.2	Logo de l'outil ngrok.	67
7.3	Schéma fonctionnel de l'Agent de Supervision Opérationnelle et ses interactions avec Slack, le LLM de validation et Google Sheets.	69
7.4	Exemple de journalisation des statuts dans Google Sheets, automatiquement mise à jour par l'agent.	71
7.5	Exemples de messages automatisés envoyés par l'agent sur Slack.	72
7.6	Vue d'ensemble du workflow n8n pour l'agent de supervision : remplissage des feuilles de document Google avec les noms des participants Slack	73
7.7	Vue d'ensemble du workflow n8n pour l'agent de supervision : Récupération des nouveaux messages envoyées par chaine Slack	73
7.8	Vue d'ensemble du workflow n8n pour l'agent de supervision : Validation des messages et marquage d'absence	74
7.9	Architecture du Nœud de l'Agent IA, montrant l'interaction entre le message entrant, le cerveau LLM et le stockage local.	76
7.10	Configuration LLM pour l'agent de supervision	77
7.11	Diagramme fonctionnel de l'agent de support client.	79
7.12	Workflow n8n pour l'agent de support : Remplissage de la base des bases connaissances	80
7.13	Workflow n8n pour l'agent de support : Obtention des tickets depuis webhook , classification et génération de note interne	81
7.14	Utilisation du base des connaissances pour répondre aux questions des agents humains	81
7.15	Processus de création de la base de connaissances vectorielle.	83
7.16	Extrait de la base de connaissances structurée au format tabulaire, montrant les paires "Question/Réponse" extraites des tickets historiques	85

7.17	Exemple d'un ticket traité par l'agent dans Zendesk. On observe le tag "Hardware" ajouté automatiquement (1) et la note interne avec la suggestion de réponse pour l'agent (2).	85
7.18	Diagramme fonctionnel de l'agent de prospection commerciale automatisée. . .	87
7.19	Interface de configuration des campagnes pour l'agent Commercial.	89
7.20	Extrait du workflow de scraping pour l'enrichissement des données de profil de l'entreprise.	90
7.21	Vue d'ensemble du workflow n8n pour l'agent de prospection : Lancement du Campagne de recherche depuis formulaire , déclencheur Ou message Utilisateur .	93
7.22	Vue d'ensemble du workflow n8n pour l'agent de prospection : Log des résultats de Google API et suppression des doublons	93
7.23	Vue d'ensemble du workflow n8n pour l'agent de prospection : Qualification des profils et enregistrement dans Document Google.	93
7.24	Diagramme fonctionnel de l'agent de confirmation de commande.	95
7.25	Journal de suivi des confirmations sur Google Sheets, avec le statut de chaque commande.	96
7.26	Workflow n8n pour le processus de confirmation de commande.	96
7.27	Exemple de message de confirmation envoyé via WhatsApp.	97
7.28	Exemple de message de relance envoyé via WhatsApp.	97
7.29	Diagramme fonctionnel de l'agent de suivi de livraison.	98
7.30	Vue du workflow n8n pour l'agent de suivi de livraison.	100
7.31	Exemples de notifications de suivi envoyées au client.	100
7.32	Extrait du rapport de suivi sur Google Sheets	101
7.33	Diagramme de cas d'utilisation de la plateforme de gestion.	102
7.34	Tableau de Bord Admin	102
7.35	Fenêtre changement des accès utilisateurs	103
7.36	Page d'authentification au système	104
7.37	Bar Graph pour l'historique des exécutions	104
7.38	Line Graph pour le taux des données récupérées par l'agent	105

7.39	Pie Graph pour le suivi des vulnérabilités	105
7.40	Vue du tableau de bord principal	106
7.41	Log des absences directement dans la plateforme	106
7.42	Données suivi en temps réel avec Redis	107
7.43	Histoire des données récupérées par l'agent	107
7.44	Controle sur les noeuds des workflows n8n depuis l'application	108
7.45	Exemple d'interaction avec l'agent commercial via l'interface de chat.	109

Liste des tableaux

4.1	Benchmark comparatif de LLMs open-source pour une utilisation dans un système multi-agents.	40
7.1	Paramètres de configuration du Nœud de l'Agent IA.	76

LISTE DES ABRÉVIATIONS

API	Application Programming Interface (<i>Interface de programmation d'application</i>)
AI	Artificial Intelligence (<i>Intelligence artificielle</i>)
LLM	Large Language Model (<i>Modèle de langage de grande taille</i>)
SaaS	Software as a Service (<i>Logiciel en tant que service</i>)
CRM	Customer Relationship Management (<i>Gestion de la relation client</i>)
RAG	Retrieval-Augmented Generation (<i>Génération augmentée par récupération</i>)
Redis	Remote Dictionary Server (<i>Serveur de dictionnaire distant</i>)
MongoDB	Base de données NoSQL orientée documents
FastAPI	Framework Python pour la création d'API performantes
Next.js	Framework JavaScript pour la création d'applications web modernes

Introduction Générale

Dans un environnement socio-économique caractérisé par une **mutation technologique et concurrentielle sans précédent**, les organisations sont confrontées à un impératif de modernisation structurelle. Afin de préserver leur niveau et leur avantage concurrentiel, une réingénierie des processus opérationnels est devenue une condition sine qua non. C'est dans ce contexte que **l'intelligence artificielle (IA) et l'automatisation des processus** émergent non plus comme de simples leviers d'optimisation, mais comme des fondements stratégiques pour la performance. Elles permettent de fluidifier les opérations, de maîtriser les coûts et d'augmenter la pertinence de la prise de décision.

L'entreprise **OFFZONE**, société de services du numérique (ESN) spécialisée dans l'accompagnement à la transformation digitale, se positionne à l'avant-garde de cette révolution. En proposant un portefeuille de solutions intégrées alliant design d'expérience, cybersécurité et ingénierie logicielle, elle permet à ses clients de naviguer avec succès les complexités d'un marché en perpétuelle évolution.

Le présent travail, mené dans le cadre d'un stage de fin d'études au sein de cette entreprise, s'inscrit directement dans cette démarche d'innovation. Notre contribution a consisté à explorer, concevoir et implémenter des solutions d'automatisation intelligentes afin de répondre à des problématiques métiers concrètes.

La mission qui m'a été confiée s'est articulée autour de la conception et du déploiement d'un ensemble **d'agents logiciels autonomes et de workflows automatisés**. Ces derniers ont été appliqués à plusieurs cas d'usage critiques pour l'entreprise, servant de preuves de concept à la fois fonctionnelles et quantifiables :

- **La gestion des ressources humaines**, avec le suivi automatisé de la présence des col-

laborateurs via l'analyse sémantique des interactions sur des plateformes collaboratives comme Slack.

- **L'optimisation du support client**, par la classification intelligente des tickets et l'assistance à la résolution basée sur une base de connaissances.
- **La fiabilisation de la chaîne logistique e-commerce**, via une gestion proactive des commandes rendue possible par l'intégration d'API tierces.

L'objectif principal de ce projet est de valider empiriquement l'hypothèse selon laquelle l'intégration de technologies de pointe, notamment les **grands modèles de langage (LLMs)** et les frameworks d'orchestration comme **LangChain** [9] ou **CrewAI** [11], peut générer un retour sur investissement mesurable. Il s'agit de démontrer leur pertinence et leur viabilité pour transformer en profondeur les processus métiers, avec pour finalité une amélioration quantifiable de la productivité et de la qualité des services internes et externes.

Pour ce faire, ce rapport s'articulera en plusieurs chapitres. Nous commencerons par définir le **contexte général** du projet et la problématique. Nous établirons ensuite un **état de l'art** des technologies et concepts clés mobilisés. Les chapitres suivants détailleront la démarche **d'analyse et de conception** de notre système, avant de présenter les **outils et frameworks** spécifiques utilisés. Nous conclurons par une présentation de **l'implémentation**, des cas d'usage et des défis rencontrés, avant d'ouvrir sur les **perspectives** et les travaux futurs.

Chapitre 1

CONTEXTE GÉNÉRAL ET POSITIONNEMENT DU PROJET

Introduction

Ce chapitre a pour vocation de poser les fondements scientifiques et opérationnels de notre projet. Il s'agit d'établir le cadre dans lequel s'inscrit cette étude, en partant d'une analyse du contexte stratégique qui motive le recours à l'automatisation intelligente, pour aboutir à une formulation précise de la problématique à résoudre. Nous définirons ensuite les objectifs spécifiques et mesurables qui guideront notre démarche de conception et de développement, ainsi que les retombées attendues. L'ensemble de ce chapitre constitue le socle conceptuel sur lequel repose la pertinence de notre contribution.

1.1 Contexte Stratégique et Justification Scientifique

1.1.1 Le Virage Paradigmatique vers l'Automatisation Intelligente

Le développement d'agents intelligents s'inscrit dans un mouvement de fond où les entreprises, particulièrement celles du secteur **SaaS (Software as a Service)**, font face à une rupture paradigmatique. La croissance exponentielle de ce secteur engendre des défis d'échelle (*scalability*) qui ne peuvent plus être adressés par une simple augmentation des ressources humaines.

Le paradigme traditionnel de la croissance linéaire (plus de clients = plus de personnel) atteint ses limites et devient un inhibiteur de performance.

Ces organisations sont confrontées à une triple problématique structurelle :

- **L'inflation des Données Non Structurées** : Chaque interaction client, ticket de support ou transaction génère un volume exponentiel de données précieuses, mais dont l'analyse manuelle est économiquement et techniquement prohibitive.
- **L'Impératif de Réactivité et de Personnalisation** : Dans un marché hyper-concurrentiel, la vitesse et la pertinence de la réponse client sont des différenciateurs stratégiques. La fidélisation dépend désormais de la capacité à offrir une expérience personnalisée et instantanée.
- **La Complexité Croissante des Workflows Internes** : La gestion du cycle de vie client, de l'acquisition à la rétention, implique des processus interconnectés dont la complexité augmente de manière non linéaire avec la croissance, créant des goulots d'étranglement et des points de friction.

Face à ces enjeux, l'automatisation par des agents intelligents n'est plus une option, mais une nécessité stratégique. Ces entités logicielles autonomes, capables d'apprendre, de raisonner et de s'adapter, représentent la solution privilégiée pour passer d'une logique opérationnelle réactive à un modèle proactif et scalable.

1.1.2 Cadre Technologique et Méthodologique

Notre projet vise à développer une nouvelle génération d'agents capables d'automatiser des processus à forte valeur cognitive, tels que la **gestion du support client**, la **qualification des prospects** ou la **communication transactionnelle**. La méthodologie adoptée repose sur des technologies de pointe pour dépasser les limites des systèmes d'automatisation traditionnels :

- **Les Grands Modèles de Langage (LLM)** : Ces modèles sont mobilisés pour leur capacité à comprendre et à générer le langage naturel avec une finesse inégalée, permettant une analyse sémantique profonde des requêtes clients.

- **La Génération Augmentée par Récupération (RAG)** : Cette technique est mise en œuvre pour ancrer les réponses des LLMs dans une base de connaissances factuelle et à jour. Elle permet de garantir la fiabilité, la précision et la traçabilité des informations générées, mitigant ainsi les risques d'hallucination.

L'intégration de ces agents au cœur des plateformes opérationnelles (**Slack, Zendesk, Shopify**) vise à démontrer empiriquement leur potentiel de création d'un avantage concurrentiel durable.

1.2 Problématique Scientifique et Enjeux Opérationnels

La problématique centrale de ce travail peut être formulée comme suit : **comment concevoir une architecture d'agents intelligents, basée sur des LLMs, capable d'automatiser des processus métier complexes dans un environnement SaaS, tout en satisfaisant des exigences strictes de précision, de sécurité et d'efficacité ?**

1.2.1 Identification des Points de Friction Opérationnels

Les processus manuels actuels se manifestent par plusieurs points de friction dont l'impact cumulé dégrade la performance globale et la satisfaction client :

- **L'Érosion de la Fidélité Client** : La gestion manuelle des tickets de support engendre des délais de réponse élevés et des erreurs de diagnostic, ce qui augmente le taux d'attrition (*churn*).
- **La Perte d'Opportunités Commerciales** : Une qualification manuelle des prospects, lente et subjective, se traduit par un allongement du cycle de vente et des opportunités manquées.
- **Les Risques Opérationnels** : La gestion manuelle des communications sur de multiples canaux est chronophage et augmente le risque d'erreurs humaines (oubli, mauvaise information) pouvant avoir un impact légal ou financier.

- **La Prise de Décision Suboptimale** : L'absence d'analyse en temps réel contraint les équipes à des prises de décision réactives, basées sur l'intuition plutôt que sur des données consolidées.

1.2.2 Les Défis Techniques et Scientifiques

La résolution de cette problématique soulève des défis majeurs :

- **Interopérabilité et Fusion de Données Hétérogènes** : Intégrer et normaliser des données non structurées (texte libre) et structurées provenant de sources multiples est un enjeu fondamental.
- **Conception d'Agents Apprenants** : Développer des agents capables non seulement d'exécuter des tâches, mais aussi de s'adapter aux évolutions des processus métiers et des comportements utilisateurs.
- **Sécurité et Confidentialité des Données ("AI Safety")** : Garantir la protection des données sensibles (RGPD) tout en permettant aux agents d'y accéder de manière contrôlée et éthique.
- **Architecture Robuste et Scalable** : Mettre en place une infrastructure logicielle capable de supporter la charge de multiples agents opérant en parallèle.

1.3 Objectifs et Portée du Projet

1.3.1 Finalité du Projet

L'objectif principal est de **concevoir, développer et valider un système multi-agents capable d'automatiser des processus à forte intensité manuelle**, dans le but d'induire une amélioration quantifiable de l'efficacité opérationnelle, de la productivité et de la qualité de service.

1.3.2 Objectifs Spécifiques et Mesurables

Pour atteindre cette finalité, les objectifs spécifiques suivants sont définis :

- Concevoir et valider un agent autonome pour le tri, la catégorisation et l'assistance à la résolution des tickets de support.
- Implémenter et évaluer un workflow automatisé pour la qualification des leads commerciaux, de la détection à l'assignation.
- Développer un système de gestion automatisée des notifications transactionnelles cross-canal.
- Mettre en place des mécanismes d'analyse en temps réel pour la prise de décision assistée par l'IA.
- Quantifier la réduction des coûts opérationnels directs et indirects liés aux tâches ciblées.

1.3.3 Résultats Attendus et Indicateurs de Succès

Les retombées attendues sont à la fois quantitatives et qualitatives :

- Une augmentation mesurable de la productivité (ex : nombre de tickets traités par heure).
- Une réduction significative des temps de cycle (ex : temps moyen de qualification d'un lead).
- Une amélioration des indicateurs de satisfaction client (CSAT, Net Promoter Score).
- Une réallocation stratégique des ressources humaines vers des tâches à plus haute valeur ajoutée.
- La constitution d'un socle technologique et méthodologique pour de futures innovations en IA au sein de l'entreprise.

Conclusion

Ce chapitre a permis de délimiter le périmètre de notre étude. En partant d'un contexte économique et technologique global, nous avons mis en lumière les défis spécifiques auxquels les

entreprises SaaS sont confrontées, justifiant ainsi la pertinence d’une approche par automatisation intelligente. La problématique a été clairement formulée, en articulant les enjeux opérationnels et les verrous scientifiques à lever. Enfin, nous avons défini un ensemble d’objectifs rigoureux et de résultats attendus qui serviront de fil conducteur et de grille d’évaluation pour la suite de notre travail. Ayant ainsi posé le cadre conceptuel et les ambitions de notre projet, le chapitre suivant se consacrera à l’étude de l’état de l’art des technologies mobilisées pour atteindre ces objectifs.

Chapitre 2

Cadre Opérationnel et Stratégique : L'Entreprise d'Accueil OFFZONE

Introduction

Ce chapitre a pour objectif de présenter l'organisation d'accueil, l'entreprise **OFFZONE**, au sein de laquelle ce projet de recherche et développement a été mené. Une compréhension approfondie de son positionnement stratégique, de sa mission, de son portefeuille de compétences et de son environnement technologique est indispensable pour contextualiser la problématique abordée et la pertinence de la solution proposée. Nous décrirons donc successivement l'identité de l'entreprise, ses domaines d'expertise, et enfin, son implantation géographique, afin d'établir le cadre opérationnel qui a servi de terrain d'expérimentation pour notre travail.

2.1 Présentation d'OFFZONE



FIG. 2.1 – Logo d'OFFZONE

2.1.1 Mission et Positionnement Stratégique

OFFZONE est une Entreprise de Services du Numérique (ESN) marocaine et un cabinet de conseil spécialisé dans l'ingénierie de la **transformation digitale**. Sa proposition de valeur fondamentale est de se positionner en tant que partenaire stratégique pour les organisations qui aspirent à naviguer avec succès la complexité de l'économie numérique. Dans un marché caractérisé par une hyper-croissance et une évolution technologique rapide, OFFZONE se différencie par une approche holistique qui orchestre trois piliers fondamentaux et interdépendants : le **design d'expérience (UX/UI)**, l'intégration de **technologies de pointe**, et la **cybersécurité** par conception (*security by design*).

La mission de l'entreprise est de réduire la complexité inhérente aux projets digitaux. Pour ce faire, OFFZONE s'engage à fournir des solutions sur mesure dont la performance ne se mesure pas seulement en termes techniques, mais aussi en termes d'accessibilité et d'intuitivité. Une emphase particulière est mise sur les principes de *Software Craftsmanship*, garantissant une haute qualité logicielle et une expérience utilisateur optimisée, afin de livrer des produits à forte valeur ajoutée.

2.1.2 Un Acteur Engagé dans l'Écosystème Numérique

Au-delà de son statut de prestataire de services, OFFZONE assume un rôle actif au sein de la communauté **Digital Morocco**. Ce réseau fédère les principaux acteurs de l'écosystème numérique marocain — startups, institutions, investisseurs et grands comptes. Cet engagement stratégique n'est pas anecdotique ; il permet à OFFZONE de maintenir un capital de connaissances de premier plan via une veille technologique constante et de contribuer activement à la maturation du marché digital national en favorisant le partage d'expertise.

2.2 Pôles d'Expertise et Environnement Technologique

La stratégie d'OFFZONE est soutenue par un portefeuille de compétences techniques couvrant une large partie de la chaîne de valeur numérique.

2.2.1 Pôles d'Expertise

L'offre de services est structurée autour de plusieurs pôles d'excellence :

- **Conception et Expérience Utilisateur (UI/UX)** : Ce pôle est chargé de traduire les objectifs métiers en parcours utilisateurs fluides et en interfaces ergonomiques, en s'appuyant sur des outils standards de l'industrie comme **Figma** et la suite **Adobe**.
- **Développement de Solutions sur Mesure** : Il s'agit du cœur de métier de l'ingénierie logicielle, incluant la création de plateformes e-commerce, d'applications web personnalisées et de systèmes de gestion de contenu (CMS). Chaque projet est mené avec des exigences élevées en matière de sécurité, de scalabilité et de robustesse, validées par des tests rigoureux.
- **Digitalisation du Commerce** : Un accompagnement spécifique est proposé aux entreprises marocaines pour leur transition vers le commerce en ligne, intégrant des solutions de paiement sécurisé, de logistique et de marketing digital.
- **Intégration de Systèmes ERP** : OFFZONE déploie des progiciels de gestion intégrés (**SAP, Odoo, Microsoft Dynamics**) pour optimiser et unifier les processus critiques de l'entreprise (finance, achats, ressources humaines, chaîne d'approvisionnement).

2.2.2 Environnement Technologique

L'expertise technique d'OFFZONE repose sur un stack technologique agnostique et moderne, lui permettant de sélectionner l'outil le plus adapté à chaque problématique client.

- **Technologies Back-End** : Java, Node.js, Django (Python)
- **Technologies Front-End** : React, Angular
- **Outils de Design** : Figma, Adobe Creative Suite
- **Solutions ERP** : SAP, Odoo, Microsoft Dynamics

2.3 Méthodologie Agile et Cadre de Travail Scrum

Afin d'assurer une gestion de projet flexible, réactive et alignée avec les objectifs stratégiques de l'entreprise, notre travail s'est inscrit dans un cadre méthodologique Agile, inspiré des principes de **Scrum**.

2.3.1 La Méthodologie Agile et le Cadre Scrum

La **méthodologie Agile** est une philosophie de gestion de projet, née en réponse aux limites des modèles traditionnels. L'Agilité valorise les individus et leurs interactions, la collaboration avec le client, la livraison de logiciels fonctionnels et l'adaptation au changement. Plutôt que de suivre un plan détaillé et figé, elle promeut un développement par cycles courts et itératifs, permettant de livrer de la valeur de manière continue et d'ajuster la trajectoire du projet en fonction des retours obtenus.

Parmi les multiples cadres d'application de la philosophie Agile, **Scrum** est l'un des plus répandus et structurés. Il ne s'agit pas d'une méthode prescriptive, mais d'un cadre au sein duquel les équipes peuvent aborder des problèmes complexes et développer des produits de manière créative et productive. Le cœur de Scrum est le **Sprint**, une période de temps fixe (généralement de une à quatre semaines) durant laquelle un incrément de produit "Fini" et potentiellement utilisable est créé.

En adoptant cette approche, OFFZONE s'est doté d'un mécanisme de pilotage qui favorise la transparence, l'inspection continue du travail et l'adaptation rapide.

2.3.2 Les réunions Hebdomadaires de Synchronisation et de Planification

Le cycle de développement était rythmé par des réunions hebdomadaires systématiques, s'apparentant aux cérémonies Scrum de revue et de planification de sprint. Ces sessions se déroulaient sur **Google Meet**, la plateforme de visioconférence de Google, choisie pour sa fiabilité et son accessibilité, permettant des échanges fluides et des partages d'écran efficaces pour les démonstrations techniques.



FIG. 2.2 – Logo de la plateforme de visioconférence Google Meet.

Ces points de synchronisation avaient plusieurs objectifs critiques :

- **Revue des Incréments** : Présenter les avancées de la semaine, effectuer des démonstrations des fonctionnalités implémentées et recueillir les retours directs des encadrants techniques et du management.
- **Discussion Stratégique** : Évaluer la trajectoire du projet par rapport à la vision globale de l'entreprise. C'était l'occasion de discuter des améliorations potentielles, de décider des fonctionnalités à ajouter, à modifier ou à déprioriser pour la semaine suivante.
- **Levée des Obstacles** : Identifier collectivement les blocages techniques ou conceptuels et définir des plans d'action pour les résoudre.

Cette boucle de rétroaction hebdomadaire garantissait que le projet restait constamment aligné sur les attentes et permettait de réorienter les efforts de manière proactive.

2.3.3 le "Compte Rendu" Quotidien

Pour assurer un suivi continu des progrès au quotidien, nous avons instauré un mode de communication asynchrone sur **Slack**. Slack, en tant que plateforme de messagerie collaborative, centralise les communications par canaux thématiques, favorisant ainsi une organisation claire des échanges.



FIG. 2.3 – Logo de la plateforme de messagerie collaborative Slack.

Chaque fin de journée, un **”compte rendu”** était publié sur un canal dédié. Ce message synthétique, équivalent d’un **”daily stand-up”** écrit, suivait une structure simple mais efficace, détaillant :

1. Les tâches accomplies durant la journée.
2. Les défis rencontrés et la manière dont ils ont été abordés.
3. Les objectifs prévus pour la journée suivante.

En synthèse, la combinaison de ces rituels hebdomadaires sur Google Meet et de la communication quotidienne structurée sur Slack a créé un environnement de travail dynamique et rigoureux. Cette méthodologie a été un facteur clé dans le projet en assurant une communication fluide, une grande capacité d’adaptation et un alignement constant avec la vision stratégique d’OFFZONE.

2.4 Présence Géographique

L’entreprise bénéficie d’une double implantation qui lui confère une passerelle stratégique entre les marchés européen et africain, lui permettant d’accompagner ses clients dans leurs ambitions internationales.

- **Maroc** : Bd Abdelkrim Al Khattabi, 40000, Marrakech
- **France** : 4 Rue Jean Moulin, 91160, Longjumeau

Conclusion

En synthèse, OFFZONE se présente comme une entreprise agile et techniquement mature, au cœur des enjeux de la transformation numérique. Son positionnement en tant que fournisseur de solutions d’automatisation et d’optimisation pour ses clients, tout en étant elle-même confrontée à des défis internes de scalabilité et d’efficacité, en fait un terrain d’expérimentation

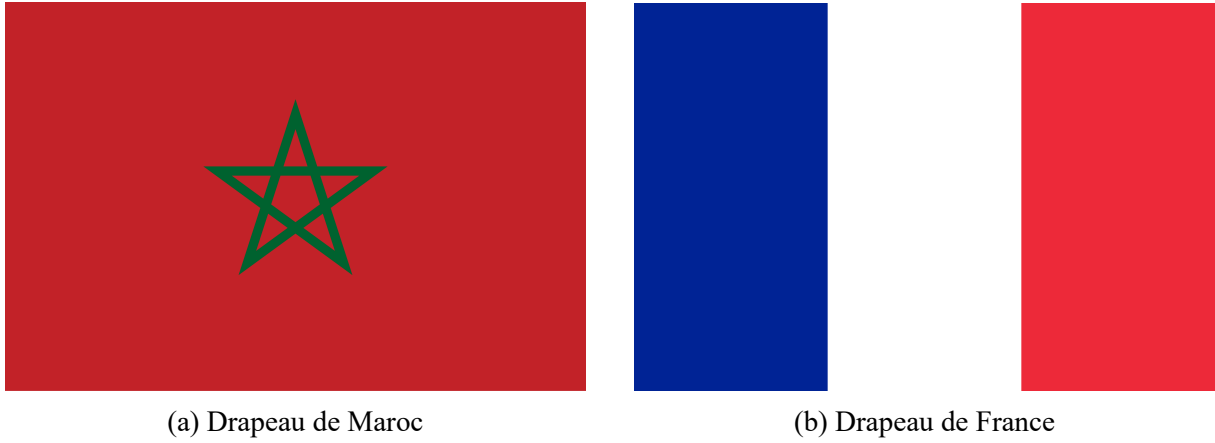


FIG. 2.4 – Les pays où OFFZONE est présent .

idéal pour notre projet. C'est dans ce contexte stimulant, alliant expertise technique et problématiques métiers réelles, que nous avons pu développer la solution d'automatisation par agents intelligents qui sera détaillée dans les chapitres suivants.

Chapitre 3

Fondements Théoriques et Concepts Clés

Introduction

Pour appréhender la conception et la portée de notre projet, il est indispensable d'en maîtriser les fondements théoriques. Ce chapitre a pour objectif de présenter les deux piliers conceptuels sur lesquels repose notre architecture : le paradigme des **Systèmes Multi-Agents (SMA)** et la technique de **Génération Augmentée par Récupération (RAG)**. La maîtrise de ces deux concepts est cruciale pour comprendre les choix de conception qui seront détaillés dans les chapitres ultérieurs.

3.1 Le Paradigme des Systèmes Multi-Agents (SMA)

3.1.1 Définition et Principes Fondamentaux

Un **Système Multi-Agents (SMA)** est un paradigme de l'intelligence artificielle et de l'informatique distribuée qui propose de modéliser un système complexe non pas comme une entité monolithique, mais comme une société d'entités informatiques autonomes appelées **agents** [3]. Chaque agent est une entité encapsulée qui possède un état interne, des capacités propres et un certain degré d'autonomie pour prendre des décisions en fonction de ses objectifs et de ses perceptions de l'environnement.

La puissance de ce paradigme ne réside pas dans la complexité d'un agent individuel, mais

dans leurs **interactions**. Plutôt que de centraliser la logique de contrôle, on la distribue. Le problème est décomposé en sous-problèmes, chacun étant assigné à un ou plusieurs agents spécialisés qui doivent communiquer, négocier, et se coordonner pour atteindre un objectif global qui dépasse la portée de chacun d'entre eux.

Schéma conceptuel d'un Système Multi-Agents

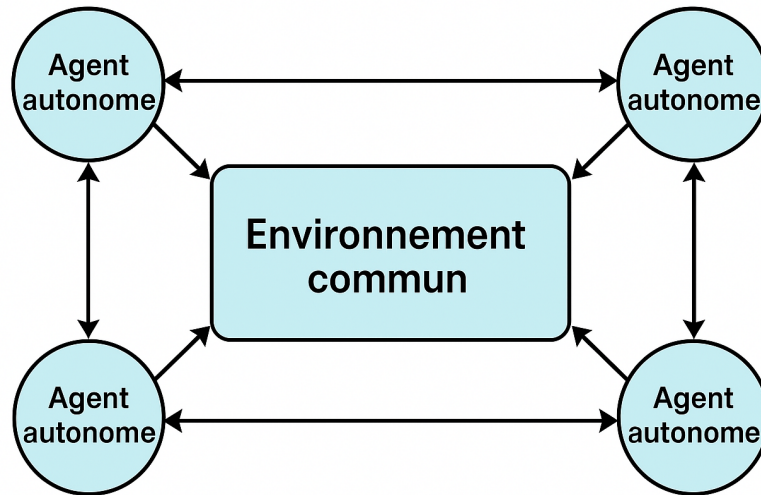


FIG. 3.1 – Schéma conceptuel d'un Système Multi-Agents, illustrant les interactions entre agents autonomes et leur environnement commun.

3.1.2 Propriétés et Avantages Architecturaux

L'adoption d'une architecture SMA confère au système des propriétés structurelles particulièrement désirables pour les environnements complexes et dynamiques :

- **Flexibilité et Adaptabilité** : L'autonomie des agents leur permet de réagir dynamiquement aux changements de l'environnement, conférant au système une grande capacité d'adaptation sans nécessiter de reconfiguration centrale.
- **Distribution et Parallélisme** : La répartition des tâches permet un traitement parallèle des problèmes, ce qui réduit les temps de calcul et élimine les goulots d'étranglement inhérents aux systèmes centralisés.

- **Résilience et Robustesse** : Un SMA ne possède généralement pas de point unique de défaillance. La panne d'un agent n'entraîne pas l'arrêt du système, qui peut continuer à fonctionner de manière dégradée mais fonctionnelle.
- **Scalabilité** : L'architecture modulaire permet d'augmenter les capacités du système par l'ajout de nouveaux agents, sans avoir à re-concevoir l'ensemble de la solution.

3.1.3 Topologies d'Interaction : Les Architectures de SMA

Le choix de l'architecture, c'est-à-dire de la topologie de communication et de contrôle entre les agents, est une décision de conception fondamentale qui conditionne le comportement global du système.

Architecture Hiérarchique (Centralisée)

Dans cette topologie, un agent **coordinateur** (ou maître) détient l'autorité, supervise l'état global et distribue les tâches aux agents subordonnés (ouvriers).

- **Avantages** : Le contrôle est centralisé, les rôles sont clairs et la conception est relativement simple à mettre en œuvre.
- **Limites** : L'agent coordinateur constitue un point unique de défaillance rendant le système vulnérable.

Architecture Horizontale (Décentralisée)

Ici, tous les agents sont des pairs et collaborent sans autorité centrale. Les décisions globales émergent de processus de négociation et de communication directe entre les agents.

- **Avantages** : L'architecture est très robuste, sans point unique de défaillance, et offre une grande flexibilité.
- **Limites** : La conception des protocoles de coordination est nettement plus complexe et peut engendrer des conflits ou des redondances.

Architectures Modernes Basées sur les LLMs

L'avènement des LLMs a favorisé l'émergence d'architectures hybrides sophistiquées, qui combinent contrôle et autonomie [5] :

- **Orchestrateur et Agents Spécialisés** : Un agent central, l'orchestrateur, agit comme un "chef de projet". Il analyse une tâche complexe, la décompose en sous-tâches, et délègue leur exécution à des agents experts (ex : un "chercheur web", un "analyste de données", un "rédacteur").
- **Agents en Boucle de Réflexion** : Inspiré des processus de décision humains, ce modèle fait interagir plusieurs agents pour débattre d'un problème. Ils critiquent mutuellement leurs propositions et convergent itérativement vers une solution plus robuste et nuancée .

3.1.4 Le Cycle Opérationnel d'un Agent : Le Paradigme Perception-Délibération-Action

Le fonctionnement de chaque agent, quel que soit son rôle, peut être modélisé par un cycle opérationnel fondamental :

1. **Perception** : L'agent collecte des données pertinentes de son environnement via diverses sources (bases de données, APIs, messages d'autres agents).
2. **Délibération** : L'agent analyse les données perçues pour mettre à jour sa représentation interne du monde et décider de la meilleure action à entreprendre pour atteindre ses objectifs. Dans notre projet, cette phase peut mobiliser des workflows **n8n** pour le traitement des données.
3. **Action** : L'agent exécute l'action choisie, modifiant ainsi l'environnement (ex : mise à jour d'une base de données, appel à une API externe).

3.2 RAG : La Synergie entre Récupération d'Information et Génération

3.2.1 Définition et Principe Fondamental

La **Génération Augmentée par Récupération (RAG)** est une architecture d'IA [2] conçue pour résoudre deux limitations majeures des LLMs standards : leur **frontière de connaissance** (ils ne connaissent rien au-delà de leur date d'entraînement) et leur tendance à l'**hallucination**.

Le principe du RAG est d'ancrer la capacité de génération du LLM dans une source de données externe, factuelle et contrôlable. Au lieu de demander au LLM de répondre "de mémoire", on lui fournit, au moment de la question, les extraits d'information pertinents pour qu'il construise sa réponse à partir de ceux-ci. Dans un SMA, le RAG est le mécanisme qui transforme un agent "généraliste" en un agent "expert", dont les affirmations sont vérifiables.

3.2.2 Fonctionnement Détaillé

Le processus RAG est une séquence en trois étapes :

1. **Recherche Sémantique (*Retrieval*)** : La requête de l'utilisateur est transformée en un vecteur numérique (embedding) qui capture son sens. Ce vecteur est ensuite utilisé pour rechercher les documents ou extraits les plus sémantiquement similaires dans une base de données vectorielle.
2. **Augmentation du Contexte** : Les extraits pertinents récupérés sont ensuite insérés dans le prompt, à côté de la question initiale, pour former un contexte enrichi.
3. **Génération Ancrée (*Grounded Generation*)** : Ce prompt augmenté est finalement soumis au LLM, avec l'instruction de baser sa réponse exclusivement sur les informations fournies dans le contexte.

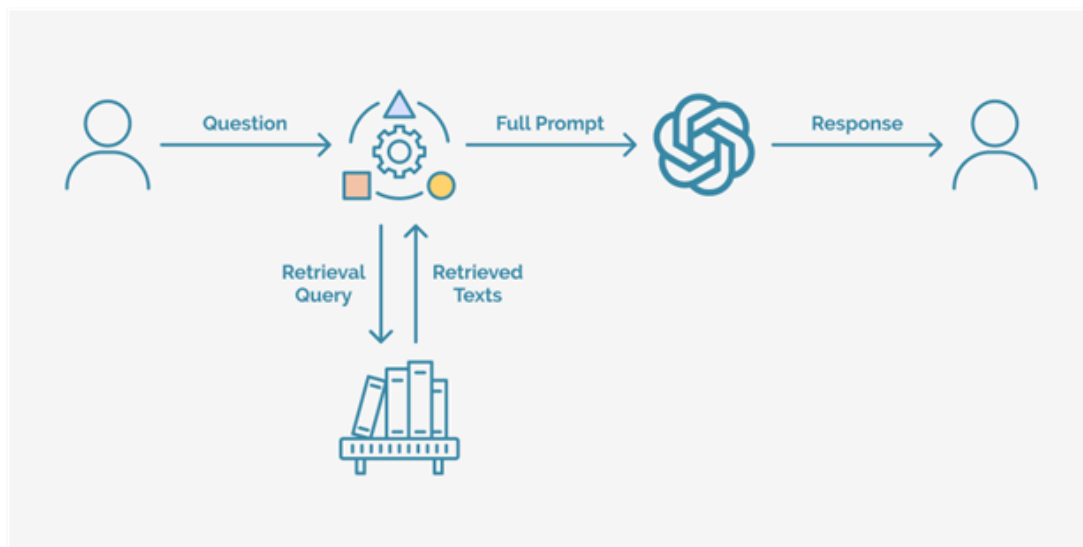


FIG. 3.2 – Workflow du modèle RAG, de la requête initiale à la réponse finale augmentée.

3.2.3 Avantages et Rôle Stratégique dans les SMA

L'intégration de RAG dans l'architecture des agents procure des avantages décisifs pour la viabilité d'un système en production :

- **Fiabilité et Précision** : Les réponses sont basées sur des données factuelles, ce qui réduit drastiquement les risques d'hallucination et renforce la confiance des utilisateurs dans le système.
- **Connaissances à Jour** : La mise à jour des connaissances de l'agent se fait simplement en actualisant la base de données externe, sans nécessiter le ré-entraînement coûteux du LLM.
- **Transparence et Auditabilité** : Le système peut citer ses sources en indiquant quels documents ont été utilisés pour générer une réponse, ce qui permet une vérification et un audit.
- **Spécialisation et Expertise** : RAG permet de créer des agents experts dans des domaines de niche (support technique, analyse juridique, etc.) en leur donnant accès à des bases de connaissances propriétaires et spécialisées. C'est le mécanisme qui élève un agent du statut de "chatbot" à celui d' "assistant expert".

Conclusion

Ce chapitre a établi les deux piliers théoriques sur lesquels notre projet est édifié. D'une part, le paradigme des **Systèmes Multi-Agents** nous fournit le cadre architectural pour concevoir une solution distribuée, robuste et scalable, capable de gérer la complexité par la collaboration d'entités spécialisées. D'autre part, la technique de **Génération Augmentée par Récupération** nous donne le moyen de doter chaque agent d'une expertise fiable et vérifiable, enracinant leurs capacités cognitives dans des bases de connaissances maîtrisées. Forts de ces fondements, nous pouvons désormais explorer dans le chapitre suivant l'état de l'art des technologies spécifiques qui nous permettront de mettre en œuvre ces concepts.

Chapitre 4

État de l'Art des Technologies

Fondamentales

Introduction

Après avoir établi les fondements théoriques des Systèmes Multi-Agents et du RAG, ce chapitre a pour objectif de passer en revue l'état de l'art des technologies spécifiques qui permettent leur mise en œuvre. Cette analyse poursuit un double objectif : d'une part, présenter le paysage technologique actuel et les outils les plus performants à notre disposition ; d'autre part, justifier de manière rigoureuse la sélection de chaque composant de notre stack technologique. Nous examinerons successivement trois piliers essentiels : le moteur cognitif de nos agents (les LLMs), l'interface de contrôle de leur comportement (le Prompt Engineering), et enfin, la couche de représentation sémantique qui leur permet de comprendre les données (les Embeddings).

4.1 Les Grands Modèles de Langage (LLM)

4.1.1 Définition et Rupture Architecturale du Transformer

Un **Grand Modèle de Langage (LLM)** est un modèle d'apprentissage profond, généralement basé sur des réseaux de neurones, qui a été entraîné sur de vastes corpus de données textuelles. Sa fonction première est de modéliser la distribution de probabilité du langage hu-

main, lui permettant de prédire le *token* suivant dans une séquence. Cette capacité fondamentale est ensuite exploitée pour des tâches complexes telles que la génération de texte, la traduction, la synthèse ou le raisonnement.

La performance des LLMs modernes est intrinsèquement liée à une rupture architecturale majeure : le **Transformer** [1]. Introduite en 2017, cette architecture abandonne les approches séquentielles traditionnelles (comme les RNNs) au profit d'un mécanisme d'**auto-attention** (*self-attention*). Ce dernier permet au modèle de traiter tous les tokens d'une séquence en parallèle et de peser dynamiquement l'importance de chaque mot par rapport à tous les autres, capturant ainsi des dépendances contextuelles à longue portée de manière extrêmement efficace.

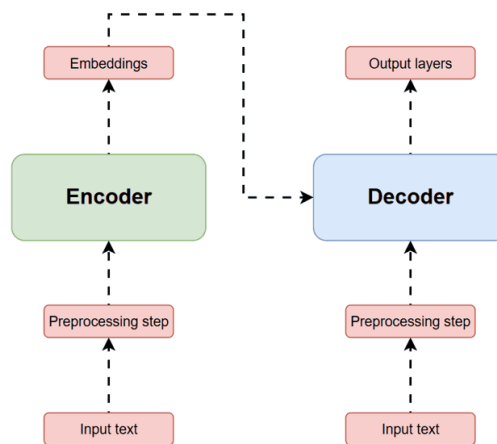


FIG. 4.1 – Schéma simplifié de l'architecture Transformer, mettant en évidence les mécanismes d'attention et les couches feed-forward.

4.1.2 Les Modèles Open Source : Le Compromis Performance-Coût

Le choix d'un LLM pour une application concrète est une décision d'ingénierie qui repose sur une analyse du compromis entre la performance (qualité du raisonnement, précision), le coût de calcul (ressources GPU/CPU) et la latence d'inférence. Le tableau 4.1 présente une analyse comparative de modèles open-source représentatifs de ce spectre.

TAB. 4.1 – Benchmark comparatif de LLMs open-source pour une utilisation dans un système multi-agents.

Modèle	Taille / Paramètres	Matériel Requis (GPU)	Profil d'Usage et Compromis
TinyLlama 1.1B	≈ 1,1 milliard	4-8 GB VRAM ou CPU quantifié	Très rapide pour tâches simples (chat, résumé). Idéal pour agents légers et nombreux. Moins performant pour raisonnement complexe.
Mistral 7B	≈ 7,3 milliards	10-14 GB VRAM	Excellent compromis performance / coût. Très performant pour conversations et instructions. Plus rapide que modèles 8B.
Llama 3 8B	8 milliards	12-16 GB VRAM	Haute qualité pour sa taille. Idéal pour agents nécessitant un raisonnement poussé et meilleure compréhension contextuelle. Coût VRAM élevé.

4.1.3 Formalisation des Critères de Sélection et Modèle Retenu

Critères de Sélection pour un Système Multi-Agents

La sélection du LLM pour notre projet a été guidée par un ensemble de contraintes formelles, découlant directement des exigences de notre système multi-agents :

- **Latence et Vitesse d'Inférence** : Pour garantir la réactivité des agents, une faible latence est primordiale. Cela favorise les modèles de taille modérée (< 10 milliards de paramètres), potentiellement quantifiés.
- **Capacité de Raisonnement et Fenêtre de Contexte** : Les agents devant analyser des documents ou conserver des historiques de conversation longs, une grande fenêtre de contexte est une exigence non-négociable.

- **Efficiencia des Ressources (VRAM/CPU) :** Le déploiement de multiples agents en parallèle impose une contrainte forte sur la mémoire. Le coût cumulé doit être compatible avec une infrastructure raisonnable.

Modèle Retenu : Llama 3.1 8B

Sur la base de ces critères, le modèle **Llama 3.1 8B** [8] a été sélectionné comme moteur cognitif de nos agents.

- **Provenance :** Publié par Meta en juillet 2024, il représente l'état de l'art des modèles de sa catégorie.
- **Performance :** Il offre un équilibre exceptionnel entre une taille maîtrisée (8 milliards de paramètres) et une capacité de raisonnement de premier ordre.
- **Fenêtre de Contexte :** Il supporte une très large fenêtre de contexte de **128 000 tokens**, ce qui répond parfaitement à notre besoin de traiter des informations complexes et étendues.

Ce modèle constitue ainsi le meilleur compromis pour notre cas d'usage, alliant performance de raisonnement et efficacité des ressources.

4.2 L'Ingénierie de Prompts (Prompt Engineering)

4.2.1 Définition et Rôle Stratégique

Le **Prompt Engineering** est la discipline d'ingénierie qui consiste à concevoir, formuler et optimiser les instructions (prompts) soumises à un LLM afin de contraindre son comportement et d'obtenir des résultats fiables, précis et alignés avec un objectif métier. C'est l'interface de contrôle principale qui permet de transformer un modèle généraliste en un outil spécialisé.

Dans une architecture multi-agents, le prompt engineering est d'une importance capitale, car le prompt système de chaque agent agit comme sa "constitution" :

- **Il définit sa personnalité (*Persona*) :** "Tu es un expert comptable rigoureux et prudent."

- **Il spécifie sa mission et ses capacités :** "Ta tâche est d'analyser et de résumer des transactions financières."
- **Il fixe ses contraintes et ses limites :** "Tu ne dois en aucun cas fournir de conseil juridique."

4.2.2 Types Fondamentales

Le prompt engineer dispose d'une "boîte à outils" de techniques pour guider le modèle avec précision :

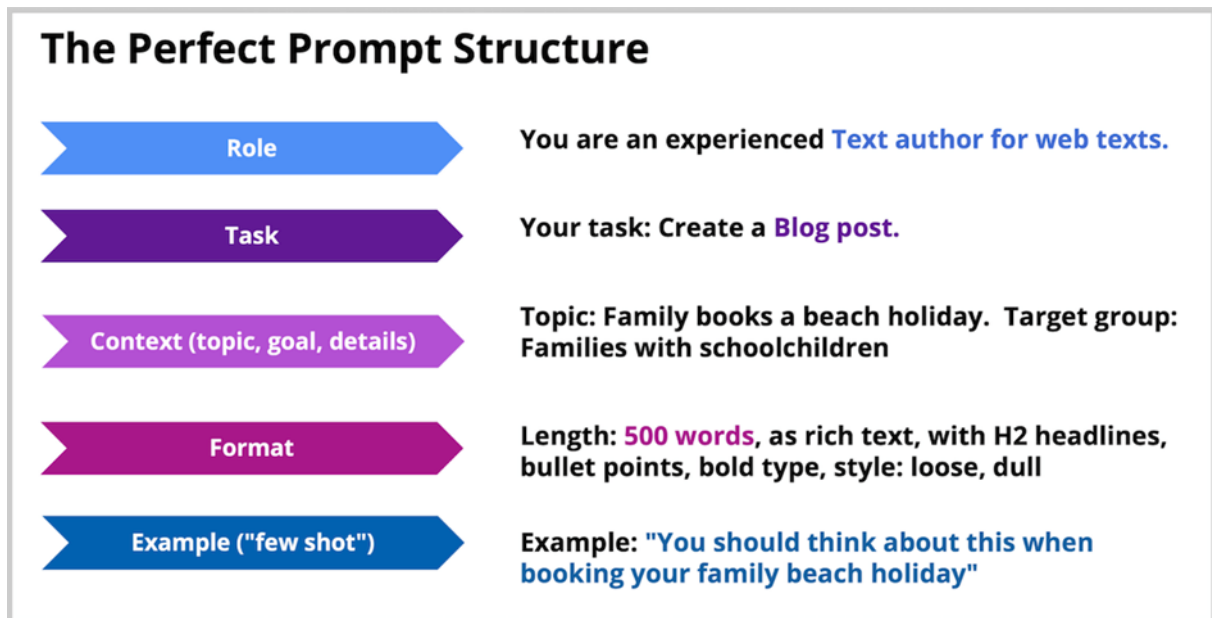


FIG. 4.2 – types des prompts pour un LLM

Role Prompting C'est la technique de base qui consiste à assigner un rôle explicite à l'agent pour cadrer son style, son domaine de connaissance et son ton. Elle est essentielle pour différencier les agents d'un système (ex : *analyste, vérificateur, communicateur*).

Few-Shot Prompting Cette technique consiste à inclure dans le prompt quelques exemples de paires "entrée-sortie" de haute qualité. En montrant au modèle le format et le type de réponse attendu, on améliore considérablement sa précision et sa capacité à suivre des instructions complexes.

Chain of Thought (CoT) On instruit explicitement le modèle de ”réfléchir étape par étape” avant de fournir sa réponse finale. Cette méthode contraint le LLM à décomposer un problème complexe en sous-problèmes logiques, ce qui améliore drastiquement la qualité de son raisonnement.

ReAct (Reason + Act) Cette approche, au cœur des agents autonomes modernes, combine le raisonnement (CoT) avec l’action. L’agent peut décider d’utiliser un outil externe (ex : appeler une API, interroger une base de données) pour obtenir de nouvelles informations, puis intégrer ces informations dans son raisonnement pour planifier l’étape suivante [6].

4.3 Les Plongements Sémantiques (Embeddings)

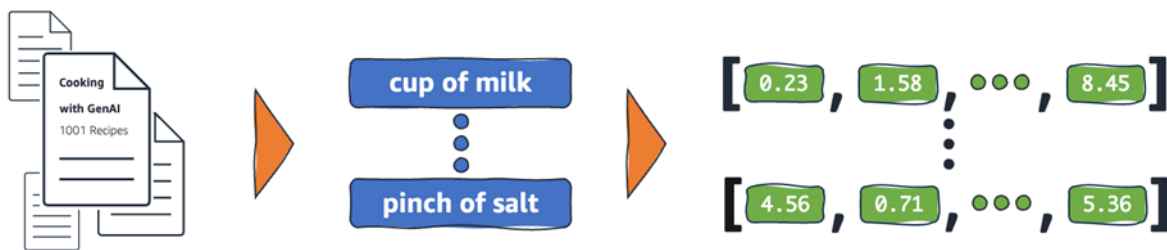


FIG. 4.3 – Fonctionnement d’Embedding

4.3.1 Définition : De la Donnée au Vecteur

Un **embedding**, ou plongement sémantique, est une représentation vectorielle dense et de haute dimension d’un objet discret (mot, phrase, document). Cette technique projette les données textuelles dans un espace géométrique continu où la **proximité spatiale entre les vecteurs reflète leur similarité sémantique**. C’est ce mécanisme qui permet aux machines de ”comprendre” la proximité de sens entre des concepts comme ”roi” et ”reine”.

4.3.2 Applications et Avantages dans le Projet

Les embeddings constituent la fondation technique de plusieurs capacités avancées de nos agents :

- **Moteur de la Recherche Sémantique (RAG) :** Les embeddings sont le cœur de la phase de *Retrieval* du RAG. En convertissant la requête utilisateur et les documents de la base de connaissances en vecteurs, le système peut identifier les informations pertinentes sur la base de la similarité de sens, et non par simple correspondance de mots-clés.
- **Amélioration de la Performance des Modèles :** Ils fournissent une représentation riche et dense de l'information, améliorant la performance des modèles pour des tâches en aval comme la classification ou le clustering de textes.

Conclusion du Chapitre

Cet état de l'art a passé en revue les trois piliers technologiques de notre projet. Nous avons vu que les **LLMs**, et en particulier l'architecture Transformer, fournissent le moteur cognitif de base. Nous avons ensuite analysé le **Prompt Engineering** comme la discipline essentielle pour contrôler et spécialiser ce moteur. Enfin, nous avons identifié les **Embeddings** comme la couche de représentation sémantique indispensable à des fonctions avancées comme la recherche d'information. La sélection de **Llama 3.1** comme modèle, l'application de techniques de prompting avancées et l'utilisation d'embeddings pour le RAG forment une stack technologique cohérente, robuste et à l'état de l'art. Ces choix justifiés, nous pouvons maintenant aborder dans le chapitre suivant la conception de l'architecture système qui intégrera ces composants.

Chapitre 5

Analyse et Conception du Système

Introduction

Après avoir posé les fondements théoriques et l'état de l'art, ce chapitre constitue le pivot de notre travail en faisant le pont entre la théorie et la mise en œuvre pratique. Il a pour objectif de détailler le plan de notre système d'agents intelligents. Nous commencerons par formaliser le cahier des charges à travers la définition des exigences fonctionnelles et non-fonctionnelles. Nous présenterons ensuite l'architecture logicielle retenue, en justifiant nos choix structurels. Nous décrirons le workflow opérationnel qui régit le traitement de l'information, avant de conclure par une analyse des risques et un positionnement de notre approche par rapport aux paradigmes existants.

5.1 Spécifications et Exigences du Système

La première étape de toute démarche d'ingénierie rigoureuse consiste à traduire les besoins métiers en un ensemble de spécifications techniques précises. Ce cahier des charges distingue les exigences fonctionnelles (ce que le système doit faire) des exigences non-fonctionnelles (comment le système doit le faire).

5.1.1 Exigences Fonctionnelles

Les exigences fonctionnelles définissent les capacités fondamentales du système :

- **Autonomie d'Exécution** : Le système doit être capable d'exécuter des processus complexes de bout en bout sans nécessiter d'intervention humaine.
- **Ingestion de Données Hétérogènes** : Le système doit pouvoir se connecter à des sources de données multiples (API, bases de données) et traiter des informations en temps réel.
- **Capacité de Décision** : Les agents doivent posséder une logique interne leur permettant de prendre des décisions éclairées et alignées avec des objectifs prédéfinis.
- **Coordination et Collaboration** : Les agents doivent pouvoir interagir via des protocoles de communication définis pour synchroniser leurs actions et atteindre des objectifs collectifs.
- **Supervision et Contrôle** : Une interface de gestion (GUI) doit permettre à un opérateur humain de configurer, de superviser et de monitorer l'activité des agents.

5.1.2 Exigences Non-Fonctionnelles

Ces exigences définissent les attributs de qualité et les contraintes de performance du système :

- **Scalabilité** : L'architecture doit être conçue pour supporter une montée en charge, que ce soit en termes de volume de données, de nombre d'agents ou de complexité des tâches.
- **Fiabilité et Robustesse** : Le système doit être résilient aux pannes partielles et intégrer des mécanismes de gestion d'erreurs et d'exceptions pour garantir la continuité de service.
- **Sécurité** : Des mesures de sécurité robustes doivent être implémentées à chaque couche de l'architecture pour protéger l'intégrité et la confidentialité des données.
- **Maintenabilité et Évolutivité** : Le code doit être modulaire, documenté et suivre des principes de conception logicielle favorisant les évolutions futures.

5.2 Architecture Générale du Système

Pour répondre à ces exigences, nous avons opté pour une architecture logicielle multi-couches. Ce choix est motivé par les principes fondamentaux de l'ingénierie logicielle, notamment la **séparation des préoccupations** (*Separation of Concerns*) et le **découplage**, qui garantissent la maintenabilité et l'évolutivité du système.

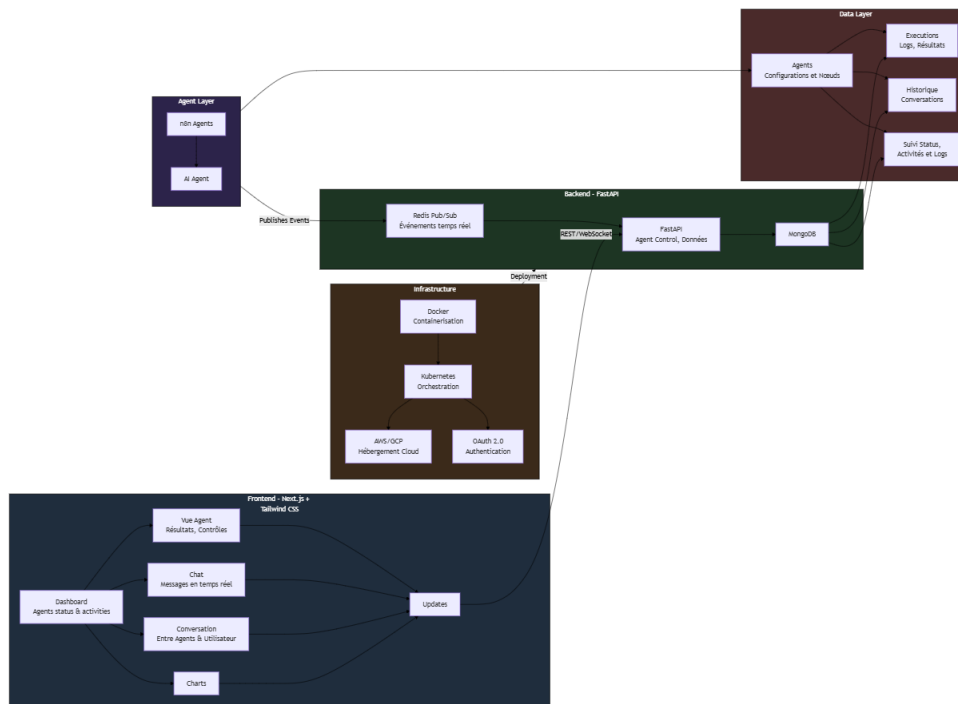


FIG. 5.1 – Diagramme de l'architecture en quatre couches du système multi-agents.

Couche Cognitive et Métier (Agents) Cœur du système, cette couche héberge les agents intelligents. Chaque agent encapsule la logique métier d'une tâche spécifique et est doté d'un modèle de langage (LLM) qui lui confère ses capacités de raisonnement.

Couche de Communication et d'Orchestration Cette couche est le système nerveux de notre architecture. Elle gère les flux de communication inter-agents, en s'appuyant sur un bus de messages ou des services dédiés pour permettre la collaboration, la synchronisation et la délégation des tâches.

Couche de Persistance des Données Elle centralise la gestion du stockage. Elle est hétérogène par nature, combinant des bases de données vectorielles (pour le RAG), des bases de données en mémoire comme **Redis** pour le cache et la gestion d'état, et potentiellement des bases NoSQL/SQL pour les données structurées.

Couche de Présentation (Interface Utilisateur) Cette couche est la porte d'entrée pour les utilisateurs humains. Développée avec **FastAPI** [12], elle expose les fonctionnalités du système via une API et une interface web, permettant la configuration et le monitoring.

5.3 Les serveurs Model Context Protocol (MCP)

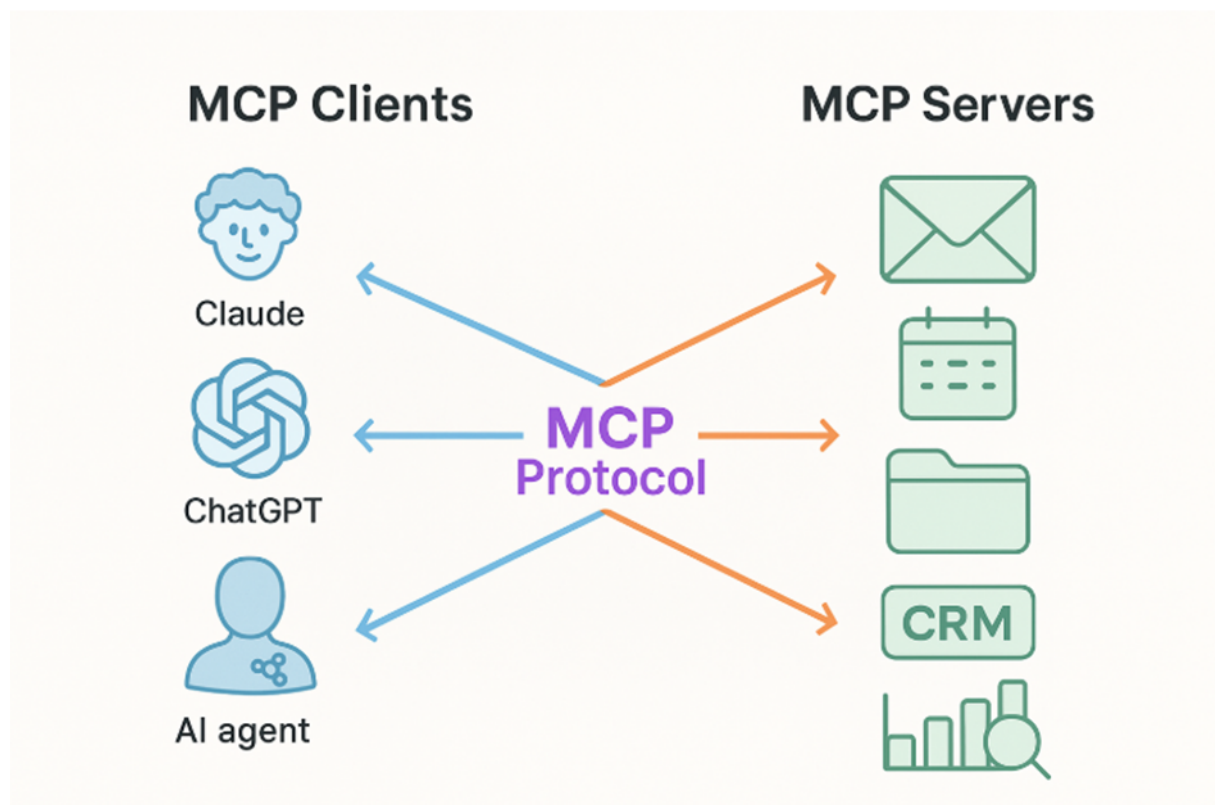


FIG. 5.2 – diagramme de fonctionnement des serveurs MCP

Pour assurer une communication fluide et standardisée entre les agents eux-mêmes qu'avec les services externes, nous avons mis en œuvre une architecture basée sur un Model Context Protocol (MCP). Il ne s'agit pas d'un protocole réseau, mais d'une convention interne définissant

une structure de données (un schéma) qui encapsule les informations et le contexte nécessaires à chaque étape d'un workflow.

5.3.1 Principe et Rôle du MCP

Le MCP agit comme une "enveloppe" de données standardisée, généralement au format JSON, que tous les agents sont capables de comprendre et de manipuler. Chaque objet MCP contient non seulement les données utiles (*payload*), mais aussi des métadonnées essentielles comme l'origine de la donnée, l'état de la tâche, un identifiant unique, et le contexte nécessaire aux agents en aval pour prendre des décisions.

Cette approche permet de découpler la logique de chaque agent. Un agent n'a pas besoin de connaître le fonctionnement interne des autres ; il lui suffit de savoir qu'il recevra un objet MCP en entrée et devra produire un objet MCP en sortie.

5.3.2 Communication Inter-Agents au sein de n8n

Le principal cas d'usage du MCP se trouve au cœur de nos workflows n8n, où il sert de "langage commun" entre les différents nœuds qui composent un agent complexe.

En standardisant la structure des données qui transitent entre les étapes, le MCP garantit la fiabilité des workflows et facilite grandement leur maintenance et leur évolution.

5.3.3 Interaction avec les Serveurs Publics

Le MCP sert également de couche d'abstraction pour interagir avec des points d'accès publics, comme les API de Shopify. Plutôt que de formater des requêtes spécifiques à chaque service dans chaque workflow, nous utilisons le MCP comme format pivot.

Cette méthode nous permet de centraliser la logique de communication avec les API externes. Si Shopify met à jour son API, nous n'avons qu'à modifier notre "client MCP" central, sans toucher à la logique des dizaines de workflows qui l'utilisent. Cela rend notre système beaucoup plus modulaire et résilient aux changements externes.

5.4 Analyse des Risques et Défis Techniques

La conception d'un tel système implique une analyse proactive des risques et des défis inhérents à sa complexité :

- **Fiabilité de la Communication Inter-Agent Asynchrone** : Garantir la livraison et le traitement corrects des messages entre agents dans un contexte asynchrone est un défi majeur.
- **Gestion de la Concurrency et de la Synchronisation** : Prévenir les conditions de course et les incohérences de données lorsque plusieurs agents opèrent sur des ressources partagées.
- **Sécurité de l'Exposition par API** : L'autonomie des agents et leur exposition via des APIs créent une surface d'attaque qui doit être rigoureusement sécurisée (authentification, autorisation, validation des entrées).
- **Complexité de l'Intégration et Dépendances** : L'intégration de multiples technologies (agents Python, workflows n8n, bases de données) crée des dépendances qui doivent être gérées avec soin pour éviter les problèmes de compatibilité.

5.5 Positionnement par Rapport aux Approches Existantes

La conception de ce système, bien que novatrice dans son application, s'appuie sur une riche histoire de recherche en systèmes multi-agents (SMA).

- **Héritage des Approches Symboliques** : Des architectures cognitives fondatrices comme **BDI (Belief-Desire-Intention)** [4], nous conservons le principe fondamental de modéliser explicitement les objectifs et les plans d'action des agents.
- **Apports de l'Apprentissage Automatique** : La transition vers les systèmes apprenants, notamment via l'apprentissage par renforcement multi-agent (MARL), a ouvert la voie à des agents adaptatifs. Notre approche s'inspire de cette adaptabilité, mais l'implémente via la flexibilité des LLMs.

- **Inscription dans l'Ère des LaMAS :** Notre système se positionne clairement dans la mouvance des **LaMAS (LLM-based Multi-Agent Systems)** [5]. À l'instar de cadres de recherche comme **CAMEL** [7], nous utilisons un LLM comme "cerveau" cognitif. Notre contribution se distingue par une approche pragmatique et orientée métier, hybridant ces capacités cognitives avec des outils d'orchestration robustes comme **n8n**.

En conclusion, notre conception vise à synthétiser le meilleur de ces mondes : la rigueur des architectures symboliques, l'adaptabilité des systèmes apprenants et la puissance cognitive des LLMs, le tout dans un cadre logiciel performant et aligné avec les pratiques de l'industrie.

Conclusion

Ce chapitre a permis de formaliser le plan directeur de notre système. En partant d'un cahier des charges rigoureux, nous avons défini une architecture multi-couches conçue pour la modularité et la scalabilité. Le workflow opérationnel a été détaillé, et la stack technologique justifiée. Cette conception, ancrée dans les principes de l'ingénierie logicielle et informée par l'état de l'art de la recherche en SMA, constitue une réponse cohérente et robuste à la problématique initialement posée. Ce plan directeur étant désormais établi, le chapitre suivant se consacrera à sa mise en œuvre concrète et à la présentation des outils spécifiques mobilisés.

Chapitre 6

Architecture Technologique et Stack d'Implémentation

Introduction

Ce chapitre a pour objectif de détailler la concrétisation de l'architecture logicielle définie précédemment. La transition d'un plan directeur conceptuel à un système fonctionnel repose sur une sélection rigoureuse de technologies où chaque composant doit répondre de manière optimale aux exigences fonctionnelles et non-fonctionnelles identifiées. Nous présenterons ici la stack technologique complète du projet en justifiant le choix de chaque outil, framework et service. L'ensemble constitue un écosystème cohérent et moderne, conçu pour la performance, la scalabilité et la maintenabilité.

6.1 Fondations de l'Intelligence Artificielle et Orchestration

Le cœur cognitif et collaboratif du système repose sur une combinaison de deux frameworks spécialisés, l'un gérant les capacités individuelles des agents, l'autre leur intelligence collective.



FIG. 6.1 – Logo de n8n

6.1.1 n8n [10]

- **Principe et Positionnement** : n8n est une plateforme d'automatisation des flux de travail open-source . Elle permet de connecter diverses applications et services via une interface visuelle basée sur des nœuds, offrant ainsi un contrôle granulaire sur l'orchestration des processus métiers et des pipelines de données.
- **Justification et Rôle dans l'Architecture** : L'intégration de n8n répond à un besoin fondamental d'orchestration externe et de connectivité de notre système multi-agents. N8N agit comme la couche d'intégration qui connecte le système à des API externes, des bases de données et des services tiers. Il a été privilégié pour sa capacité à être auto-hébergé, garantissant ainsi la souveraineté des données, ainsi que pour sa grande **extensibilité** et sa **fiabilité** pour l'exécution de tâches planifiées ou déclenchées par des événements.
- **Fonctionnalités Clés Mises en Œuvre** :
 - **Nœuds (Nodes)** : Chaque nœud représente une application ou une fonction spécifique (par exemple, lire une base de données, appeler une API REST). Nous les utilisons pour encapsuler des actions unitaires.
 - **Flux de travail (Workflows)** : Les flux de travail constituent la séquence logique des nœuds. Ils sont employés pour définir les processus complexes que les agents peuvent déclencher, comme la génération de rapports périodiques ou l'agrégation de données provenant de sources multiples.
 - **Déclencheurs (Triggers)** : Pour initier automatiquement des flux de travail en réponse à des événements externes (par exemple, un webhook, une planification ho-

raire), permettant ainsi une interaction proactive du système avec son environnement.

6.1.2 CrewAI [11]



FIG. 6.2 – Logo de crewai

- **Principe et Positionnement** : CrewAI est un framework d'orchestration de plus haut niveau, spécifiquement conçu pour modéliser et gérer la collaboration entre plusieurs agents autonomes.
- **Justification et Rôle dans l'Architecture** : Si LangChain fournit les briques, CrewAI fournit le ciment. Il a été choisi pour répondre à notre exigence de **coordination** inter-agent. Il permet de formaliser les rôles, les tâches et les protocoles de délégation, transformant une collection d'agents individuels en une équipe cohérente (*crew*).

6.2 Couche de Service et Exposition (Back-End)

6.2.1 FastAPI [12]



FIG. 6.3 – Logo de FastApi

- **Principe et Positionnement** : FastAPI est un framework web Python moderne, basé sur les standards ASGI, conçu pour la création d'API haute performance.

- **Justification et Rôle dans l'Architecture :** Le choix de FastAPI répond directement à nos exigences de **performance**, de **scalabilité** et de **maintenabilité**. Sa nature asynchrone est idéale pour gérer les opérations I/O non-bloquantes (appels aux LLMs, requêtes aux bases de données). De plus, sa génération automatique de documentation interactive (Swagger UI) à partir du typage de données Python accélère le cycle de développement et facilite l'intégration avec le front-end.

6.3 Interface Utilisateur (Front-End)

6.3.1 Next.js [13]



FIG. 6.4 – Logo de NextJs

- **Principe et Positionnement :** Next.js est un framework de production basé sur React, spécialisé dans la création d'applications web hybrides (rendu côté serveur et génération statique).
- **Justification et Rôle dans l'Architecture :** Pour l'interface de supervision, nous avons besoin d'une expérience utilisateur réactive et performante. Next.js a été choisi pour sa capacité à construire des interfaces complexes de manière structurée et pour ses optimisations de performance natives (division du code, pré-chargement), garantissant une interaction fluide pour l'opérateur humain.

6.4 Gestion des Données et Persistance

Notre architecture de données est hétérogène, chaque base de données étant choisie pour un usage spécifique.

6.4.1 MongoDB



FIG. 6.5 – Logo de MongoDB.

- **Principe et Positionnement** : MongoDB est une base de données NoSQL orientée documents.
- **Justification et Rôle dans l'Architecture** : Choisie pour sa flexibilité, MongoDB sert de base de données principale pour les données applicatives dont le schéma est amené à évoluer, comme les logs de conversation, les profils des agents ou les résultats des tâches.

6.4.2 Qdrant [14]



FIG. 6.6 – Logo de Qdrant

- **Principe et Positionnement** : Qdrant est une base de données vectorielle optimisée pour la recherche de similarité à grande échelle.
- **Justification et Rôle dans l'Architecture** : Qdrant est un composant non-négociable de notre implémentation RAG. Sa capacité à stocker et à interroger des millions de vecteurs avec une faible latence est essentielle pour que les agents puissent retrouver des informations pertinentes en temps réel.

6.4.3 Redis



FIG. 6.7 – Logo de Redis.

- **Principe et Positionnement** : Redis est un magasin de données en mémoire, principalement utilisé pour sa très haute performance en lecture et en écriture. Il structure les données sous forme de paires clé-valeur.
- **Justification et Rôle dans l'Architecture** : Redis est un composant central pour la gestion des données volatiles et la communication inter-agents. Il remplit trois rôles critiques : **cache** pour réduire la latence des requêtes fréquentes, **gestionnaire de sessions** pour l'interface web, et **courtier de messages (message broker)** pour les files d'attente de tâches asynchrones.
- **Mise en Œuvre du Paradigme Publish/Subscribe (Pub/Sub)** : En complément des files d'attente, nous exploitons le système Pub/Sub de Redis pour la communication en temps réel.
 - **Principe du Pub/Sub** : Ce modèle de messagerie permet à des composants, les "éditeurs" (Publishers), d'envoyer des messages sur des "canaux" (channels) nommés,

sans avoir connaissance des destinataires. D'autres composants, les "abonnés" (Subscribers), s'abonnent à ces canaux et reçoivent instantanément tous les messages qui y sont publiés.

- **Application dans le Système Multi-Agents :** Le Pub/Sub est essentiel pour la diffusion d'événements et la synchronisation d'état entre les agents et notre système de management . Par exemple, lorsqu'un agent termine une étape cruciale, il peut publier une notification sur un canal "system_events". Le système abonne à ce canal peut alors réagir à cet événement de manière asynchrone et découplée, permettant une architecture réactive et modulaire.

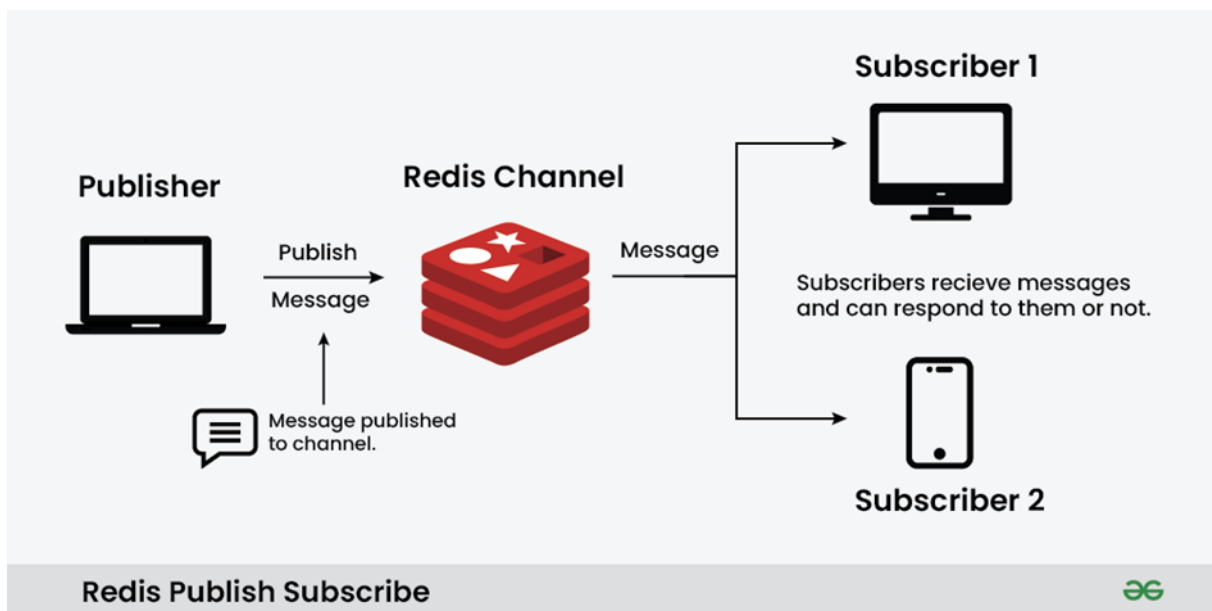


FIG. 6.8 – Modèle redis Pub/Sub

6.5 Intégration de Services Tiers et Canaux de Communication

6.5.1 WhatsApp Business Cloud API



FIG. 6.9 – Schéma API Whatsapp

- **Principe et Positionnement** : API officielle de Meta pour intégrer WhatsApp dans des applications tierces.
- **Justification et Rôle dans l'Architecture** : Pour interagir avec les utilisateurs finaux sur un canal largement adopté, l'API WhatsApp a été intégrée. Elle sert de "connecteur" entre nos agents et le monde extérieur, leur permettant d'envoyer des notifications et de recevoir des requêtes.

6.5.2 Retell AI



FIG. 6.10 – Logo de retell Ai

- **Principe et Positionnement** : Plateforme spécialisée dans la création d’agents vocaux.
- **Justification et Rôle dans l’Architecture** : Pour démontrer la **multi-modalité** potentielle du système, Retell AI a été intégré. Il permet de doter nos agents d’une interface vocale, transformant la logique textuelle en une conversation téléphonique fluide et naturelle.

6.6 Industrialisation et Déploiement

6.6.1 Docker

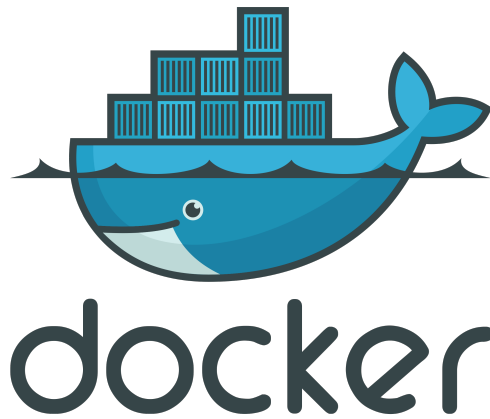


FIG. 6.11 – Logo de Docker

- **Principe et Positionnement** : Docker est la plateforme standard de l’industrie pour la conteneurisation d’applications.
- **Justification et Rôle dans l’Architecture** : Docker est essentiel pour répondre à nos exigences de **fiabilité** et de **maintenabilité**. En encapsulant chaque service (back-end, front-end, bases de données) dans un conteneur isolé, nous garantissons la reproductibilité des environnements entre le développement et la production. Cela simplifie drastiquement le déploiement et la gestion des dépendances.

Conclusion du Chapitre

Ce chapitre a détaillé la stack technologique qui concrétise notre architecture système. La sélection de chaque composant a été le fruit d’une analyse rigoureuse visant à constituer un

écosystème cohérent, performant et scalable. La synergie entre des frameworks d'IA spécialisés comme LangChain et CrewAI, une couche de service robuste basée sur FastAPI, des solutions de persistance adaptées à chaque type de donnée, et des outils d'industrialisation comme Docker, forme une fondation technique solide. C'est sur ce socle que repose la mise en œuvre des cas d'usage et l'obtention des résultats qui seront présentés dans le chapitre suivant.

Chapitre 7

Implémentation et Cas d'Usage

Ce chapitre présente la concrétisation du projet, depuis l'implémentation d'agents spécialisés jusqu'au développement du système web complet qui sert d'interface de gestion. Il se conclut par une analyse des défis techniques rencontrés durant la phase de réalisation.

7.1 Conteneurisation avec Docker

Afin de garantir la portabilité, la scalabilité et surtout la souveraineté des données, l'ensemble de l'écosystème d'agents a été déployé dans un environnement conteneurisé et auto-hébergé. Cette approche nous permet de créer un système hermétique, où toutes les données, des informations des prospects aux conversations internes, restent sur une infrastructure maîtrisée. Le choix s'est porté sur Docker et Docker Compose pour orchestrer les multiples services qui composent la solution.

☐	Port(s)	Name	Container ID	Image	CPU (%)	Memory usage...	Me	Actions
☐	3002:3000	my-fullstack-i	dc6bdd134592	self-hosted	0.23%	147.1MB / 7.68Gi		☐ : <input type="trash"/>
☐	5678:5678	n8n	0f99f62603a4	n8n-python	0%	365.6MB / 7.68Gi		☐ : <input type="trash"/>
☐	-	n8n-import	bacc33bcd4d5	n8n-python	0%	0B / 0B		☐ : <input type="trash"/>
☐	6379:6379	redis	9ee47cf17784	redis:latest	0.51%	11.09MB / 7.68Gi		☐ : <input type="trash"/>
☐	6333:6333	qdrant	92d71d1fbb5a	qdrant/qdrant	0.12%	204MB / 7.68Gi		☐ : <input type="trash"/>
☐	5432:5432	postgres-1	7883d203119b	postgres:16	0%	44.25MB / 7.68Gi		☐ : <input type="trash"/>
☐	27017:27017	mongodb	d031dad3c3a2	mongo:latest	0.55%	136.5MB / 7.68Gi		☐ : <input type="trash"/>
☐	-	ollama-pull-lle	ac8bd33f2aaf	ollama/ollama	0%	0B / 0B		☐ : <input type="trash"/>
☐	11434:11434	ollama	20fa91138dfc	ollama/ollama	0%	40.02MB / 7.68Gi		☐ : <input type="trash"/>

FIG. 7.1 – Conteneur Docker avec tout l’environnement n8n

7.1.1 n8n AI Kit Auto-Hébergé

La base de notre déploiement est le "AI Kit" fourni par n8n, une solution pré-configurée qui facilite grandement la mise en place d’un environnement d’automatisation intelligent. Ce kit, orchestré via un fichier `docker-compose.yml`, déploie un ensemble de services inter-connectés :

- **n8n** : Le cœur de l’orchestration, où tous les workflows de nos agents sont conçus et exécutés.
- **Ollama** : Un service permettant d’exécuter localement des grands modèles de langage (LLM) comme Llama 3.1. C’est le cerveau de nos agents, garantissant que les données sensibles traitées ne quittent jamais notre serveur.
- **Qdrant** : Une base de données vectorielle utilisée pour la construction de nos bases de connaissances (notamment pour l’agent de support), permettant des recherches par similarité sémantique à haute vitesse.
- **PostgreSQL** : La base de données relationnelle sous-jacente, utilisée par n8n pour stocker les workflows, les exécutions et les informations d’identification.

À cette base, nous avons ajouté deux services essentiels pour les besoins de notre système de gestion web :

- **MongoDB** : Une base de données NoSQL choisie pour sa flexibilité, utilisée par notre système web pour stocker les logs d'erreurs, suivre l'état des agents et gérer les accès utilisateurs.
- **Redis** : Un système de cache en mémoire, utilisé pour la gestion des sessions utilisateurs sur la plateforme web et pour la communication en temps réel entre les agents et l'interface de supervision.

7.1.2 Personnalisation de l'Image n8n pour les Besoins Spécifiques

Le service n8n standard, bien que puissant, ne contient pas toutes les dépendances nécessaires à l'exécution de certaines tâches spécifiques, notamment le scraping web dynamique de l'agent commercial. Pour y remédier, nous avons créé une image Docker personnalisée de n8n en utilisant un `Dockerfile`.

L'objectif principal était d'intégrer la bibliothèque Python **Selenium** ainsi qu'un navigateur web headless, **Chromium**, directement dans le conteneur n8n. Cela permet aux nœuds "Execute Code" d'exécuter des scripts de scraping complexes sans dépendances externes.

Le `Dockerfile` suivant illustre cette personnalisation :

```
# Utiliser l'image officielle de n8n comme base
FROM n8nio/n8n:latest

# Passer temporairement à l'utilisateur root pour
    installer des paquets système
USER root

# Mettre à jour les paquets et installer Chromium et son
    driver
RUN apt-get update && \
    apt-get install -y chromium-driver chromium

# Revenir à l'utilisateur non-privilégié par défaut pour
    des raisons de sécurité
USER node

# Installer les bibliothèques Python nécessaires via pip
RUN pip install selenium beautifulsoup4 pandas
```

Ce fichier est ensuite utilisé dans notre `docker-compose.yml` pour ”construire” (*build*) l’image n8n au lieu de simplement la télécharger, assurant ainsi que notre environnement d’exécution dispose de tous les outils requis.

7.1.3 Gestion Centralisée des Secrets et Configurations

Pour maintenir un système sécurisé et facilement configurable, aucune information sensible (clé d’API, mot de passe, identifiant) n’est écrite en dur dans les workflows ou le code. La gestion de ces secrets est entièrement prise en charge par Docker Compose, via l’utilisation de variables d’environnement.

Un fichier `.env` est placé à la racine du projet. Ce fichier, qui est explicitement exclu du contrôle de version (via `.gitignore`), contient toutes les informations de configuration sous forme de paires clé-valeur :

```
# Fichier .env

# Identifiants de base de données
POSTGRES_USER=n8n_user
POSTGRES_PASSWORD=supersecretpassword

# Clés d'API pour les services externes
SLACK_API_KEY=xoxb-xxxxxxxxxxxxx
ZENDESK_API_TOKEN=xxxxxxxxxxxxxxxxx
GOOGLE_API_KEY=AIzaSyxxxxxxxxxxxxx

# Variables pour le scraping
LINKEDIN_USERNAME=mon_email@exemple.com
LINKEDIN_PASSWORD=mon_mot_de_passe
```

Ce fichier est ensuite chargé par Docker Compose et les variables sont injectées dans les conteneurs respectifs. Ainsi, pour déployer l'ensemble du système sur une nouvelle machine, il suffit de configurer ce unique fichier `.env` puis de lancer la commande `docker-compose up`. Cette méthode garantit non seulement la sécurité des données sensibles mais offre également une expérience de déploiement "prête à l'emploi", renforçant notre contrôle sur l'ensemble de l'écosystème.

7.1.4 Exposition Sécurisée pour le Développement avec ngrok



FIG. 7.2 – Logo de l’outil ngrok.

Bien que l’objectif final soit un système auto-hébergé et isolé, la phase de développement et de test requiert une communication bidirectionnelle avec des services web externes. De nombreuses plateformes que nous intégrons (Shopify, Slack, Zendesk) fonctionnent avec des webhooks, c’est-à-dire qu’elles doivent pouvoir envoyer des notifications à notre instance n8n lorsqu’un événement se produit (une nouvelle commande, un nouveau ticket, etc.). Or, par défaut, notre environnement Docker fonctionnant sur une machine locale (`localhost`) est inaccessible depuis l’Internet public.

Pour résoudre ce problème de connectivité temporaire, nous utilisons l’outil **ngrok**. Ngrok est un service qui crée un tunnel sécurisé entre une URL publique sur Internet et un port spécifique de notre machine locale. En pratique, il nous fournit une adresse web temporaire (par exemple, `https://random-string.ngrok.io`) qui redirige tout le trafic reçu vers le port de notre conteneur n8n (le port 5678).

L’utilisation de ngrok est fondamentale pour plusieurs raisons :

- **Test des Webhooks** : Il nous permet de configurer les webhooks dans les interfaces de Shopify, Slack ou Zendesk en utilisant l’URL publique fournie par ngrok. Nous pouvons ainsi recevoir et tester en temps réel les déclencheurs de nos workflows comme s’ils étaient sur un serveur de production.
- **Validation des API** : Il offre la possibilité de tester les points d’entrée (endpoints) de nos propres API depuis un appareil externe (comme un smartphone) pour valider leur comportement dans des conditions réelles.

- **Démonstrations Client** : Il facilite la présentation de l'avancement du projet à des parties prenantes externes en leur donnant un accès direct et temporaire à l'application web ou à certaines de ses fonctionnalités, sans nécessiter un déploiement complexe.

Ngrok agit donc comme un pont indispensable durant la phase de développement, nous permettant de valider de bout en bout des intégrations complexes avant de passer au déploiement final sur un serveur de production avec une adresse IP publique et un nom de domaine permanents.

7.2 Agent de Supervision

Un agent dédié à la supervision des activités opérationnelles . Son objectif principal est d'automatiser le suivi des rapports journaliers des collaborateurs, garantissant ainsi la conformité et la centralisation des informations pour le management.

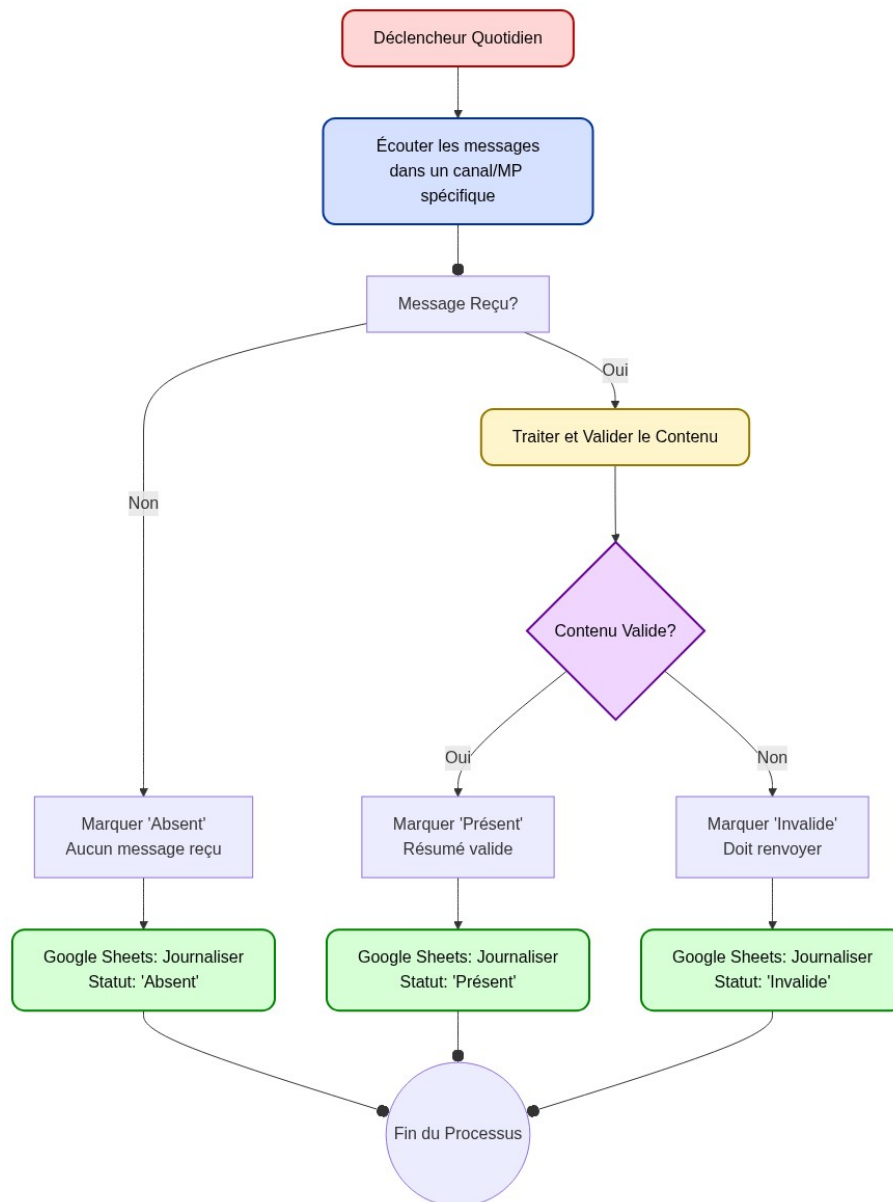


FIG. 7.3 – Schéma fonctionnel de l’Agent de Supervision Opérationnelle et ses interactions avec Slack, le LLM de validation et Google Sheets.

Comme illustré par la Figure 7.3, l’agent de supervision exécute un cycle de tâches critiques :

- Remplissage automatique du document Google Sheets par les noms des collaborateurs avec chaque chaîne dans une feuille précise Et la date actuelle .
- L’écoute active et continue des messages sur un canal de communication d’entreprise (Slack).
- Le traitement sémantique des messages reçus pour valider leur conformité en tant que

rapport d'activité.

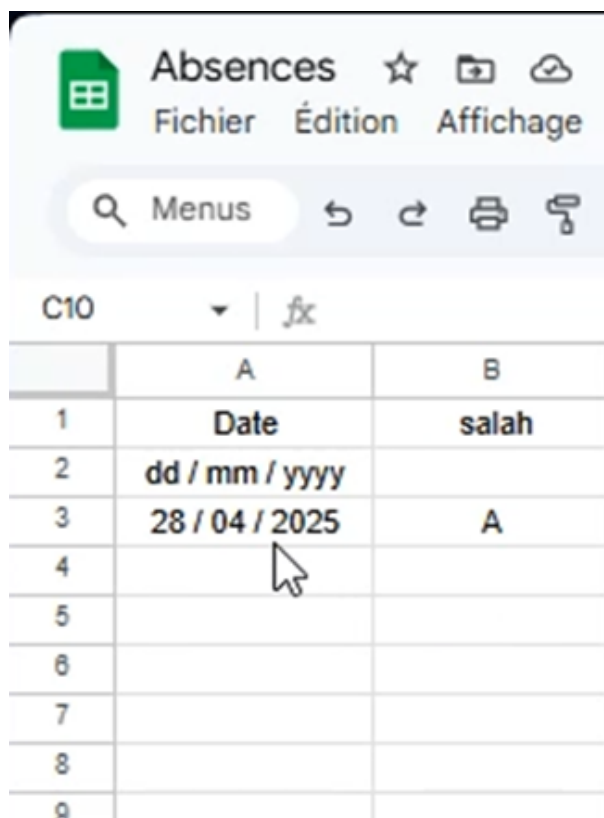
- L'envoi de notifications et de rappels ciblés pour assurer la complétion des tâches.
- La journalisation automatique et structurée des statuts (présent, absent, rapport invalide) dans une base de données centralisée (Google Sheets).

7.2.1 Fonctionnalités Principales

- **Collecte Automatisée des Données** : L'agent se connecte à l'espace de travail Slack et surveille un canal prédéfini. Il est programmé pour collecter tous les messages postés quotidiennement avant une heure butoir.
- **Validation par Traitement du Langage Naturel (NLP)** : Chaque message collecté est analysé par un LLM. L'agent détermine si le contenu constitue un rapport de travail valide selon des critères préétablis. Si le message est non conforme (ex : "Bonjour", "Je serai en retard"), il est identifié comme tel.
- **Système de Notifications Intelligent** : Selon le résultat de l'analyse, l'agent exécute une action :
 - **Rapport Valide** : Le collaborateur est marqué comme "Présent" dans Google Sheets.
 - **Rapport Invalide** : Un message de rappel est envoyé au collaborateur, lui demandant de soumettre un rapport conforme.
 - **Absence de Rapport** : Après l'heure limite, l'agent identifie les collaborateurs n'ayant envoyé aucun message et les marque comme "Absent", tout en pouvant notifier un responsable.

7.2.2 Avantages de l'Agent de Supervision

- **Fiabilisation du Suivi** : L'automatisation élimine les erreurs humaines et les oublis, assurant un suivi des présences et des activités rigoureux et impartial.



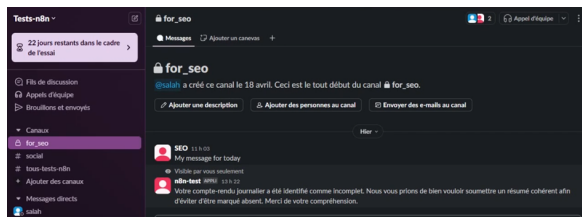
	A	B
1	Date	salah
2	dd / mm / yyyy	
3	28 / 04 / 2025	A
4		
5		
6		
7		
8		
9		

FIG. 7.4 – Exemple de journalisation des statuts dans Google Sheets, automatiquement mise à jour par l’agent.

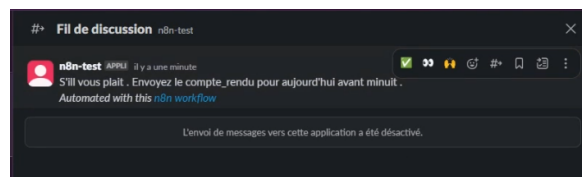
- **Gain de Temps pour le Management :** Les responsables n’ont plus besoin de vérifier manuellement le canal de communication. Ils consultent un tableau de bord unique et toujours à jour, leur permettant de se concentrer sur des tâches à plus forte valeur ajoutée.
- **Amélioration de la Conformité :** Le système de rappels automatiques incite les collaborateurs à respecter les procédures internes, améliorant ainsi la discipline et la communication au sein des équipes.

7.2.3 Exemples de Notifications Automatisées

La communication de l’agent avec les collaborateurs est un aspect fondamental de son efficacité. Les messages envoyés sont conçus pour être clairs, directs et contextuels, afin de guider l’utilisateur sans ambiguïté. Deux scénarios principaux déclenchent une notification, comme illustré dans la Figure 7.5.



(a) Notification pour un rapport jugé non-conforme par le LLM.



(b) Rappel envoyé après l'heure limite en l'absence de rapport.

FIG. 7.5 – Exemples de messages automatisés envoyés par l'agent sur Slack.

- **Gestion du Message Invalide** : Si un collaborateur envoie un message qui n'est pas reconnu comme un rapport d'activité (par exemple, un simple "Bonjour" ou une question), l'agent répond immédiatement en privé. La notification (Figure 7.5a) précise que le message n'a pas été validé et rappelle brièvement le format attendu, invitant l'utilisateur à soumettre un rapport correct.
- **Gestion de l'Absence de Rapport** : Si l'heure limite est dépassée et qu'un collaborateur n'a envoyé aucun message, l'agent lui envoie un rappel (Figure 7.5b). Ce message l'informe qu'il a été manqué d'envoyer son compte rendu pour la journée .

7.2.4 Orchestration du Flux de Travail avec n8n

L'ensemble de la logique de l'agent de supervision est orchestré par un flux de travail sur la plateforme n8n. Cette approche visuelle et modulaire permet une grande flexibilité et facilite la maintenance du processus. Le workflow, représenté dans les figures suivantes est structuré autour de plusieurs nœuds clés.

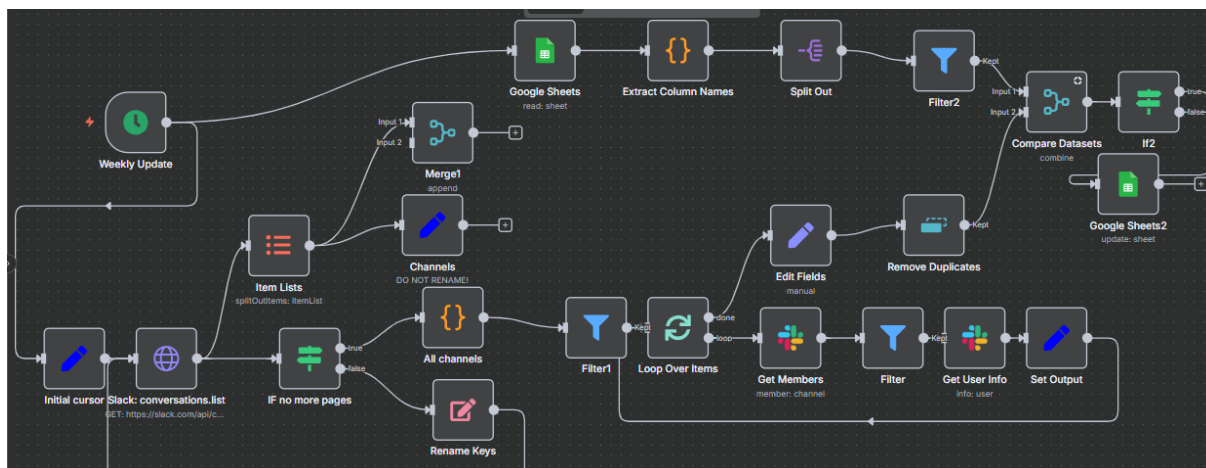


FIG. 7.6 – Vue d’ensemble du workflow n8n pour l’agent de supervision : remplissage des feuilles de document Google avec les noms des participants Slack .

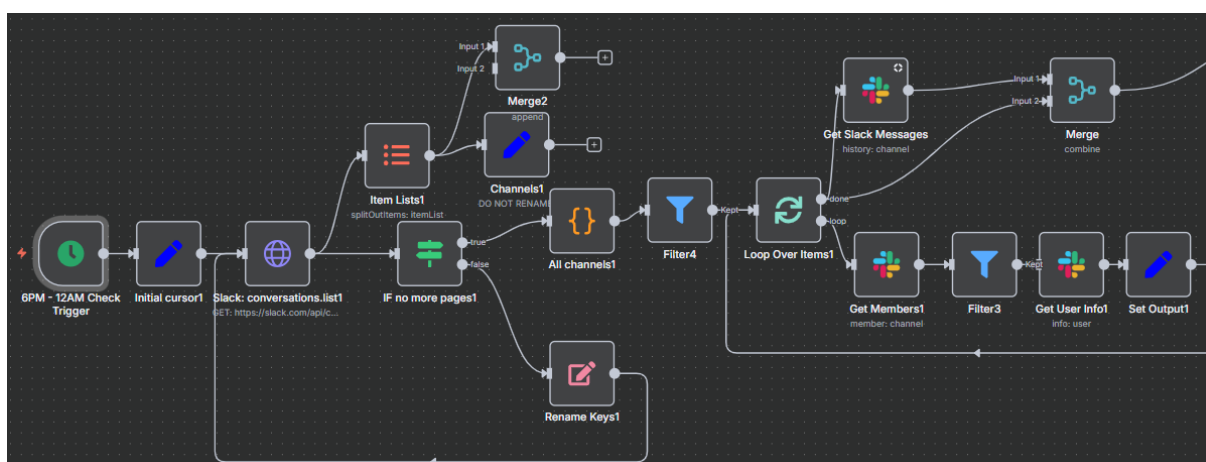


FIG. 7.7 – Vue d’ensemble du workflow n8n pour l’agent de supervision : Récupération des nouveaux messages envoyés par chaîne Slack .

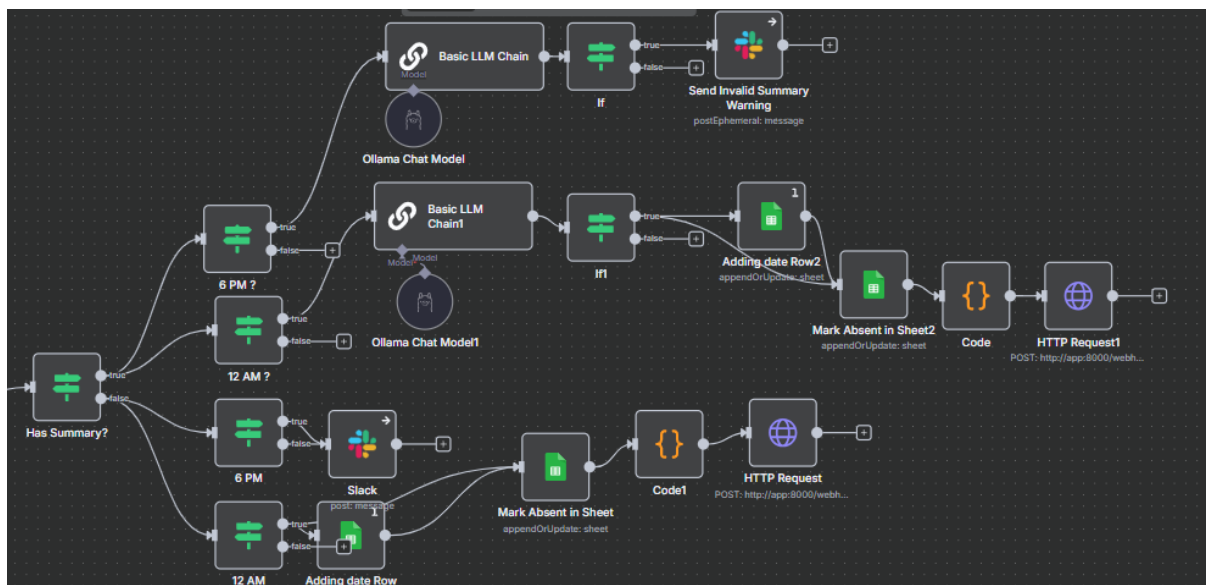


FIG. 7.8 – Vue d’ensemble du workflow n8n pour l’agent de supervision : Validation des messages et marquage d’absence .

La configuration des nœuds principaux est la suivante :

- **Nœud Déclencheur (Cron)** : Le workflow est initié automatiquement à une heure fixe chaque jour ouvré (18h00 et 00h00). Ce nœud agit comme le point de départ de l’ensemble du processus de vérification.
- **Nœud Google Sheets (Get Rows)** : La première étape consiste à récupérer la liste complète des collaborateurs à superviser depuis Slack . Cela permet d’ajuster dynamiquement la liste dans google sheets sans modifier le workflow.
- **Nœud Slack (Read Messages)** : L’agent se connecte ensuite au canal Slack désigné et télécharge l’historique des messages pour la journée en cours.
- **Nœud de Logique (Code / IF)** : C’est le cœur du traitement. Le workflow boucle sur chaque collaborateur récupéré de Google Sheets. Pour chacun, il vérifie si un message correspondant a été trouvé dans les données Slack.
 - Si un message est trouvé, il est transmis à un **nœud LLM** pour validation.
 - Un nœud **IF** aiguille ensuite le flux en fonction de la réponse du LLM (“Valide” ou “Invalide”).

- Si aucun message n’est trouvé, le flux est dirigé vers le statut ”Absent”.
- **Nœuds d’Action (Slack & Google Sheets)** : En fonction de la branche logique suivie, des actions terminales sont exécutées :
 - **Nœud Slack (Send Message)** : Envoie la notification appropriée (message invalide ou rappel d’absence) au collaborateur concerné.
 - **Nœud Google Sheets (Update Row)** : Met à jour la ligne du collaborateur dans la feuille de suivi avec le statut final pour la journée (”Présent”, ”Absent”, ou ”Message Invalide”).

7.2.5 Cœur du Traitement : Le Nœud de l’Agent IA

Au centre de la logique de validation des messages se trouve le nœud de l’Agent IA. Ce composant est le cerveau de l’opération, chargé d’analyser le contenu textuel des messages Slack pour en déterminer la validité. Il s’appuie sur le grand modèle de langage (LLM) Llama 3.1 pour sa capacité de raisonnement et de compréhension du langage naturel.

Architecture du Nœud IA

Le nœud est conçu autour de deux composants principaux : le **Cerveau (Llama 3.1)** qui effectue l’analyse, et un **Stockage Local** qui lui sert de mémoire à court terme pour contextualiser les conversations. Cette architecture, illustrée dans la Figure 7.9, assure un traitement autonome et contextuel de chaque message.

Le flux de traitement est le suivant : un **Message Slack** est soumis au **Cerveau**. Celui-ci consulte le **Stockage Local** pour récupérer le contexte des échanges précédents avec l’employé, si nécessaire. Après analyse, le cerveau produit une **Action**, qui est une classification binaire : ”Valide” ou ”Invalide”. Cette sortie est ensuite utilisée par le workflow n8n pour diriger le flux vers la branche de traitement appropriée.

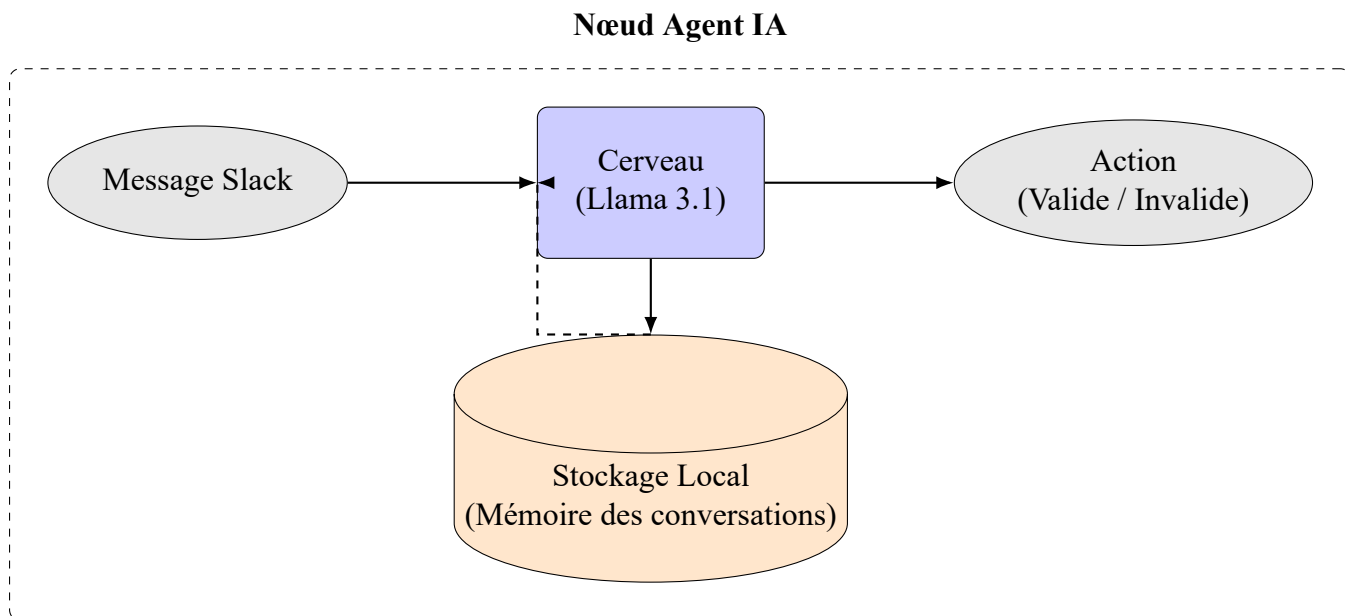


FIG. 7.9 – Architecture du Nœud de l’Agent IA, montrant l’interaction entre le message entrant, le cerveau LLM et le stockage local.

Configuration du Nœud et des Paramètres LLM

Le comportement de l’agent IA est finement contrôlé par une série de paramètres qui définissent à la fois le fonctionnement du nœud et la manière dont le modèle Llama 3.1 génère ses réponses.

Configuration du Nœud. Ces paramètres assurent l’intégration correcte du nœud au sein du workflow.

Paramètre	Description
llm_model	Définit le modèle de langage à utiliser. Dans notre cas, Llama 3.1.
storage_path	Spécifie le chemin d’accès local où l’historique des conversations est sauvegardé.
max_context_length	Limite le nombre de messages précédents à considérer comme contexte pour préserver les ressources de calcul.

TAB. 7.1 – Paramètres de configuration du Nœud de l’Agent IA.

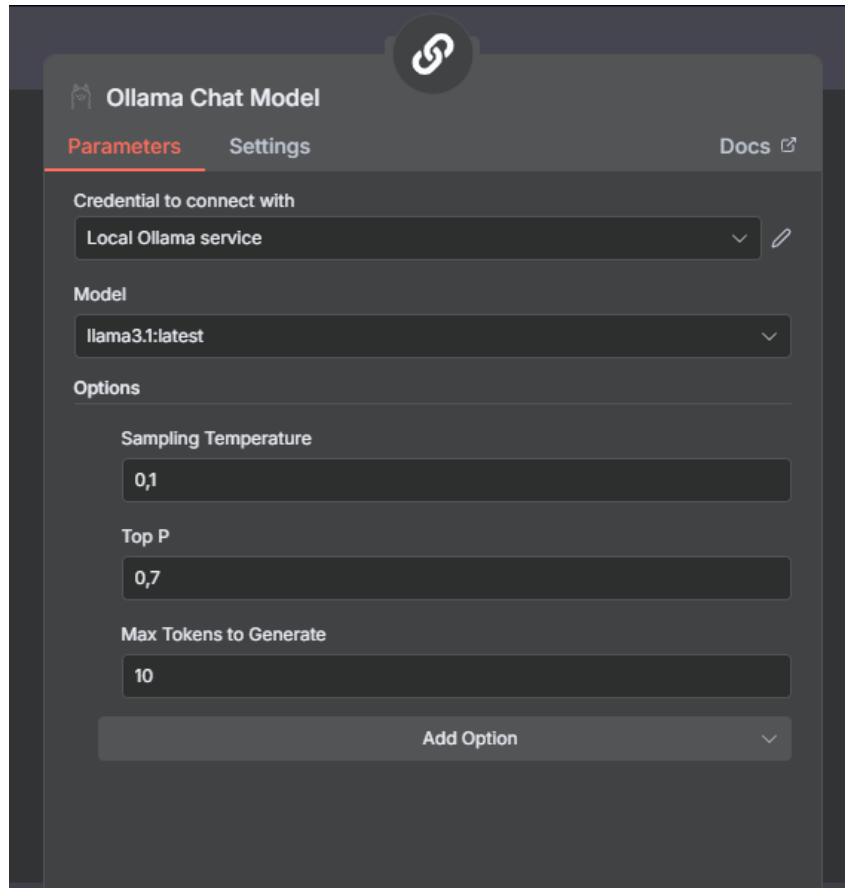


FIG. 7.10 – Configuration LLM pour l’agent de supervision

Paramètres du LLM. Ces hyperparamètres ajustent la manière dont le LLM génère son analyse visant une fiabilité et une cohérence maximales.

- **Prompt Système :** Un prompt initial est fourni au LLM pour cadrer sa mission. Il lui est demandé d’agir en tant qu’assistant de supervision dont le but est d’évaluer la validité des résumés de travail selon des critères précis (présence de verbes d’action, description de tâches, etc.).
- **Température :** Ce paramètre est réglé sur une valeur basse (ex : 0.1) pour rendre les réponses du modèle plus déterministes et factuelles. Cela réduit la créativité et favorise la précision, ce qui est essentiel pour une tâche de classification.
- **Top-p :** Maintenu à une valeur élevée (ex : 0.95), l’échantillonnage nucleus garantit que le modèle se base sur un vocabulaire probable et cohérent, évitant les réponses trop originales ou hors sujet.

- **Max Tokens** : Le nombre maximal de jetons (tokens) générés est volontairement limité pour forcer le modèle à produire une réponse concise (par exemple, uniquement "Valide" ou "Invalide"), optimisant ainsi la vitesse de traitement.

7.3 Agent d'Assistance à la Gestion des Tickets de Support

La gestion des tickets de support client est une tâche cruciale mais souvent répétitive. Pour optimiser ce processus, nous avons développé un agent intelligent capable de s'intégrer à des plateformes de service client comme Zendesk. L'objectif principal de cet agent est d'automatiser la classification initiale des tickets et de fournir aux agents humains des suggestions de réponse pertinentes, réduisant ainsi les délais de traitement et uniformisant la qualité du support.

7.3.1 Architecture et Principe de Fonctionnement

L'agent fonctionne en continu, surveillant l'arrivée de nouveaux tickets. Dès qu'un ticket est créé, son contenu (sujet et description) est analysé par un LLM qui exécute deux tâches simultanément : la classification du problème et la génération d'une proposition de réponse.

Le diagramme de la Figure 7.11 illustre ce flux de traitement.

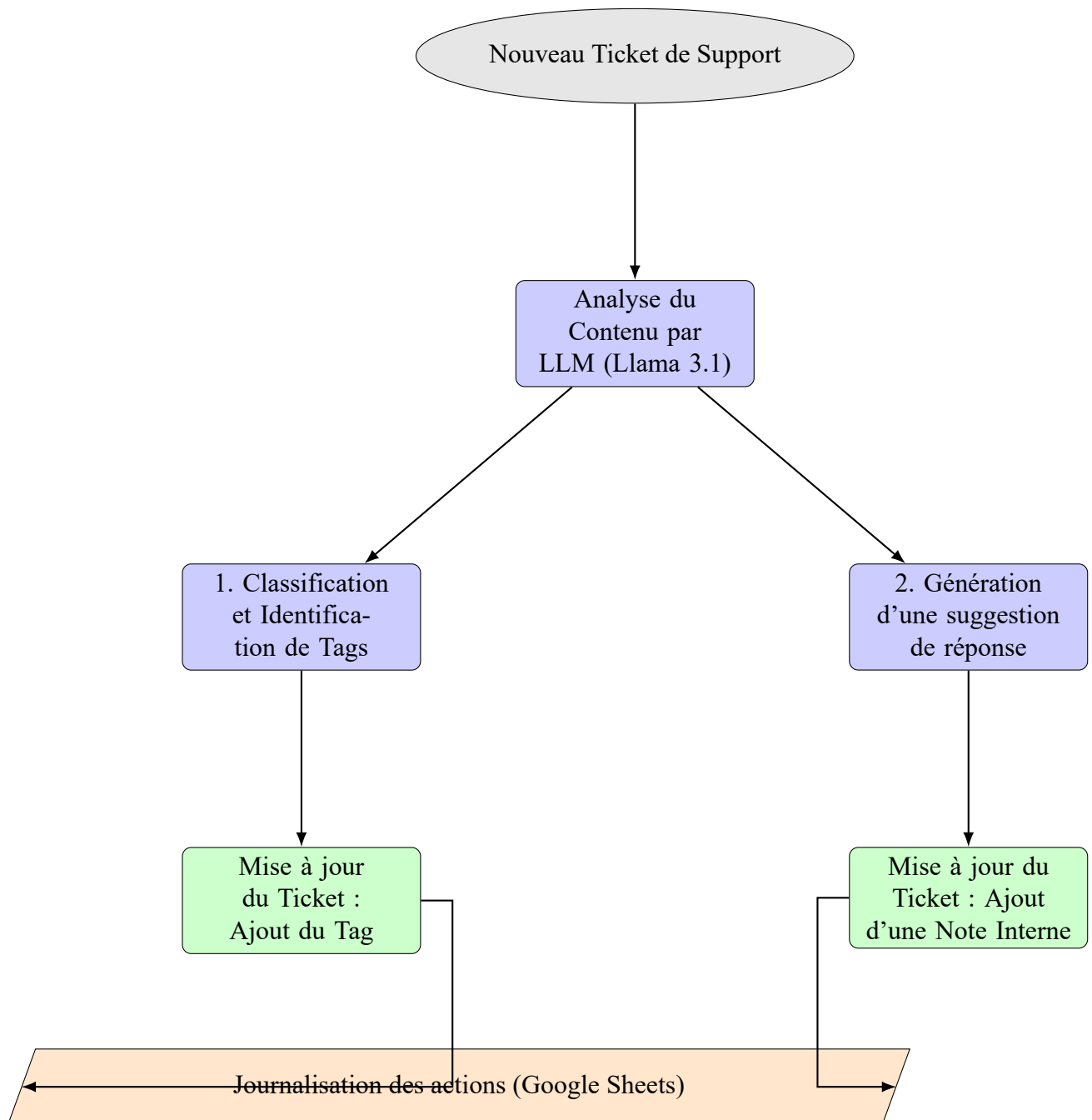


FIG. 7.11 – Diagramme fonctionnel de l'agent de support client.

Le processus se décompose comme suit :

1. **Écoute** : L'agent est connecté à l'API de la plateforme de support et est notifié de la création de chaque nouveau ticket.
2. **Analyse sémantique** : Le contenu du ticket est transmis au LLM, qui est pré-configuré avec un prompt spécifique lui demandant d'identifier la nature du problème (ex : "pro-

blème de facturation”, ”question technique”, ”demande de fonctionnalité”) et de proposer un tag correspondant.

3. **Génération de suggestion** : En se basant sur cette classification et sur le contenu du ticket, le LLM rédige une ébauche de réponse sous forme de note interne, invisible pour le client. Cette suggestion peut contenir des liens vers la documentation, des questions de clarification ou une solution directe.
4. **Mise à jour** : L’agent utilise l’API de la plateforme pour automatiquement ajouter le tag identifié et publier la suggestion en tant que note interne dans le ticket.
5. **Journalisation** : Toutes les opérations (ID du ticket, tag ajouté, suggestion générée) sont enregistrées dans une feuille Google Sheets à des fins de suivi et d’analyse de performance.

7.3.2 Orchestration n8n

L’ensemble de ce processus est orchestré par un workflow n8n dédié, assurant la robustesse et la modularité de l’agent. Les figures suivants présente une vue d’ensemble de ce workflow.

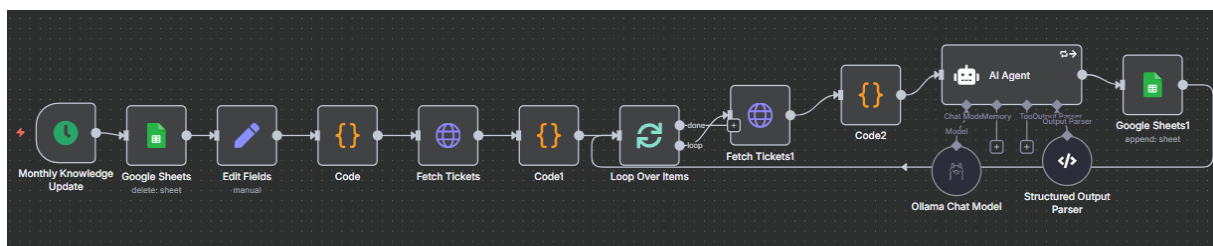


FIG. 7.12 – Workflow n8n pour l’agent de support : Remplissage de la base des bases connaissances .

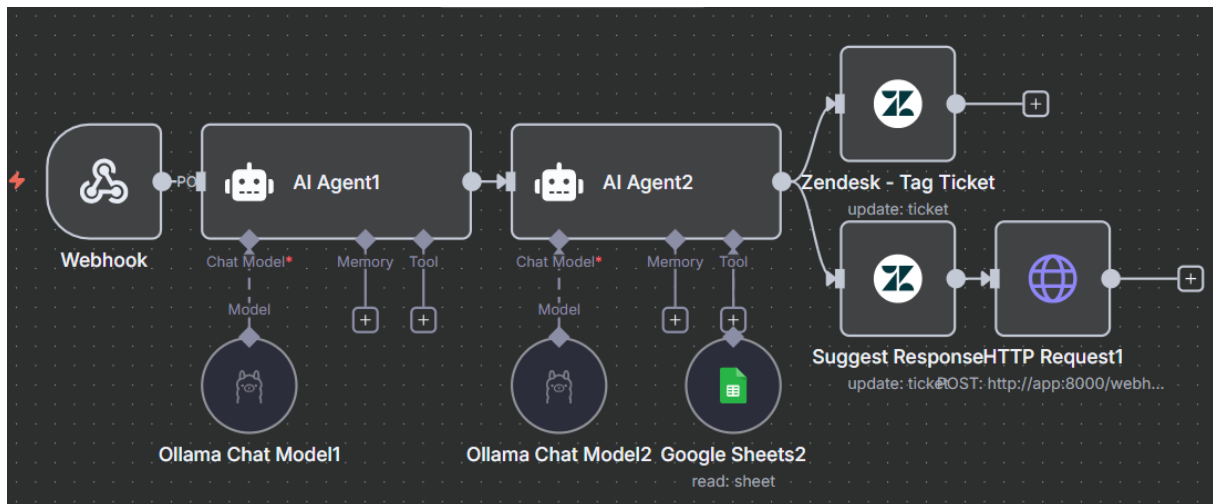


FIG. 7.13 – Workflow n8n pour l’agent de support : Obtention des tickets depuis webhook , classification et génération de note interne .

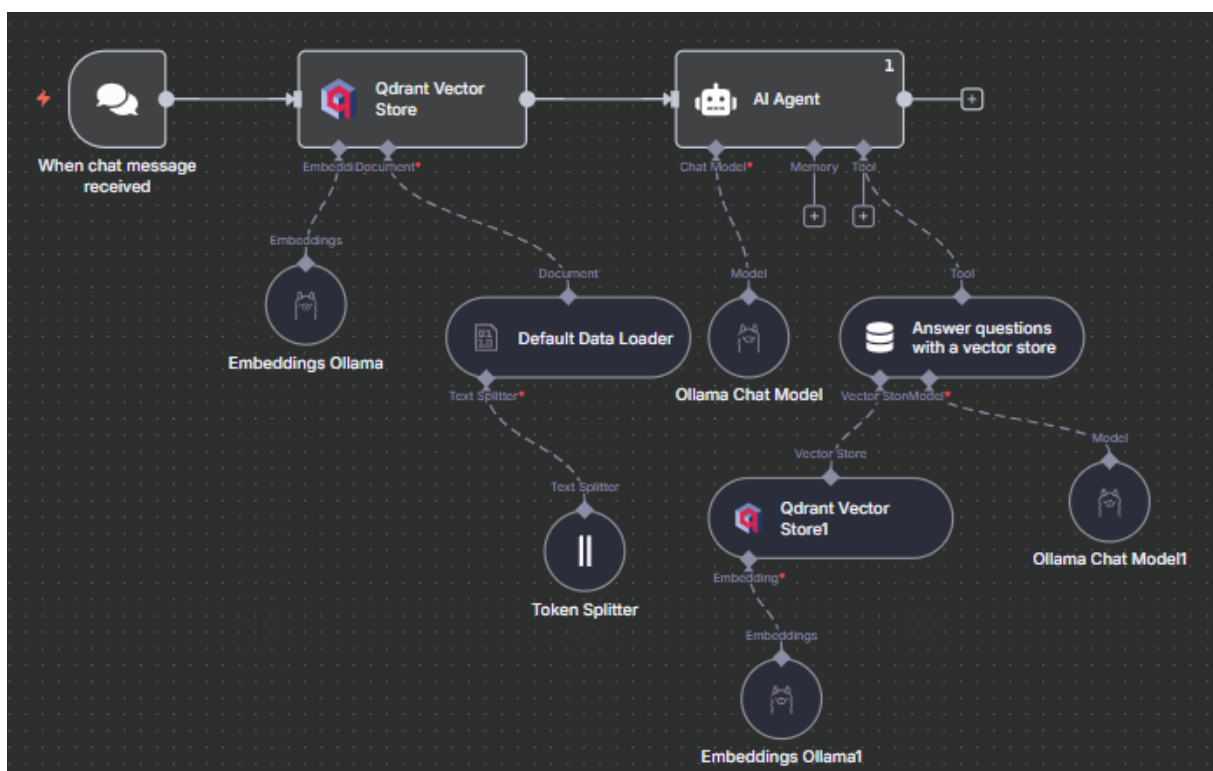


FIG. 7.14 – Utilisation du base des connaissances pour répondre aux questions des agents humains

Les nœuds principaux de ce workflow sont :

- **Nœud Déclencheur (Zendesk Trigger) :** S’active instantanément à la création d’un nouveau ticket.

- **Nœud LLM (Classification) :** Reçoit le texte du ticket et retourne un objet JSON contenant le tag suggéré (ex : “tag” : ”facturation”).
- **Nœud LLM (Suggestion) :** Reçoit le texte du ticket ainsi que le tag, et génère une proposition de réponse textuelle.
- **Nœud Zendesk (Update Ticket) :** Met à jour le ticket avec les informations générées par les nœuds LLM.
- **Nœud Google Sheets (Append Row) :** Ajoute une nouvelle ligne dans le journal de suivi.

7.3.3 Constitution de la Base de Connaissances par Apprentissage Rétrospectif

Pour que les suggestions de l’agent soient non seulement génériques mais aussi profondément ancrées dans l’historique des problèmes spécifiques à l’entreprise, une phase d’initialisation est cruciale. Cette étape consiste à construire une base de connaissances vectorielle à partir des tickets déjà résolus. Cette base de données servira de mémoire à long terme pour l’agent, lui permettant d’utiliser une approche de *Retrieval-Augmented Generation* (RAG) pour formuler des réponses plus précises.

Le processus de création de cette base de connaissances, schématisé dans la Figure 7.15, est une opération effectuée en amont du déploiement de l’agent.

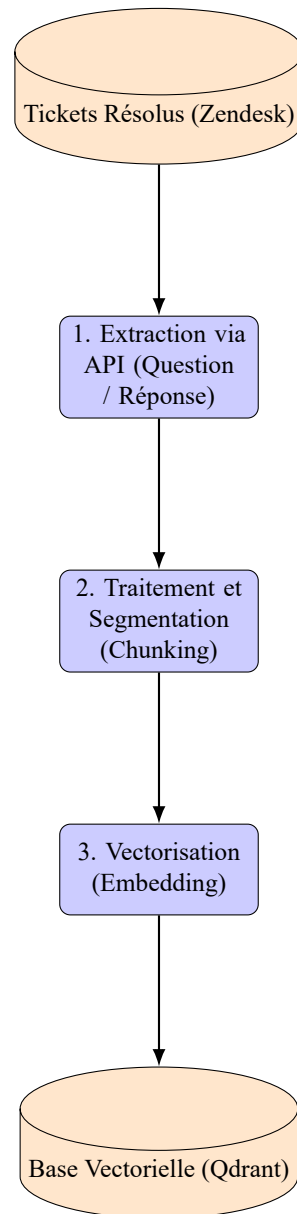


FIG. 7.15 – Processus de création de la base de connaissances vectorielle.

Ce processus se déroule en quatre étapes fondamentales :

1. **Extraction des Données Historiques** : En utilisant l'API de Zendesk, nous extrayons un volume significatif de tickets dont le statut est "Résolu". Pour chaque ticket, nous isolons les informations pertinentes : le message initial du client (la question) et la réponse finale de l'agent de support qui a mené à la résolution (la solution).
2. **Traitement et Segmentation (Chunking)** : Les paires question/solution ainsi formées sont nettoyées pour retirer les éléments superflus (signatures, formules de politesse répé-

titives, etc.). Si les textes sont longs, ils sont divisés en segments (chunks) plus petits et sémantiquement cohérents. Cette segmentation est essentielle pour garantir une vectorisation précise et pertinente.

3. **Vectorisation (Embedding) :** Chaque segment de texte traité est ensuite passé à travers un modèle d'embedding . Ce modèle convertit le texte en un vecteur numérique de haute dimension qui capture son sens sémantique. Le résultat est que des questions ou des solutions similaires auront des vecteurs proches dans l'espace vectoriel.
4. **Indexation dans Qdrant :** Finalement, ces vecteurs, accompagnés de leur contenu textuel original et de métadonnées utiles (comme l'ID du ticket d'origine), sont stockés et indexés dans une base de données vectorielle, Qdrant. Qdrant est optimisé pour effectuer des recherches de similarité ultra-rapides, ce qui permettra à l'agent de retrouver quasi-instantanément les tickets passés les plus pertinents par rapport à une nouvelle question.

Une fois cette base de connaissances constituée, l'agent peut l'exploiter en temps réel. Lorsqu'un nouveau ticket arrive, son contenu est d'abord vectorisé, puis utilisé pour interroger Qdrant. Les solutions des tickets historiques les plus similaires sont récupérées et injectées dans le prompt du LLM. Cette information contextuelle ("augmented generation") permet au modèle de générer une suggestion de réponse beaucoup plus informée et adaptée que s'il se basait uniquement sur ses connaissances générales.

7.3.4 Résultats en Pratique

Le fondement de notre base de connaissances repose sur l'extraction et la simplification des tickets historiques. Chaque interaction est soigneusement structurée sous la forme d'une paire question/réponse claire.

Ticket ID	Question	Answer
1		
2	Are OFFZONE's products and materials sustainably and ethically sourced?	This information is not currently available in our knowledge base. Please contact our sustainability team at sustainability@offzone.com for detailed information about our sourcing practices and ethical standards for specific products.
3	How can I change the delivery address on my order after it's been placed?	To change the delivery address on your order, please contact our shipping coordinator Salah directly via phone. There is no option to modify the shipping address through the website after an order has been placed. Contact should be made as soon as possible as there may be limitations on when address changes can be processed.
4	How are shipping costs calculated and what factors influence them?	Shipping costs are determined by several factors including the package weight, dimensions, destination, and selected shipping method. While we don't currently offer a pre-purchase shipping calculator, you can view the exact shipping costs during checkout after entering your shipping address but before completing the purchase. This allows you to review all fees before finalizing your order.

FIG. 7.16 – Extrait de la base de connaissances structurée au format tabulaire, montrant les paires "Question/Réponse" extraites des tickets historiques

L'intégration de cet agent transforme l'interface de la plateforme pour l'agent humain. Comme le montre la Figure 7.17, le ticket arrive dans la file d'attente déjà trié avec un tag pertinent et enrichi d'une note interne.

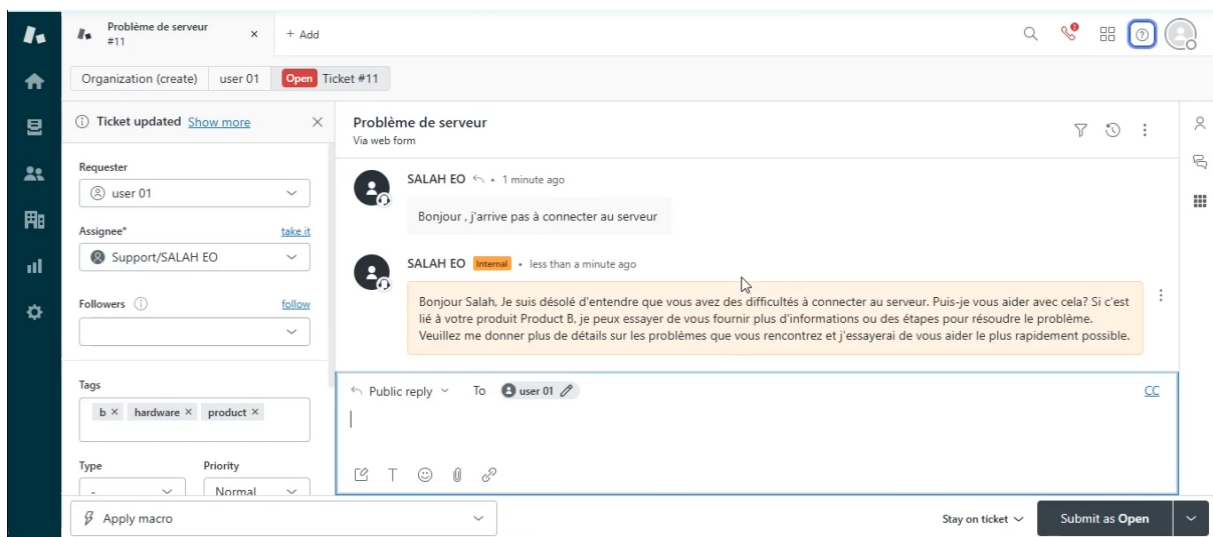


FIG. 7.17 – Exemple d'un ticket traité par l'agent dans Zendesk. On observe le tag "Hardware" ajouté automatiquement (1) et la note interne avec la suggestion de réponse pour l'agent (2).

L'agent humain peut alors immédiatement prendre connaissance de la suggestion, la valider, la modifier ou l'ignorer, ce qui représente un gain de temps considérable sur les tâches de diagnostic initial.

7.4 Agent de Prospection Commerciale

La génération de prospects qualifiés (lead generation) est un pilier de la croissance commerciale, mais elle est souvent manuelle et chronophage. Pour répondre à ce défi, nous avons conçu un agent de prospection commerciale dont la mission est d'automatiser l'ensemble du cycle, depuis l'identification de nouvelles cibles jusqu'à la prise de contact initiale. Cet agent combine la recherche ciblée, l'enrichissement de données par scraping et l'engagement automatisé pour construire un flux continu de prospects pertinents.

7.4.1 Architecture Globale du Processus de Prospection

L'agent suit un processus séquentiel et logique, conçu pour filtrer et qualifier progressivement les contacts avant d'initier toute communication. Ce flux garantit que seules les cibles les plus prometteuses sont contactées, maximisant ainsi l'efficacité des campagnes. Le diagramme de la Figure 7.18 schématise les grandes étapes de ce processus.

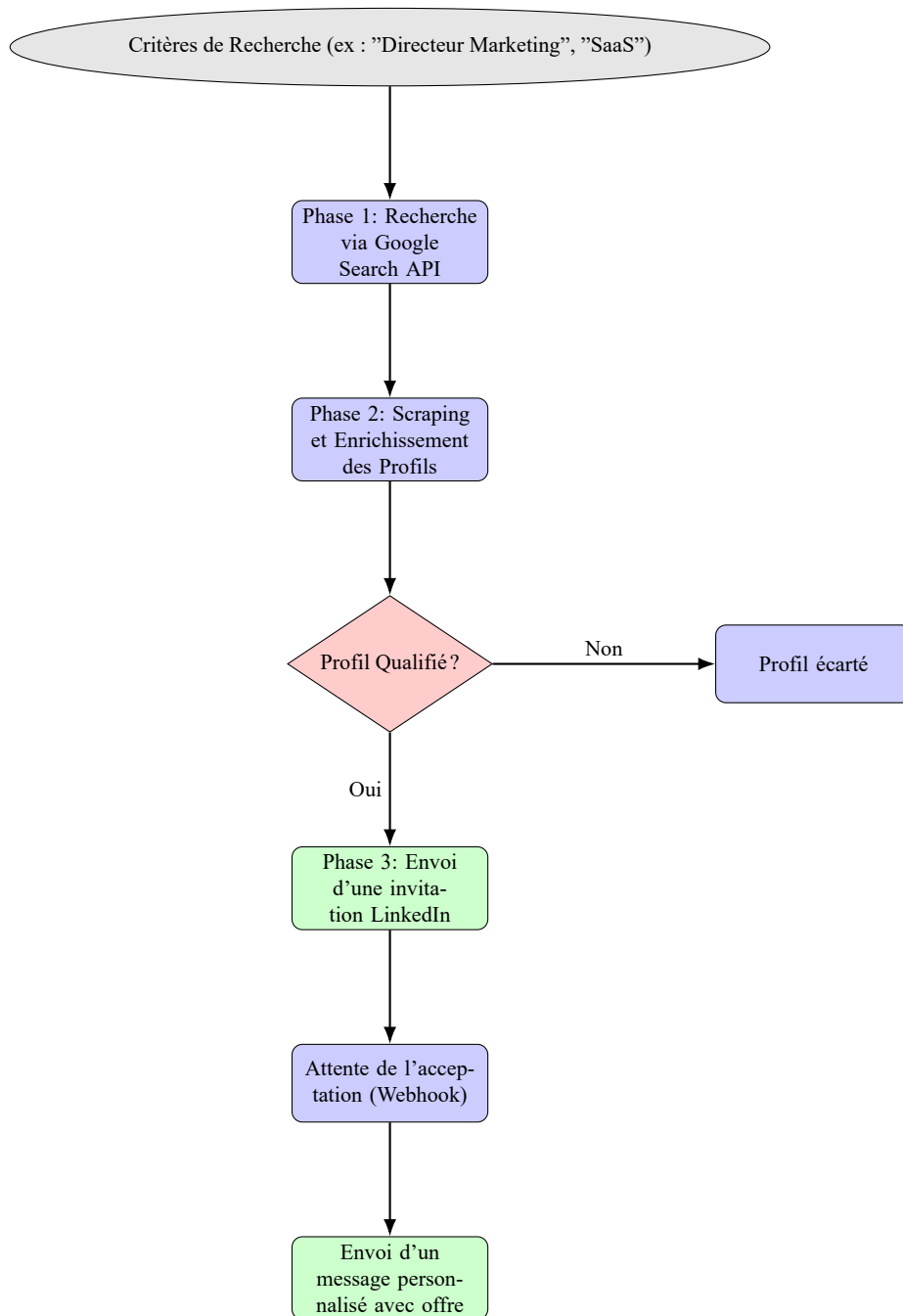


FIG. 7.18 – Diagramme fonctionnel de l'agent de prospection commerciale automatisée.

7.4.2 Phase 1 : Identification des Cibles via l'API Google Search

La première étape consiste à identifier une liste brute de prospects potentiels. Plutôt que de s'appuyer sur les outils de recherche internes et parfois limités des plateformes sociales, nous utilisons l'API Google Search pour sa puissance et sa flexibilité.

L'accès programmatique à Google Search nous permet de construire des requêtes extrêmement précises en utilisant des opérateurs de recherche avancée. La stratégie principale consiste à cibler un site spécifique, en l'occurrence LinkedIn, pour y trouver des profils correspondant à nos critères. Une requête type envoyée à l'API pourrait ressembler à ceci :

```
site:linkedin.com/in/ "CEO" "Paris" "AI"
```

Cette commande demande à Google de ne retourner que les résultats provenant des profils publics LinkedIn ('site :linkedin.com/in/') qui contiennent les mots-clés exacts spécifiés. L'API renvoie alors une liste d'URLs de profils LinkedIn, qui constitue notre liste de prospects initiaux à investiguer. Cette méthode est particulièrement efficace pour découvrir des profils qui ne seraient pas facilement accessibles via une recherche manuelle.

Pour garantir la flexibilité de l'agent et permettre son utilisation par des équipes non techniques, les critères de recherche ne sont pas codés en dur. Nous avons mis en place un formulaire de configuration dédié, présenté dans la Figure 7.19. Via cette interface, un utilisateur, tel qu'un commercial, peut définir précisément les paramètres de sa campagne de prospection en renseignant des champs comme le poste recherché, le secteur d'activité, la localisation géographique ou la taille de l'entreprise. Ces informations sont ensuite automatiquement compilées par le système pour construire la requête de recherche avancée qui sera envoyée à l'API Google. Cette approche découple la complexité technique de l'utilisation pratique, rendant l'agent adaptable à différents besoins sans modifier le workflow.

Agent LinkedIn

Position

CEO HR ...

Location

Morocco France casablanca ...

Domaine

ai dev ...

Domaines non considérées

-medecine -finance

Submit

FIG. 7.19 – Interface de configuration des campagnes pour l’agent Commercial.

7.4.3 Phase 2 : Enrichissement et Qualification des Données

Une URL seule ne suffit pas pour qualifier un prospect. Il est nécessaire de collecter des informations supplémentaires. Pour ce faire, un workflow de scraping est initié pour chaque URL collectée. Ce processus visite la page du profil et en extrait des données structurées telles que le nom complet, l’entreprise actuelle, l’intitulé du poste, la localisation, et d’autres informations pertinentes sur l’entreprise (taille, secteur, etc.).

La Figure 7.20 détaille les étapes de ce micro-processus d’extraction de données.

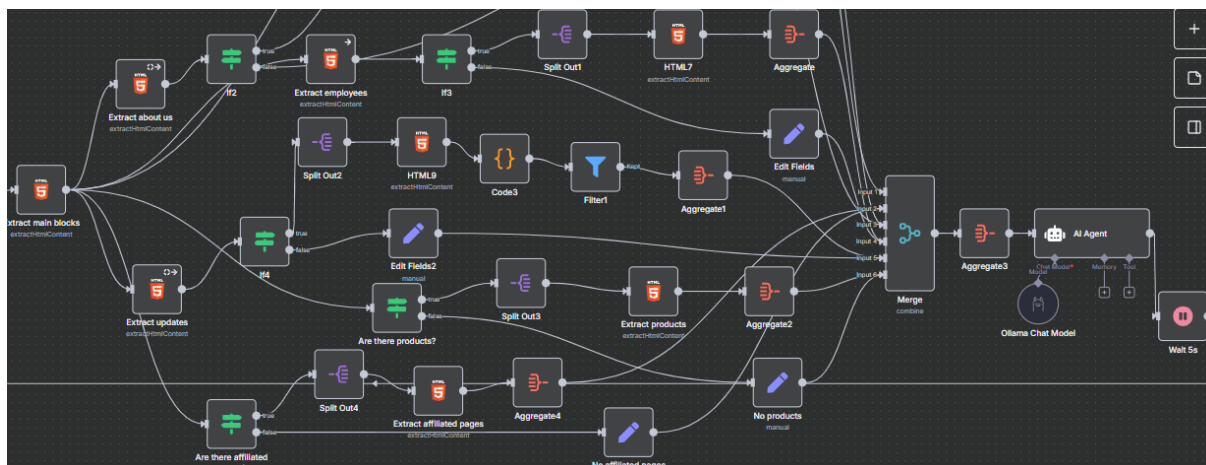


FIG. 7.20 – Extrait du workflow de scraping pour l’enrichissement des données de profil de l’entreprise.

Une fois le profil enrichi, un ensemble de règles de qualification est appliqué. Ces règles, définies en amont (par exemple, l’entreprise doit avoir plus de 50 employés, être dans le secteur technologique, et le contact doit avoir un poste de direction), permettent de comparer les données extraites aux critères de notre client idéal. Si le profil correspond, il est marqué comme ”qualifié” ; sinon, il est écarté.

7.4.4 Phase 3 : Séquence d’Engagement Automatisé via l’Automatisation du Navigateur

Une fois qu’un prospect est qualifié, la phase d’engagement est initiée. L’utilisation des API officielles de LinkedIn pour l’envoi de messages est extrêmement restreinte et principalement réservée à des cas d’usage très spécifiques, excluant la prospection à grande échelle. Pour contourner cette limitation et éviter les blocages de compte, nous avons opté pour une approche de contrôle direct du navigateur (*browser automation*) en utilisant la bibliothèque Selenium. Cette méthode simule de manière fidèle le comportement d’un utilisateur humain, rendant l’automatisation difficilement détectable.

Le Rôle de Selenium dans l'Interaction

Selenium est un framework qui fournit une interface de programmation pour contrôler un navigateur web. Au lieu d'envoyer des requêtes à une API, notre agent prend le contrôle d'une instance de navigateur (par exemple, Chrome ou Firefox, souvent en mode "headless" pour s'exécuter en arrière-plan sur un serveur) et exécute les actions comme le ferait une personne : cliquer sur des boutons, remplir des champs de texte et naviguer entre les pages.

Analyse de la Structure HTML et des Sélecteurs CSS

Le succès de cette approche repose sur notre capacité à identifier de manière fiable les éléments interactifs (boutons, zones de texte) sur les pages de LinkedIn. Pour ce faire, nous effectuons une analyse approfondie de la structure HTML des pages et des styles CSS qui leur sont appliqués.

Chaque élément d'une page web possède des attributs uniques (comme un `id` ou des `class`) qui peuvent être utilisés comme des "adresses" pour que Selenium puisse les localiser. Le processus est le suivant :

1. **Identification des Sélecteurs Stables** : Nous analysons le code source de la page d'un profil LinkedIn pour trouver les sélecteurs CSS ou les expressions XPath les plus stables pour les éléments clés. Par exemple, le bouton "Message" peut être identifié par une combinaison de sa balise HTML (`button`) et d'un attribut spécifique comme `aria_label`. Un sélecteur pourrait ressembler à `button[aria_label="Message à"]`.
2. **Localisation et Clic** : Le script Selenium reçoit l'instruction de trouver l'élément correspondant à ce sélecteur. Une fois localisé, il simule un clic dessus, ce qui ouvre la fenêtre de messagerie de LinkedIn.
3. **Composition du Message** : À l'intérieur de la fenêtre de messagerie, un nouveau champ de texte (`textarea`) est présent. Nous utilisons un autre sélecteur pour identifier cette zone de saisie. Le message personnalisé, généré en amont avec les variables du prospect, est alors "tapé" dans ce champ par le script.

4. **Envoi** : Finalement, le bouton "Envoyer" de la messagerie est localisé via son propre sélecteur et le script simule un dernier clic pour expédier le message.

Stratégies Anti-Détection et Fiabilité

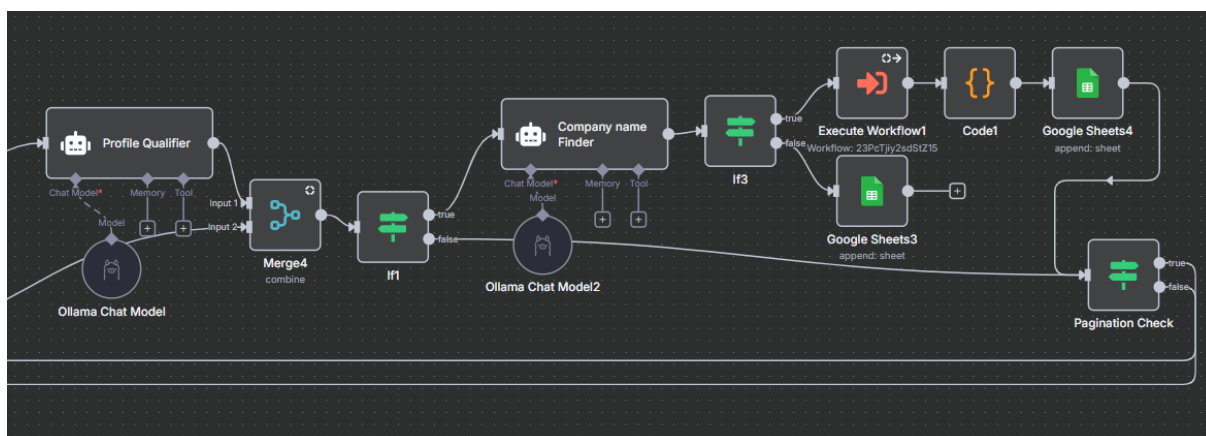
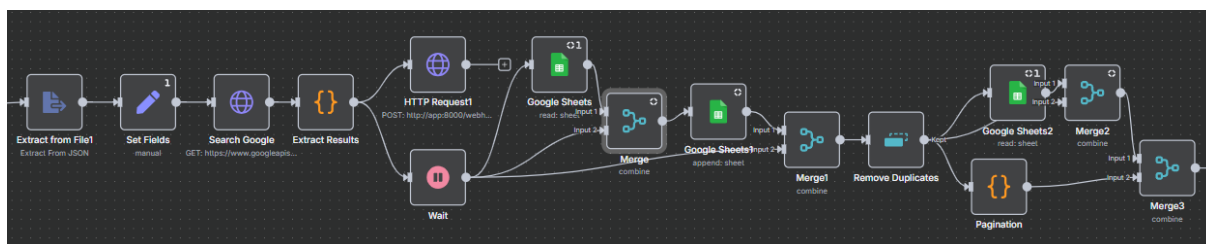
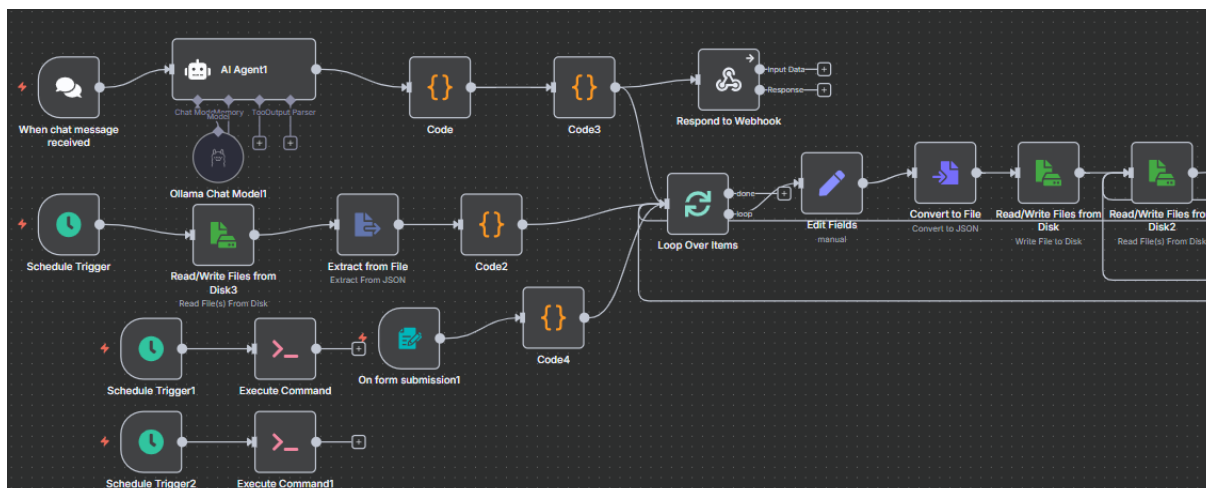
L'automatisation via le navigateur doit être mise en œuvre avec précaution pour éviter d'être détectée comme un comportement de bot. Plusieurs stratégies sont employées :

- **Temporisation Aléatoire** : Des délais aléatoires (quelques secondes) sont introduits entre chaque action (chargement de la page, clic, saisie de texte). Cela imite le rythme d'un utilisateur humain et évite d'exécuter des actions à une vitesse surhumaine.
- **Gestion de Session** : L'agent se connecte une seule fois et sauvegarde les cookies de session. Ces cookies sont réutilisés pour les actions futures, évitant ainsi des connexions répétées qui pourraient être suspectes.
- **Maintenance des Sélecteurs** : L'inconvénient majeur de cette approche est sa dépendance à la structure du site de LinkedIn. Une mise à jour du design de LinkedIn peut modifier les sélecteurs et "casser" l'automatisation. Une maintenance régulière est donc nécessaire pour mettre à jour les sélecteurs et garantir le bon fonctionnement de l'agent.

Grâce à cette technique avancée, l'agent de prospection peut envoyer des invitations et des messages de manière autonome et fiable, dépassant les contraintes imposées par les API et permettant une automatisation complète et efficace du cycle de prise de contact.

7.4.5 Orchestration du Workflow avec n8n

L'ensemble de ces phases, de la recherche Google à l'envoi du message final, est orchestré et automatisé au sein d'un workflow n8n central. Cette plateforme nous permet de connecter les différentes briques logicielles (API Google, outil de scraping, API LinkedIn) et de gérer la logique conditionnelle (qualification), les délais (attente d'acceptation) et la journalisation de chaque étape.



7.5 Agents Spécialisés pour le E-Commerce

Le secteur du e-commerce, en particulier le modèle de paiement à la livraison (Cash on Delivery - COD) prévalent au Maroc, présente des défis logistiques et communicationnels uniques. Pour répondre à ces enjeux, nous avons développé des agents spécialisés visant à automatiser les phases critiques du cycle de vie d'une commande : la confirmation post-achat et le suivi de la livraison.

7.5.1 Agent de Confirmation Proactive des Commandes

L'un des principaux obstacles dans le modèle COD est le taux élevé d'annulation ou d'échec de livraison dû à un manque d'engagement du client ou à des informations erronées. L'agent de confirmation a pour objectif de qualifier et de confirmer chaque nouvelle commande de manière rapide et efficace, maximisant ainsi les chances de réussite de la livraison.

Principe de Fonctionnement

Dès qu'une nouvelle commande est enregistrée sur la plateforme e-commerce (ex : Shopify, WooCommerce), l'agent déclenche une séquence de communication automatisée. L'objectif est d'obtenir une confirmation verbale ou écrite du client en moins de deux heures. Le flux opérationnel est illustré dans la Figure 7.24.

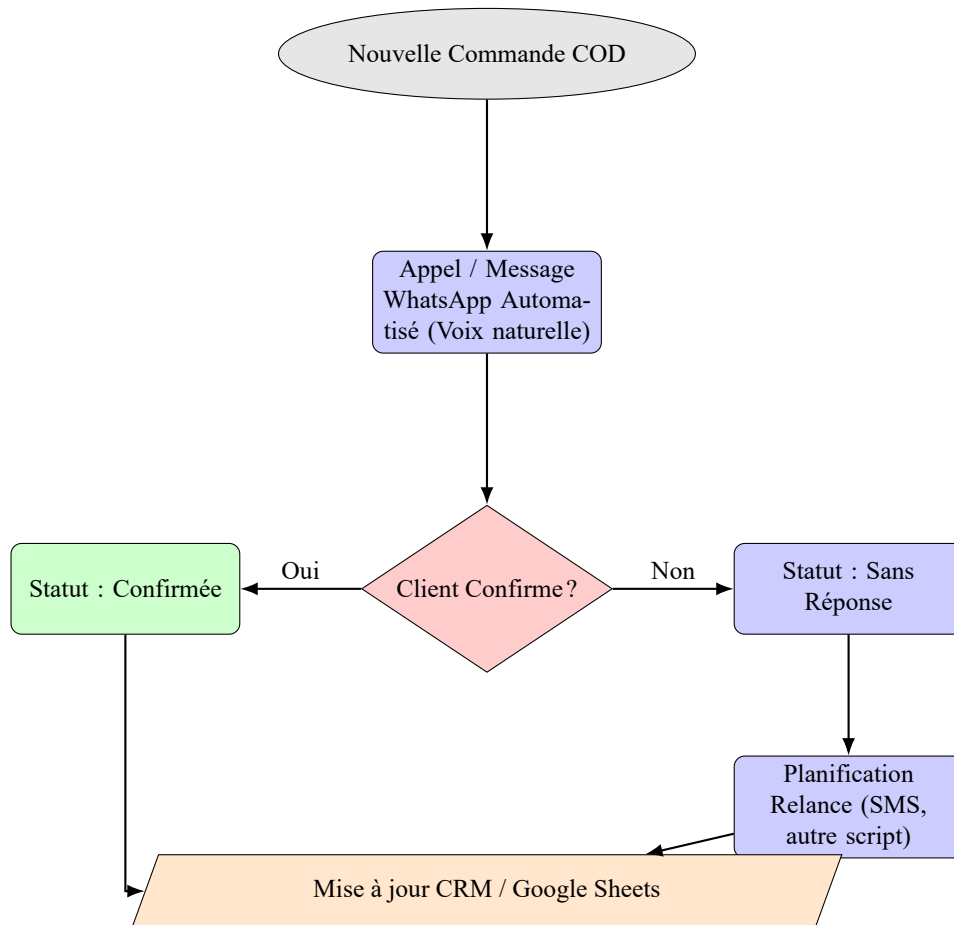


FIG. 7.24 – Diagramme fonctionnel de l’agent de confirmation de commande.

Le processus se déroule comme suit :

1. **Déclenchement** : Un webhook notifie l’agent de la nouvelle commande.
2. **Prise de Contact Initiale** : Un appel téléphonique ou un message vocal WhatsApp est automatiquement envoyé au client. Le script, énoncé par une voix de synthèse naturelle, demande au client de confirmer son intention d’achat et de valider ses coordonnées (adresse, disponibilité).
3. **Qualification** : Si le client confirme, l’agent met à jour le statut de la commande à "Confirmée" dans le système de suivi.
4. **Relances Automatisées** : En cas d’échec (pas de réponse, numéro invalide), l’agent planifie une séquence de relances utilisant différents canaux (SMS, messages WhatsApp) et des scripts variés à des moments opportuns.

5. **Journalisation et Analyse :** Toutes les interactions et les changements de statut sont enregistrés en temps réel dans une feuille Google Sheets. De plus, les raisons des annulations sont collectées pour générer des rapports d'optimisation commerciale.

	A	B	C	D	E	F	G	H	I	J	K
1	Order Id	Order	Price	Name	Phone	Address	Email	Statut			
2	450789469	n, iPod Nano - 8gb - red, i	597.00	Salah Eddine	+212643736231	Chestnut Street 92	man@mail exam	non confirmé			
3											
4											
5											
6											
7											
8											
9											

FIG. 7.25 – Journal de suivi des confirmations sur Google Sheets, avec le statut de chaque commande.

Orchestration et Outils

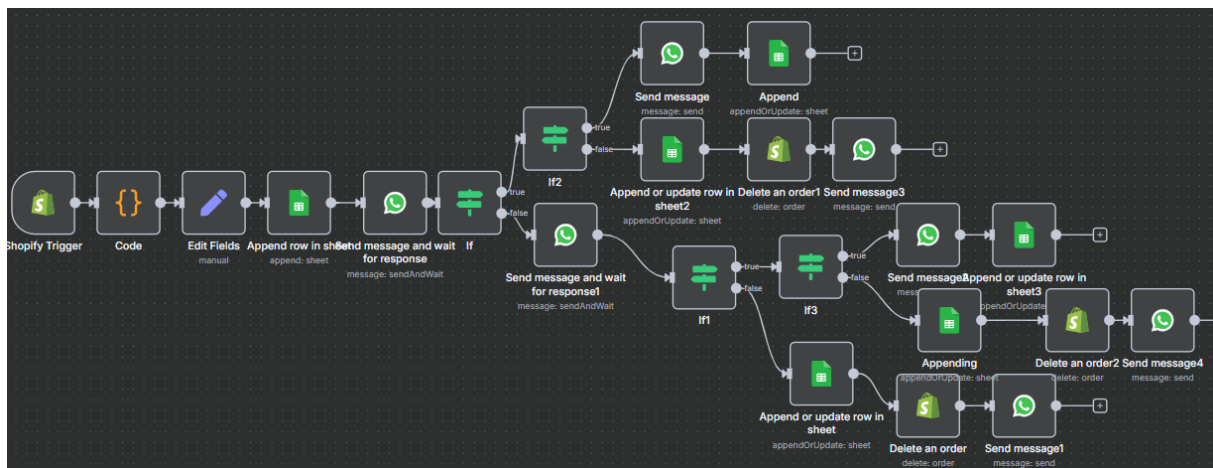


FIG. 7.26 – Workflow n8n pour le processus de confirmation de commande.

Ce processus est entièrement géré par un workflow n8n, qui intègre les API des plateformes e-commerce, des services de communication (Twilio pour les appels/SMS, services WhatsApp Business) et Google Sheets.

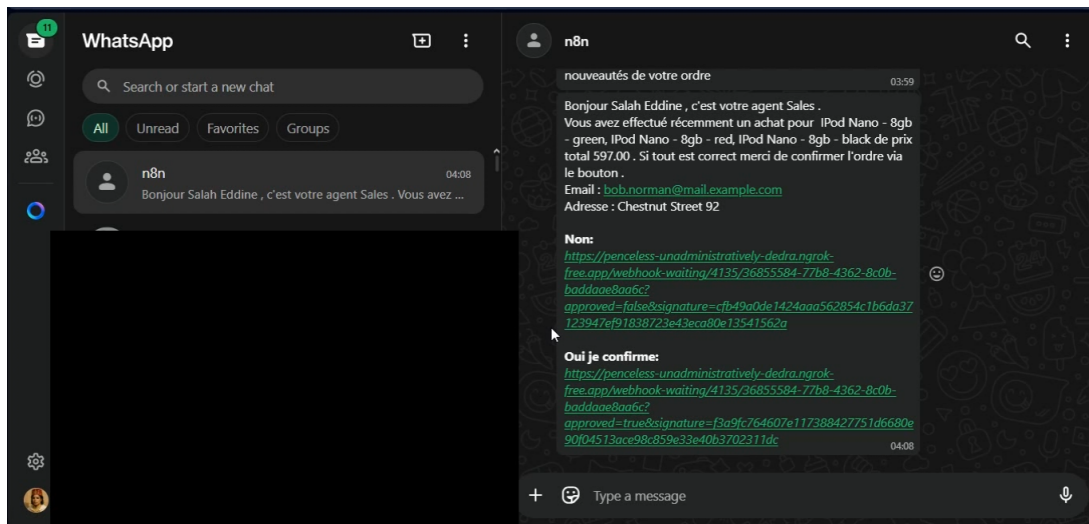


FIG. 7.27 – Exemple de message de confirmation envoyé via WhatsApp.

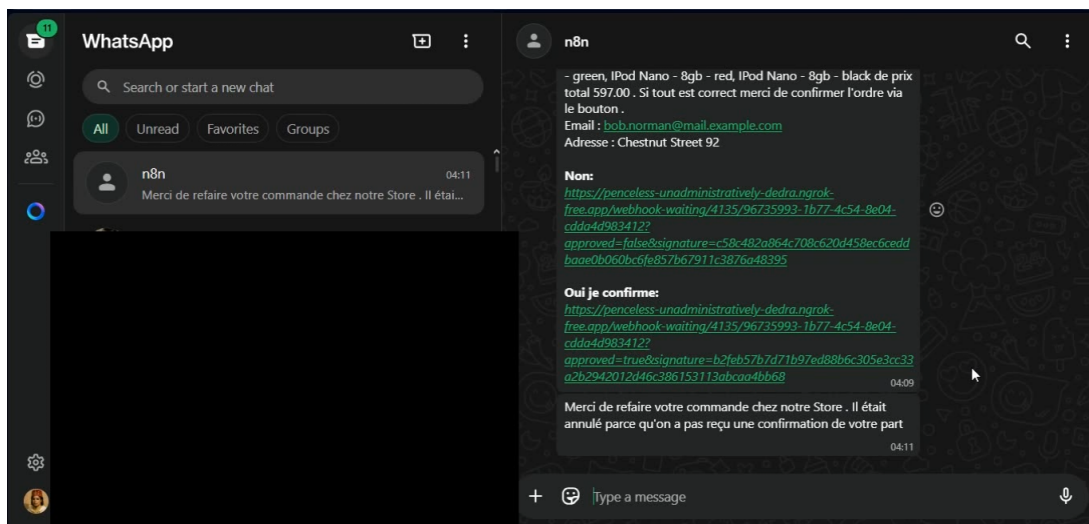


FIG. 7.28 – Exemple de message de relance envoyé via WhatsApp.

Bénéfices Clés

- Augmentation de 20 à 30% du taux de confirmation des commandes COD.
- Réduction significative des appels manqués et engagement accru des clients.
- Libération de temps pour les équipes commerciales, qui peuvent se concentrer sur des tâches à plus haute valeur ajoutée.

7.5.2 Agent de Suivi Actif des Livraisons

Après la confirmation, l'enjeu majeur devient la réussite de la livraison. L'agent de suivi a pour mission d'automatiser la surveillance des colis et de communiquer proactivement avec le client final, afin de réduire les retours et d'améliorer la satisfaction client.

Principe de Fonctionnement

Une fois qu'une commande est expédiée, l'agent se connecte aux API des sociétés de livraison marocaines (par exemple, Amana, Aramex, etc.) pour suivre en temps réel l'avancement de chaque colis. Il agit comme un pont d'information entre le transporteur, l'e-commerçant et le client. Le fonctionnement de cet agent est décrit dans la Figure 7.29.

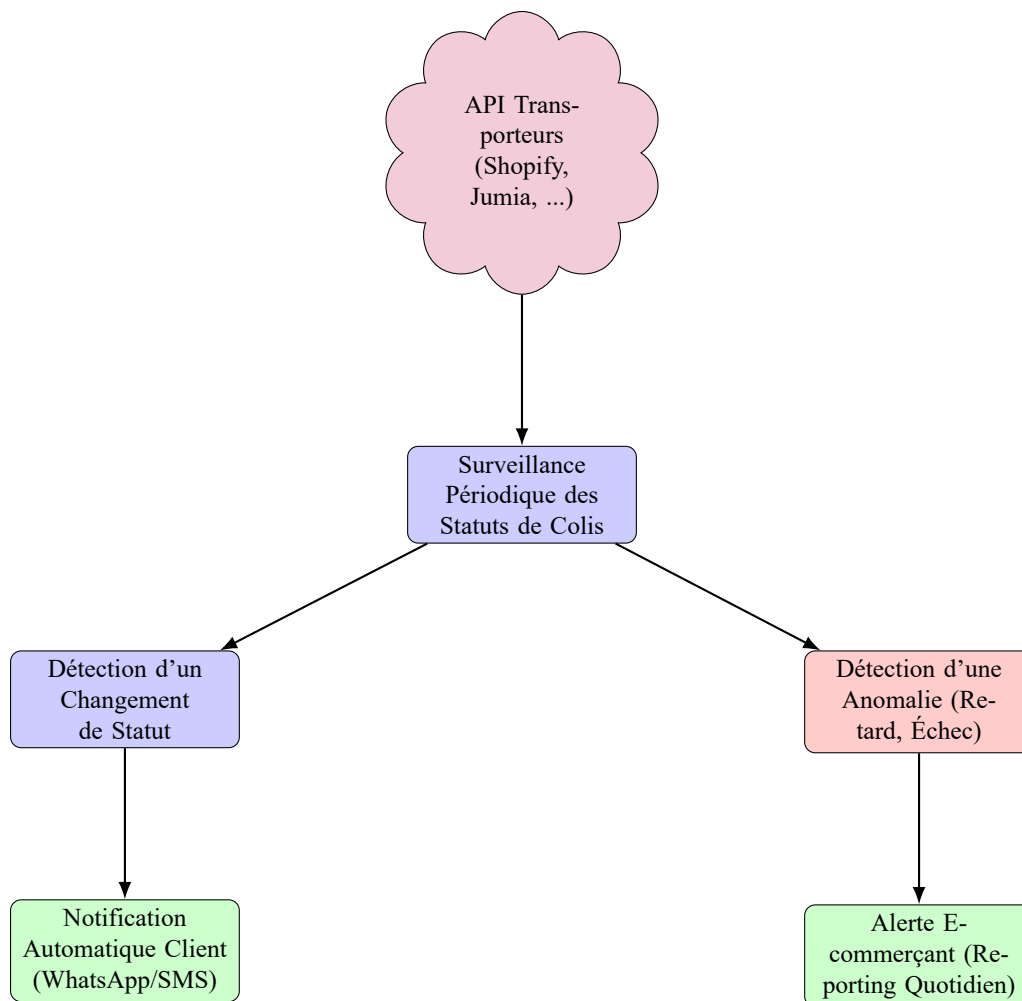


FIG. 7.29 – Diagramme fonctionnel de l'agent de suivi de livraison.

Les fonctionnalités principales sont :

1. **Suivi en Temps Réel** : L'agent interroge périodiquement les API des transporteurs pour récupérer le dernier statut de chaque colis en cours d'expédition.
2. **Notifications Client Proactives** : À chaque étape clé (ramassage, en transit, en cours de livraison), un message WhatsApp ou SMS est automatiquement envoyé au client, l'informant du statut et de la date de livraison estimée.
3. **Gestion des Alertes** : Si une anomalie est détectée (colis bloqué, échec de livraison, retard important), l'agent en informe immédiatement l'e-commerçant via un rapport quotidien, mettant en évidence les commandes nécessitant une intervention manuelle.
4. **Réduction des Retours** : En informant le client à l'avance, l'agent s'assure de sa disponibilité et réduit les risques de retour pour motif "Client non joignable" ou "absent".

En complément des notifications proactives destinées au client final, l'agent met également à disposition de l'e-commerçant une fonctionnalité de consultation *à la demande*. L'e-commerçant peut envoyer un simple message, par exemple via WhatsApp, au numéro de téléphone de l'agent. Ce message doit contenir un mot-clé suivi du numéro de la commande (par exemple, Statut #1052). L'agent interprète instantanément la requête, interroge en temps réel les API des transporteurs pour la commande spécifiée, et renvoie une réponse immédiate avec le dernier statut connu du colis. Ce mécanisme offre un moyen de vérification extrêmement rapide et pratique, sans nécessiter de se connecter à l'interface du transporteur ou au back-office du site.

Mise en Œuvre et Résultats

Comme pour l'agent de confirmation, un workflow n8n orchestre les appels API et la logique de notification. Les communications sont standardisées mais claires, renforçant la confiance du client.

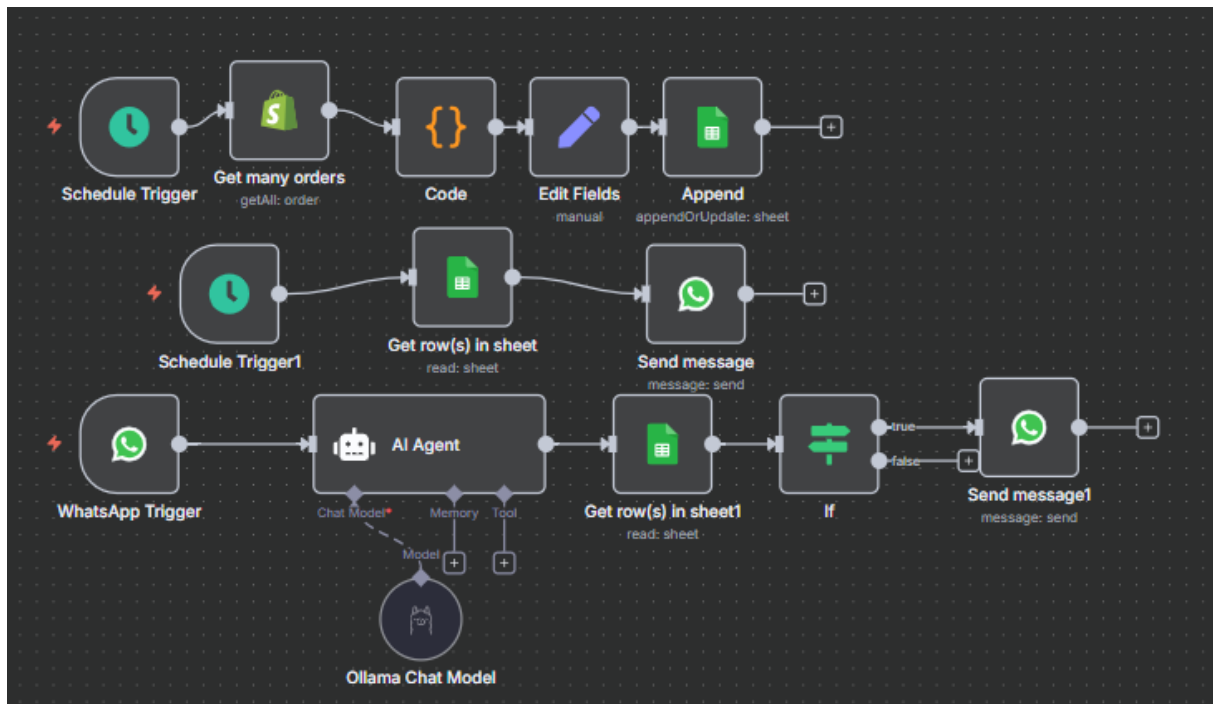


FIG. 7.30 – Vue du workflow n8n pour l’agent de suivi de livraison.

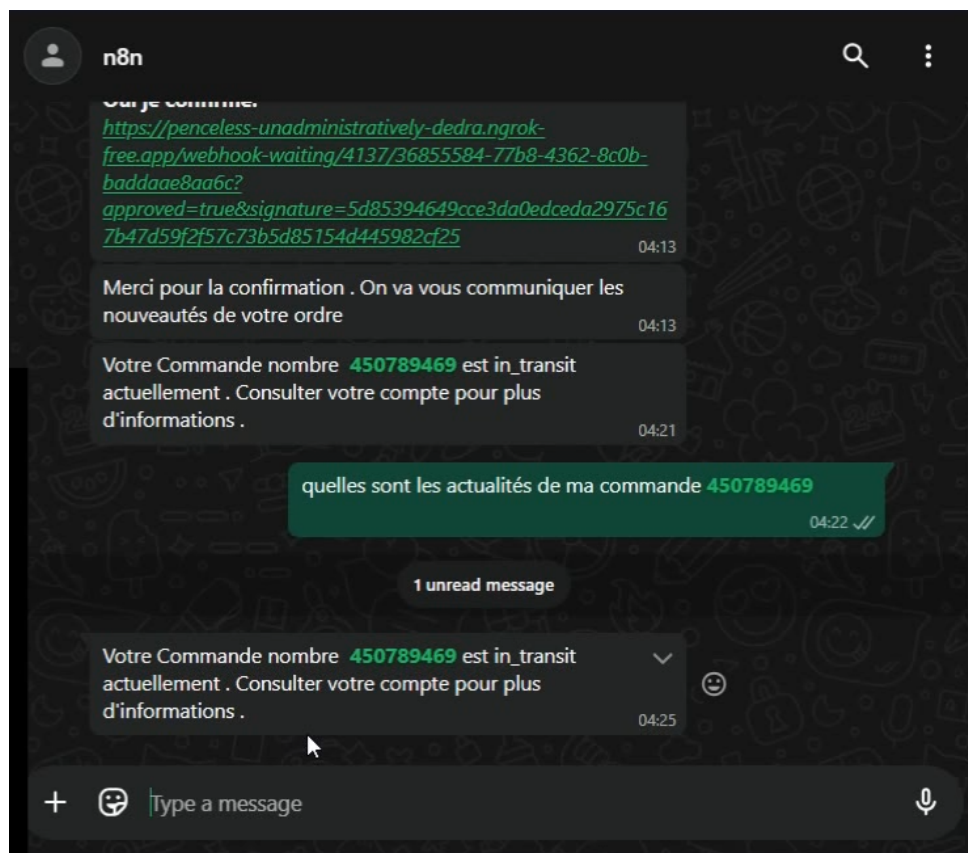


FIG. 7.31 – Exemples de notifications de suivi envoyées au client.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Order Id	Customer	Phone	Status									
2	450789460	Bob Norman	+1(502)-459-218	confirmed									
3	450789469	Salah Eddine	+212643736231	in_transit									
4													
5													
6													
7													
8													
9													
10													
11													
12													
13													
14													
15													
16													
17													
18													
19													
20													
21													

FIG. 7.32 – Extrait du rapport de suivi sur Google Sheets

Bénéfices Clés

- Diminution notable du taux de retour des colis COD.
- Amélioration de l'expérience et de la satisfaction client grâce à une communication transparente.
- Optimisation du temps pour le service client, qui n'a plus à suivre manuellement les colis.

7.6 Plateforme de Gestion Centralisée

Pour orchestrer, superviser et interagir avec l'ensemble des agents créés, j'ai développé une application web centralisée. Cette plateforme agit comme un centre de commandement unique, offrant une vue d'ensemble des opérations, des outils de gestion fine des accès, des tableaux de bord analytiques et des interfaces d'interaction en temps réel. Elle a été conçue pour offrir un contrôle total tout en simplifiant la complexité sous-jacente des workflows d'automatisation.

7.6.1 Architecture Fonctionnelle et Cas d'Utilisation

L'application est structurée autour de deux rôles principaux : l'Administrateur, qui dispose de tous les droits, et l'Utilisateur, qui a un accès restreint aux agents qui lui sont assignés. Le diagramme de cas d'utilisation de la Figure 7.33 illustre les principales fonctionnalités accessibles à chaque rôle.

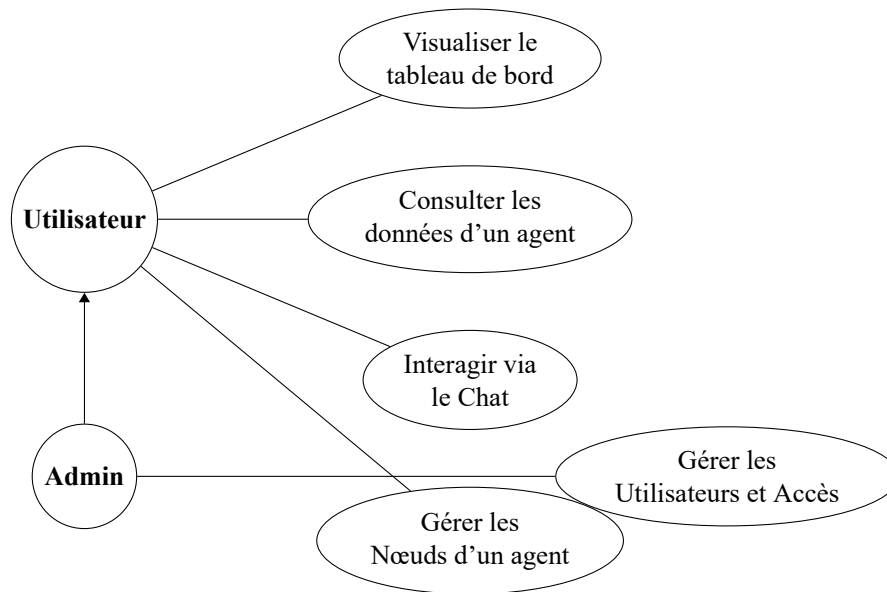


FIG. 7.33 – Diagramme de cas d'utilisation de la plateforme de gestion.

7.6.2 Gestion des Utilisateurs et des Permissions (Accès Admin)

Le tableau de bord Admin de l'Offzone AI System présente une interface sombre avec un header contenant le logo et le nom du système, ainsi qu'un bouton 'Dashboard'. Le titre principal est 'Admin Panel'. Une barre verte 'Add New User' est visible. Le contenu principal est un tableau de gestion des utilisateurs.

USERNAME	ROLE	WORKFLOW ACCESS	
admin@example.com	admin	None	Edit Delete
salaheddine.ouirra-etu@etu.univh2c.ma	user	Agent Commercial	Edit Delete
wireh65350@etpaper.com	admin	ZenDesk Agent	Edit Delete

FIG. 7.34 – Tableau de Bord Admin

Edit User: salaheddine.ouirra-etu@etu.univh2c.ma

Role

User

User

Admin

☐ n8n Law Guide	☐ Slack Agent With LLM
☐ ZenDesk Agent	☐ Extractor
☒ Agent Commercial	☐ DM Agent
☐ Slack Agent	☐ Sales Closer
☐ LogistiTrack	

Cancel Save Changes

FIG. 7.35 – Fenêtre changement des accès utilisateurs

Le contrôle d'accès est une fonctionnalité exclusivement réservée aux administrateurs. Depuis une interface dédiée, ils peuvent :

- **Ajouter ou supprimer des utilisateurs :** Gérer les comptes ayant accès à la plateforme.
- **Attribuer des rôles :** Promouvoir un utilisateur au rang d'administrateur ou révoquer ce statut.
- **Gérer les permissions par agent :** Assigner à chaque utilisateur l'accès à un ou plusieurs agents spécifiques. Cela permet de cloisonner les responsabilités, un commercial n'ayant par exemple accès qu'à l'agent de prospection.

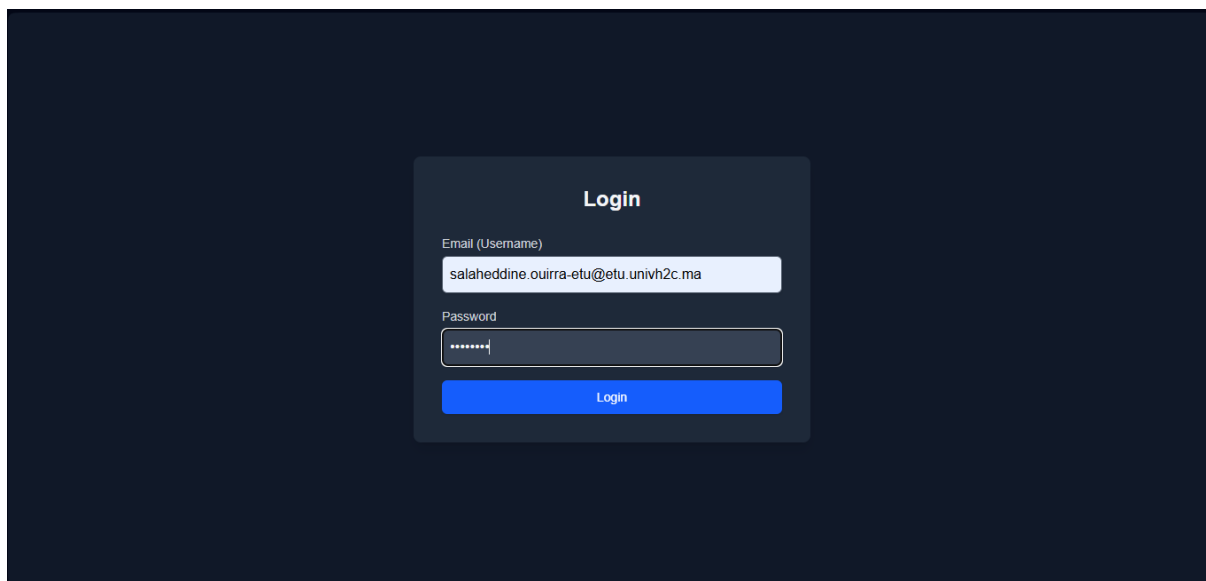


FIG. 7.36 – Page d’authentification au système

7.6.3 Tableau de Bord : Suivi des Performances et des Erreurs

La page d’accueil de la plateforme est un tableau de bord analytique qui offre une vision macroscopique de l’activité de l’ensemble des agents. Il est composé de trois graphiques principaux :

- **Historique des Exécutions** : Un diagramme en barres compare le nombre d’exécutions réussies et échouées sur une période donnée, permettant d’évaluer la fiabilité globale du système.

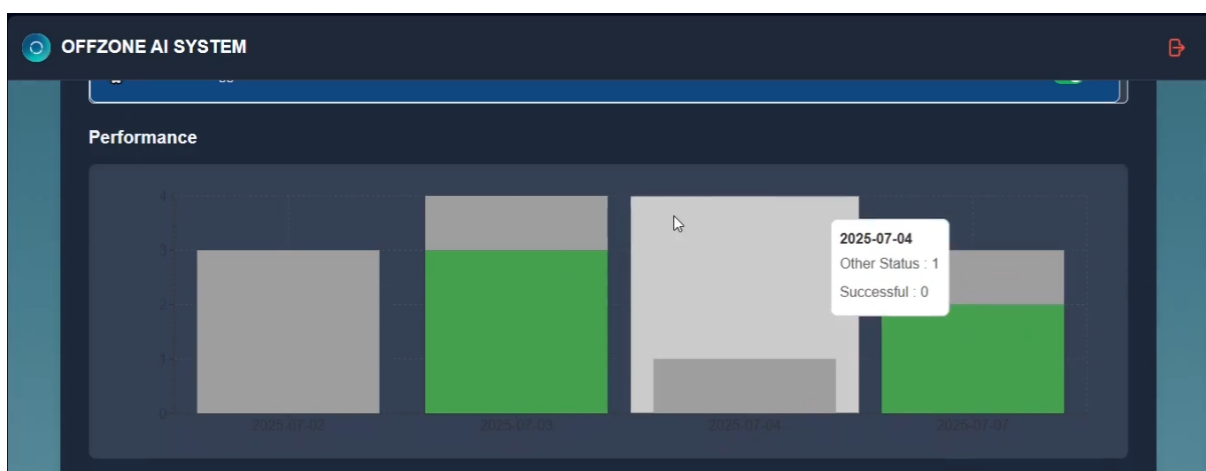


FIG. 7.37 – Bar Graph pour l’historique des exécutions

- **Volume de Données Traitées :** Une courbe de tendance montre l'évolution du nombre de données retournées par les agents, offrant un aperçu de leur charge de travail et de leur efficacité.

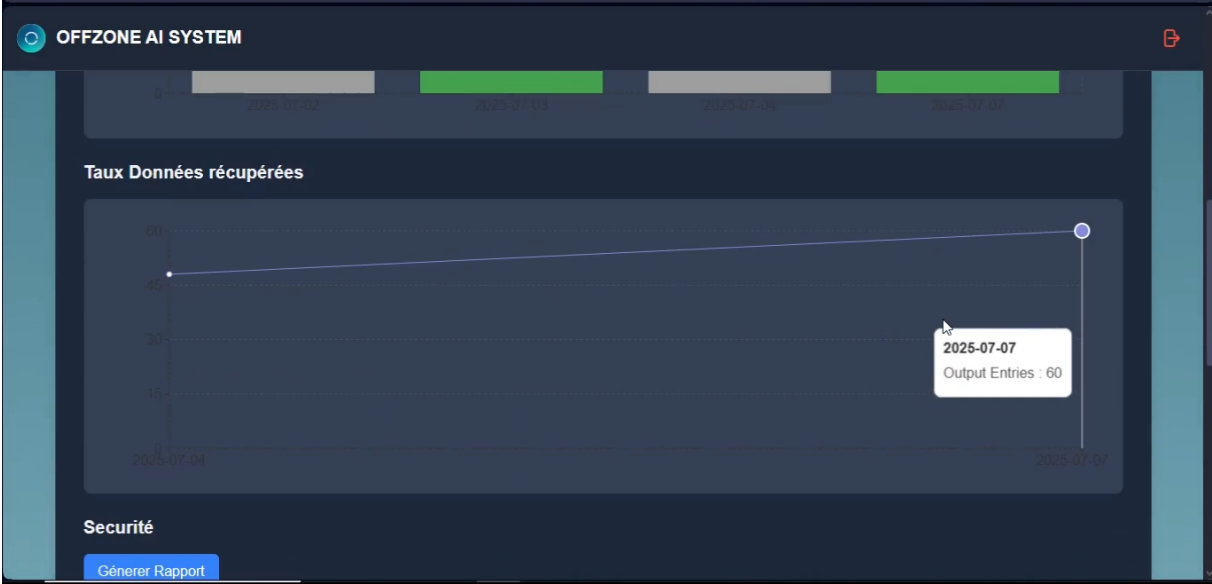


FIG. 7.38 – Line Graph pour le taux des données récupérées par l'agent

- **Analyse des Vulnérabilités :** Un diagramme circulaire répartit les erreurs ou les échecs par agent, aidant à prioriser les efforts de maintenance et de correction sur les composants les plus fragiles.

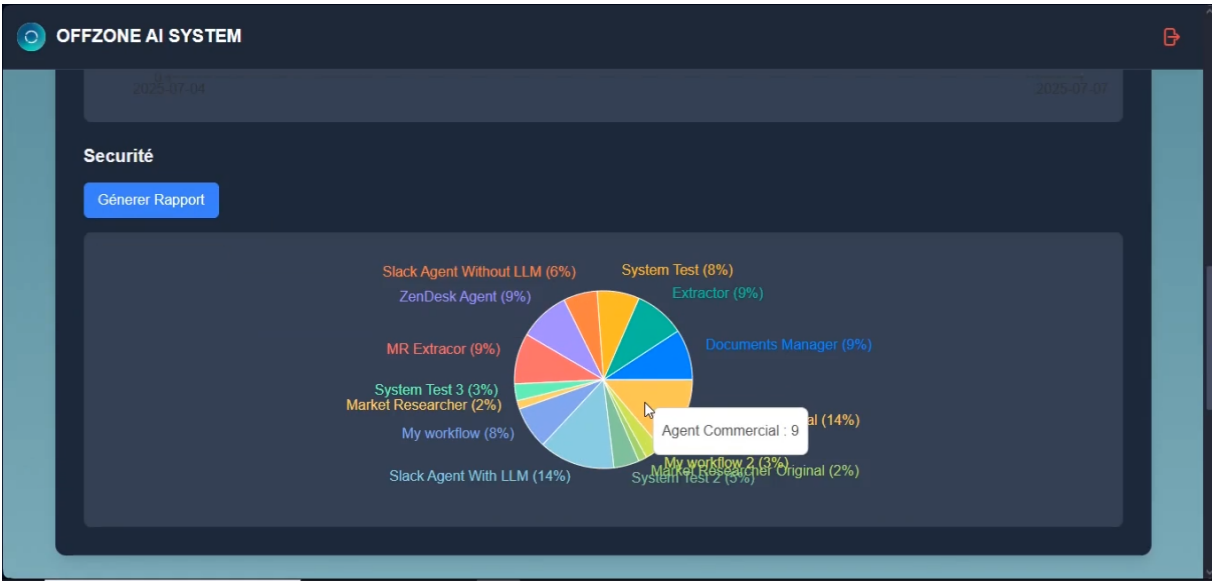


FIG. 7.39 – Pie Graph pour le suivi des vulnérabilités

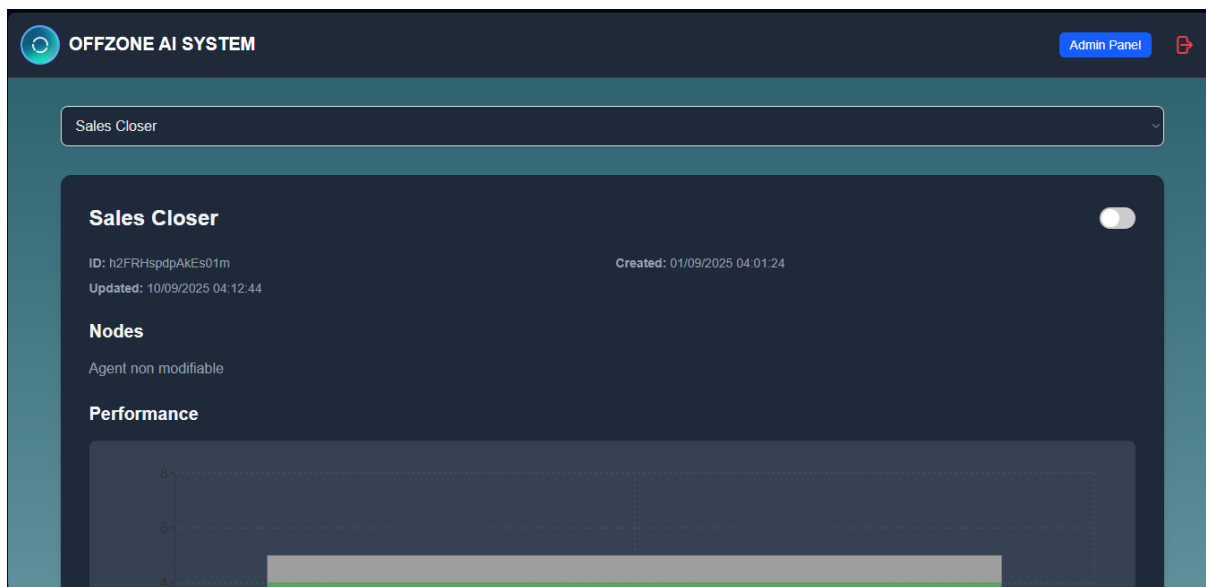


FIG. 7.40 – Vue du tableau de bord principal

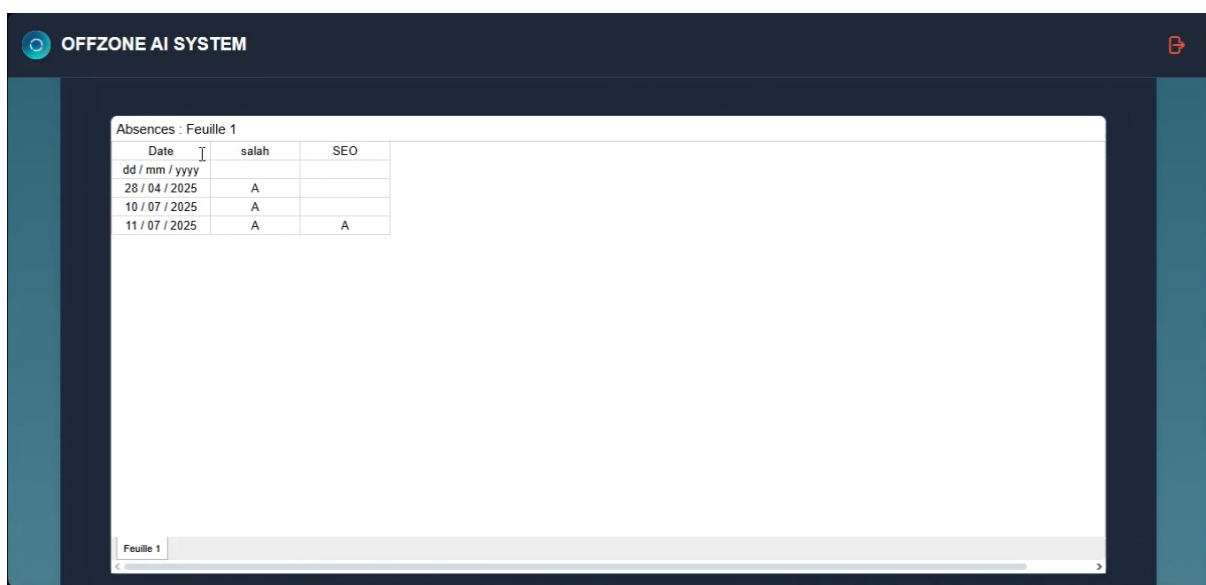


FIG. 7.41 – Log des absences directement dans la plateforme

7.6.4 Supervision des Agents en Temps Réel et Historique

Chaque agent dispose de sa propre page de supervision, offrant une double perspective :

- **Flux de Données en Temps Réel** : Grâce à une connexion avec **Redis**, l'interface affiche en direct les nouvelles données dès leur réception par l'agent (un nouveau prospect, un nouveau ticket, etc.). Cela permet une supervision active et immédiate des opérations.

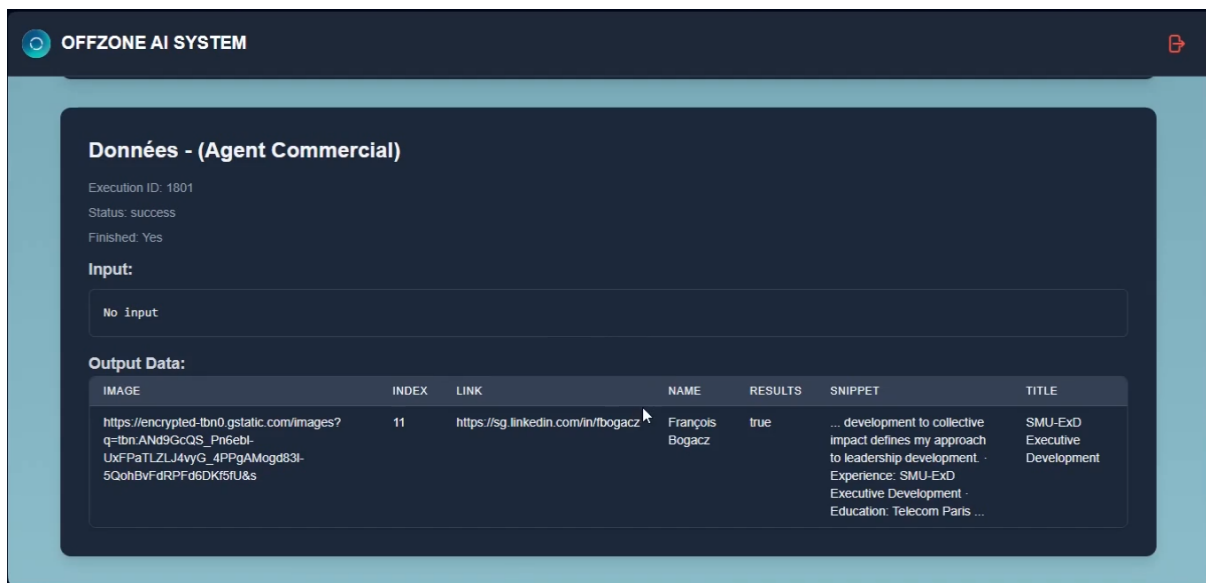


FIG. 7.42 – Données suivi en temps réel avec Redis

- **Historique des Données :** Toutes les données traitées sont persistées dans une base de données **MongoDB**. La plateforme permet de consulter, filtrer et rechercher dans cet historique, ce qui est essentiel pour l'analyse a posteriori et le débogage.

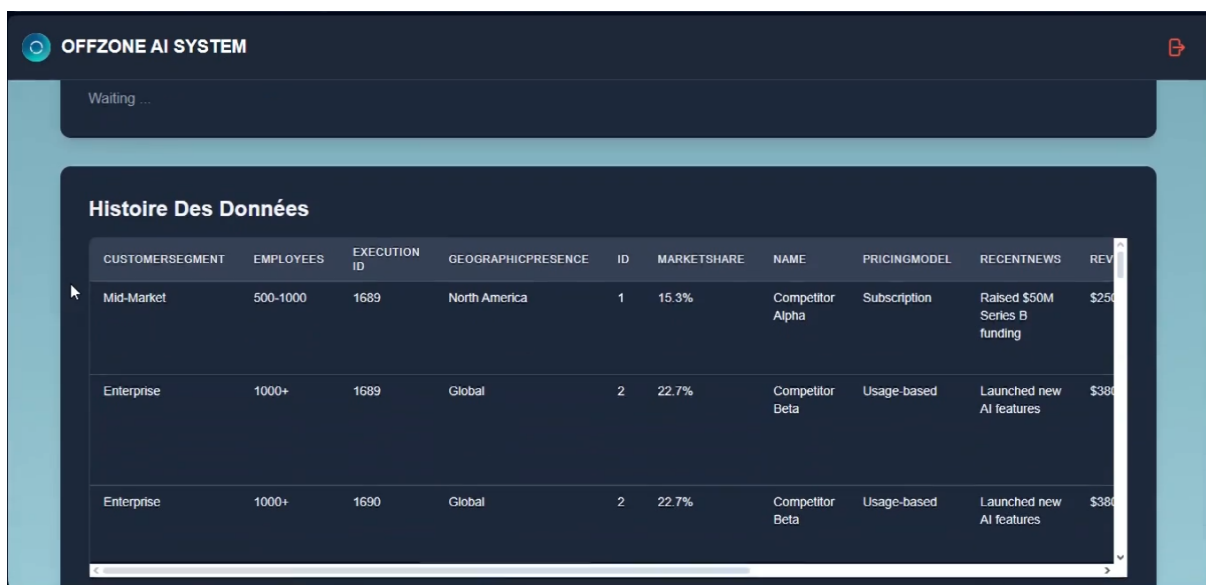


FIG. 7.43 – Histoire des données récupérées par l'agent

- **Gestion des Nœuds du Workflow :** L'utilisateur autorisé peut interagir avec la logique de l'agent. L'interface présente une vue simplifiée du workflow, exposant uniquement les nœuds dont la modification est sécuritaire (par exemple, un nœud de notification).

L'utilisateur peut alors activer, désactiver ou même supprimer ces nœuds pour ajuster le comportement de l'agent sans risque de corrompre le flux principal.

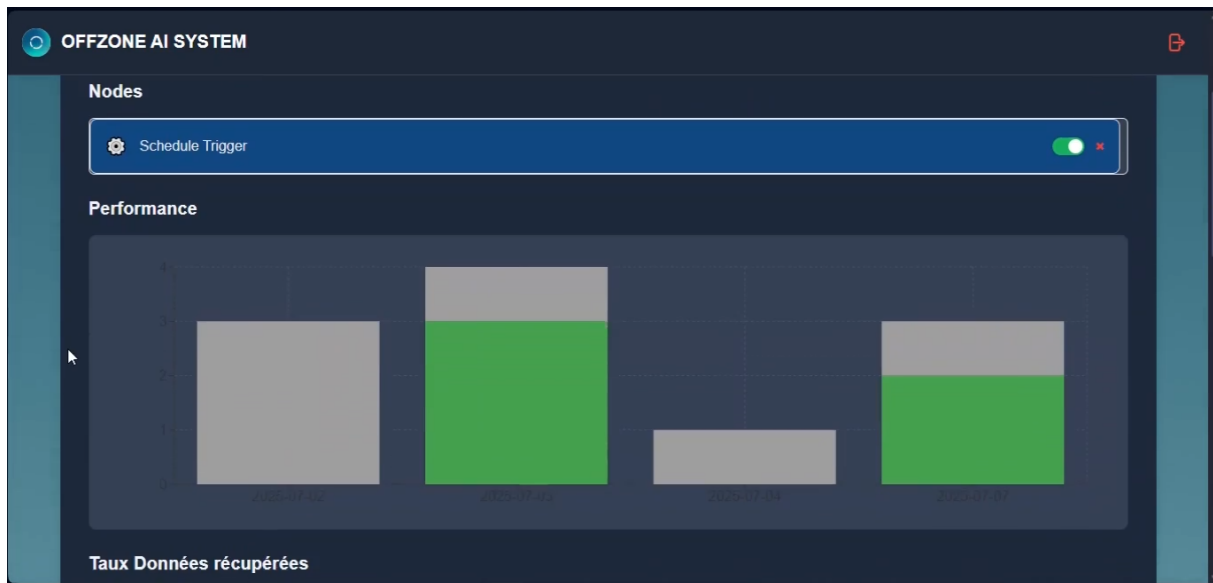


FIG. 7.44 – Contrôle sur les nœuds des workflows n8n depuis l'application

7.6.5 Interaction Conversationnelle avec les Agents

Pour offrir une méthode de contrôle plus intuitive, une page de "Chat" est disponible. Cette interface permet à un utilisateur de converser en langage naturel avec les agents auxquels il a accès. Par exemple, un commercial peut écrire :

```
"Lance une nouvelle recherche de prospects pour les  
directeurs marketing  
à Paris dans le secteur de la fintech."
```

L'agent commercial, via le LLM, interprète cette instruction, la traduit en paramètres structurés et démarre le workflow de prospection correspondant. Il confirme ensuite l'action dans le chat avec un message tel que : "Compris. Lancement d'une nouvelle campagne de prospection avec les paramètres fournis." Cette fonctionnalité transforme la gestion des agents d'une tâche de configuration à une simple conversation.

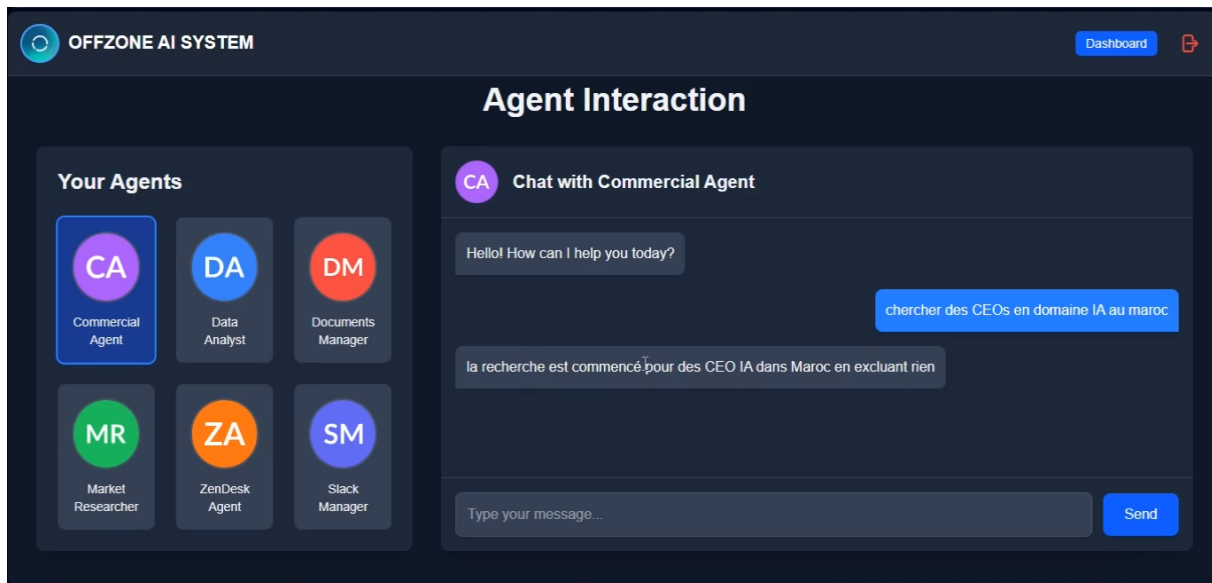


FIG. 7.45 – Exemple d’interaction avec l’agent commercial via l’interface de chat.

7.7 D  fis Rencontr  s et Solutions Apport  es

Le d  veloppement d’un syst  me aussi complexe a pr  sent   plusieurs d  fis techniques et organisationnels.

7.7.1 Tests et Validation

- **Probl  me :** La nature non-d  terministe des LLMs et la complexit   des interactions entre agents ont rendu les tests exhaustifs difficiles. Il   tait compliqu   de simuler toutes les conditions r  elles d’utilisation.
- **Approche de Solution :** Une strat  gie de test    plusieurs niveaux a   t   mise en place, combinant des tests unitaires pour la logique m  tier, des tests d’int  gration pour les interactions entre composants, et des tests de validation manuels bas  s sur des sc  narios utilisateurs pr  cis pour   valuer la pertinence des r  ponses des agents.

7.7.2 Optimisation des Performances

- **Problème :** Les appels aux LLMs peuvent introduire une latence significative. Assurer que le système puisse traiter les données et prendre des décisions en temps réel sans délai perceptible était un défi majeur.
- **Approche de Solution :** Plusieurs optimisations ont été implémentées : utilisation intensive du cache avec **Redis** pour les requêtes fréquentes, quantification des modèles de langage pour réduire leur taille et accélérer l'inférence, et traitement asynchrone des tâches non-bloquantes.

Conclusion Générale et Perspectives

7.8 Bilan du Projet et Synthèse des Apports

Au terme de ce projet, nous avons démontré la faisabilité et la pertinence de l'implémentation d'un **système multi-agents intelligent** pour l'automatisation des processus métier. En nous appuyant sur un écosystème technologique moderne combinant des frameworks comme **N8N** [10], la puissance des LLMs avec **Llama 3.1** [8], et une architecture robuste basée sur **FastAPI**, nous avons jeté les bases d'une solution à la fois performante et scalable.

Le développement de cas d'usage concrets, notamment l'agent e-commerce, a permis de valider l'approche et de matérialiser les gains potentiels en termes d'efficacité opérationnelle, d'amélioration de l'expérience client et de réduction des coûts. Ce travail ne se limite pas à une simple preuve de concept ; il constitue une fondation solide sur laquelle des innovations futures peuvent être construites.

7.9 Perspectives d'Évolution et Travaux Futurs

Les possibilités d'évolution de ce système sont vastes. Nous les organisons ici en trois horizons temporels, allant des améliorations directes aux visions les plus ambitieuses.

7.9.1 Court Terme : Consolidation et Améliorations Incrémentales

- **Industrialisation du Déploiement** : Mettre en place une chaîne d'intégration et de déploiement continu (CI/CD) pour automatiser les mises à jour. Déployer le système sur une infrastructure cloud scalable comme Kubernetes et mettre en place des outils de mo-

nitoring (observabilité) pour suivre la performance et la santé des agents en temps réel.

- **Agent Proactif et Anticipation des Besoins** : Intégrer des modèles de Machine Learning, comme proposé, non seulement pour prédire les recherches, mais pour anticiper les besoins des clients. Par exemple, un agent pourrait détecter un retard potentiel de livraison et notifier proactivement le client avant même qu’il ne pose la question.

7.9.2 Moyen Terme : Extensions Stratégiques

- **Agents Multi-Modaux** : Faire évoluer les agents au-delà du texte en leur donnant la capacité de **comprendre et de générer des contenus multi-modaux**. Un agent de support pourrait analyser une image envoyée par un client pour diagnostiquer un problème, ou un agent marketing pourrait générer des visuels de produits à la volée.
- **Intelligence Émotionnelle et Analyse de Sentiments** : Intégrer des capacités d’analyse de sentiments en temps réel. Les agents pourraient ainsi détecter le niveau de satisfaction ou de frustration d’un client dans une conversation et adapter dynamiquement leur ton, leur niveau d’empathie, ou décider de transférer l’échange à un humain si la tension est trop élevée.

7.9.3 Long Terme : Visions d’Avenir et Potentiel de Rupture

- **Le Jumeau Numérique Organisationnel (*Organizational Digital Twin*)** : Transformer le système multi-agents en une réplique numérique dynamique de l’entreprise. Ce ”jumeau” ne se contenterait pas d’automatiser les tâches existantes ; il pourrait **simuler l’impact de décisions stratégiques avant leur mise en œuvre**. Par exemple : ”Quelles seraient les conséquences sur la charge du support client si nous lançons cette nouvelle fonctionnalité?”. Le système pourrait exécuter une simulation et fournir des prédictions chiffrées.
- **Négociation Autonome Inter-Entreprises** : Doter les agents de la capacité d’agir au-delà des frontières de l’entreprise. Un agent logistique pourrait, de manière autonome,

négozier avec les agents de différents fournisseurs pour réapprovisionner les stocks, en optimisant en temps réel les coûts, les délais et la qualité, sans aucune intervention humaine.

En conclusion, si les agents logiciels sont aujourd'hui de puissants outils d'automatisation, les perspectives d'avenir les positionnent comme de véritables partenaires cognitifs. Le passage de l'automatisation de tâches à la simulation stratégique et à l'optimisation autonome représente la prochaine frontière, promettant de redéfinir en profondeur la manière dont les entreprises opèrent, innovent et créent de la valeur.

Bibliographie

- [1] VASWANI, Ashish, et al. Attention is all you need. In : *Advances in neural information processing systems*, 2017, vol. 30.
- [2] LEWIS, Patrick, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In : *Advances in Neural Information Processing Systems*, 2020, vol. 33, p. 9459-9474.
- [3] WOOLDRIDGE, Michael. *An introduction to multiagent systems*. 2ème édition. John Wiley & Sons, 2009.
- [4] GEORGEFF, Michael P., et al. Belief-desire-intention model of agency. In : *Foundations of rational agency*. Springer, Dordrecht, 1999. p. 1-46.
- [5] XI, Zhen-Yang, et al. The Rise and Potential of Large Language Model Based Agents : A Survey. *arXiv preprint arXiv :2309.07864*, 2023.
- [6] PARK, Joon Sung, et al. Generative Agents : Interactive Simulacra of Human Behavior. *arXiv preprint arXiv :2304.03442*, 2023.
- [7] QIAN, Chen, et al. Communicative Agents for Software Development. *arXiv preprint arXiv :2307.07924*, 2023. (Projet CAMEL).
- [8] META AI. The Llama 3 Herd of Models. *Blog de Meta AI*, 2024. Disponible sur : <https://ai.meta.com/blog/meta-llama-3/>

- [9] CHASE, Harrison, et al. *LangChain : Building applications with LLMs through composability*. GitHub repository, 2022. Disponible sur : <https://github.com/langchain-ai/langchain>
- [10] OBERHAUSER, Jan, et al. *n8n : A source-available framework for workflow automation*. Plateforme en ligne, 2019. Disponible sur : <https://n8n.io/>
- [11] MOURA, Joao, et al. *CrewAI : A framework for orchestrating role-playing, autonomous AI agents*. GitHub repository, 2023. Disponible sur : <https://github.com/joaoandmoura/crewAI>
- [12] RAMÍREZ, Sebastián. *FastAPI : A modern, fast web framework for building APIs with Python*. GitHub repository, 2018. Disponible sur : <https://github.com/tiangolo/fastapi>
- [13] VERCEL INC. *Next.js : The React Framework for the Web*. Site officiel, 2016. Disponible sur : <https://nextjs.org/>
- [14] QDRANT TEAM. *Qdrant - Vector Database & Vector Search Engine*. Site officiel, 2020. Disponible sur : <https://qdrant.tech/>